

A media company is building an AI-powered content moderation system by using an FM in Amazon Bedrock. The system requires immediate detection and prevention of sensitive information and policy violations. Text and image content should not contain personally identifiable information (PII), misinformation, hate speech, and unsafe content. The solution must stop text and image content that violates these policies before the content reaches the editorial review process.

The company's compliance framework requires comprehensive documentation of FM limitations and biases with proper version control. Additionally, the policy requires event-driven monitoring that invokes automated compliance validation workflows within seconds of guardrail intervention.

Which solution will meet these requirements with the LEAST operational overhead?

A

Configure Amazon Bedrock Guardrails with content filters for PII, misinformation, hate speech, and unsafe multimodal content. Document FM biases and limitations in standardized model cards with versioning enabled in Amazon S3. Configure an Amazon CloudWatch alarm to monitor the `InvocationsIntervened` Guardrails metric. Create an Amazon EventBridge rule that invokes an AWS Lambda compliance workflow when the alarm is triggered.

Correct. Guardrails provide built-in content filtering capabilities for text and multimodal content. Guardrails provide immediate detection and prevention of policy violations with minimal setup. This solution uses Amazon S3 for version-controlled model card documentation. This solution uses CloudWatch to monitor the built-in Guardrails metric `InvocationsIntervened`. This solution uses EventBridge with Lambda for automated compliance workflows. Therefore, this solution minimizes operational overhead by using built-in service features and integrations.

Learn more about [Guardrails](#).

Learn more about [CloudWatch metrics for Guardrails](#).

B

Configure Amazon Bedrock Guardrails with content filters for PII, misinformation, hate speech, and unsafe multimodal content. Store model card documentation in Amazon S3 with lifecycle policies enabled. Create Amazon EventBridge rules to invoke AWS Lambda functions that parse AWS CloudTrail API logs for policy violations.

Incorrect. Guardrails provide built-in content filtering. However, parsing CloudTrail logs for policy violations increases operational overhead and creates potential delays.

Additionally, S3 Lifecycle policies do not provide the comprehensive version control that you need for model documentation.

Learn more about [CloudTrail monitoring](#).

C

Set up Amazon Comprehend for PII detection and sentiment analysis on user-generated content. Pass the content to Amazon Bedrock. Configure Amazon Bedrock Guardrails with content filters for misinformation, hate speech, and unsafe multimodal content. Maintain comprehensive model governance documentation with automated versioning in Amazon DynamoDB with global secondary indexes. Create AWS Lambda functions with Amazon EventBridge to perform continuous compliance monitoring and generate detailed violation reports.

Incorrect. Amazon Comprehend creates redundant PII detection capabilities when guardrails already provide integrated sensitive information filtering. DynamoDB does not provide the built-in document versioning capabilities that you need for proper model card governance, even with global secondary indexes. EventBridge provides periodic monitoring rather than event-driven, real-time monitoring. Therefore, this solution does not meet the immediate response requirement.

Learn more about [Amazon Comprehend](#).

Learn more about [DynamoDB constraints and limitations](#).

D

Configure Amazon Bedrock Guardrails with content filters for PII, misinformation, hate speech, and unsafe multimodal content. Integrate Amazon Rekognition content moderation with custom trained models for enhanced visual content analysis and bulk processing capabilities. Document FM capabilities and limitations in standardized model cards with automated versioning in Amazon S3. Create AWS Lambda functions with Amazon EventBridge integration to orchestrate parallel processing workflows between Amazon Bedrock and Amazon Rekognition APIs.

Incorrect. This solution creates unnecessary service redundancy that violates the immediate response requirement. Guardrails support multimodal toxicity detection for images. Therefore, the integration with Amazon Rekognition is redundant for the same content moderation tasks. The proposed parallel processing workflows between Amazon Bedrock and Amazon Rekognition APIs would introduce significant latency through multiple API calls. Additionally, this solution requires complex orchestration logic and prevents the system from meeting the requirement for immediate content blocking. This solution doubles processing costs by running two content moderation

services on identical content. The orchestration complexity introduces multiple failure points that compromise system reliability.

Learn more about [Amazon Rekognition](#).

Learn more about [Guardrails](#).

2/75

A company wants to enhance a customer support AI assistant with up-to-date product information. A GenAI developer must implement a custom RAG solution by using Amazon Bedrock and Amazon OpenSearch Service. The GenAI developer needs to maintain complex relationships between product variants. The GenAI developer must implement hybrid search that combines semantic matching and keyword matching.

Select and order each step from the following list to implement the RAG solution. Select each step one time. (Select and order FOUR.)

- Create initial embeddings and configure automated embedding generation for product updates.
- Deploy an FM and develop a prompt template with context retrieval.
- Implement vector search in OpenSearch Service with hybrid search capabilities.
- Set up a data ingestion pipeline to process and update product information.

Step 1:

Set up a data ingestion pipeline to process and update product information.

Step 2:

Create initial embeddings and configure automated embedding generation for product updates.

Step 3:

Implement vector search in OpenSearch Service with hybrid search capabilities.

Step 4:

Deploy an FM and develop a prompt template with context retrieval.

This use case requires a strict sequence because of component dependencies. You must test and verify the solution at each phase.

First, you must set up the data ingestion pipeline. This step establishes the foundation for all subsequent steps. You must test and verify the pipeline to ensure proper processing and formatting of the product information. This step is necessary so that you can implement subsequent components effectively.

Second, you must create initial embeddings and configure automated embedding generation. This step requires processed data from the verified pipeline. This step ensures that the initial embedding creation and ongoing updates work correctly before you proceed. The automation configuration occurs with the initial creation because of the shared common components and testing requirements.

Third, you must implement vector search. This step requires the previous components of a verified data pipeline and functional embedding generation. You must test the hybrid search capabilities with actual embeddings. You must verify the embeddings for both semantic matching and keyword matching functionality before the final phase.

Finally, you must deploy the FM and develop the prompt template. All previous components must be operational and verified to make this step possible. This step depends on all previous components being operational and verified. You must test the prompt template with the working vector search to ensure proper context retrieval and integration.

Learn more about [how to build a RAG application by using Amazon Bedrock](#).

Learn more about [vector search](#).

Learn more about [embeddings](#).

3/75

A large financial services company needs to implement a document processing solution by using generative AI (GenAI). The solution must process mortgage applications that contain sensitive customer information. Each application takes 5–15 minutes to process completely. Each application requires multiple FM calls for information extraction, analysis, and summarization. The solution must handle up to 1,000 concurrent requests during peak hours. The solution must provide audit trails for all AI model interactions. After processing, the results should be available for users to retrieve.

Which solution will meet these requirements with the LEAST operational overhead?

A

Create an Amazon API Gateway API with AWS Lambda functions that make synchronous calls to Amazon Bedrock for document processing. Store results in Amazon DynamoDB after processing.

Incorrect. API Gateway is a fully managed service that you can use to create and manage APIs. API Gateway has a 29-second timeout quota. Lambda functions have a 15-minute timeout quota. These quotas make synchronous processing unsuitable for documents that take 5–15 minutes to process. This solution would fail for longer-running processes. This solution does not efficiently handle concurrent requests during peak hours.

Learn more about [API Gateway quotas](#).

Learn more about [Lambda quotas](#).

B

Use Amazon S3 to store document uploads. Set up S3 Event Notifications to invoke an AWS Lambda function. Configure the Lambda function to invoke an AWS Step Functions standard workflow that orchestrates Amazon Bedrock model calls. Store processing status and results in Amazon DynamoDB for retrieval.

Correct. Amazon S3 provides scalable, secure object storage for documents. S3 Event Notifications can invoke an AWS Lambda function. Step Functions standard workflows can run for up to 1 year and maintain detailed execution histories for auditing. Step Functions can handle the orchestration of multiple Amazon Bedrock model calls. DynamoDB provides fast, consistent access to processing status and results. This solution uses fully managed services that provide concurrent processing, audit trails, and result retrieval. Therefore, this solution requires the least operational overhead.

Learn more about [Step Functions workflow types](#).

Learn more about [Step Functions service integration](#).

C

Create an Amazon Kinesis data stream to receive documents. Use Kinesis consumer AWS Lambda functions to process documents in parallel through Amazon Bedrock APIs. Store results in Amazon OpenSearch Service for retrieval.

Incorrect. Amazon Kinesis Data Streams provides real-time streaming data ingestion. Data streams can scale to handle concurrent requests. For this solution, you must manage stream sharding, monitor Lambda consumer functions, and maintain an

OpenSearch Service cluster. You must manage processing state and audit trails. Therefore, this solution requires additional operational overhead.

Learn more about [Kinesis Data Streams](#).

Learn more about [Lambda and Kinesis Data Streams](#).

D

Configure Amazon API Gateway with WebSocket APIs to maintain connections while Amazon Bedrock processes documents. Create an AWS Lambda function to send status updates to connected clients.

Incorrect. API Gateway WebSocket APIs provide two-way communication between clients and servers. However, maintaining connections for 5–15 minutes for each document is resource-intensive. WebSocket connections can remain open for up to 2 hours. However, managing thousands of concurrent long-lived connections increases operational overhead. You must handle connection state, implement reconnection logic, and manage client notifications.

Learn more about [WebSocket APIs](#).

Learn more about [WebSocket API quotas](#).

4/75

A medical company that operates multiple clinics runs a generative AI (GenAI) application on AWS. The application uses AWS Step Functions to orchestrate two AWS Lambda functions. One function calls Amazon Transcribe Medical to transcribe clinic audio data. The second function uses the Amazon Nova Pro model in Amazon Bedrock to summarize the data.

The company is onboarding additional clinics. Each clinic has a unique clinic ID. A GenAI developer must modify the architecture to store each clinic's data in a shared Amazon S3 bucket. The GenAI developer must use the clinic ID as the key prefix. The solution must track summarization costs for each clinic.

Which combination of steps will meet these requirements MOST cost-effectively? (Select TWO.)

A

Configure the S3 bucket to use Amazon S3 Event Notifications to capture PutObject events. Create a Lambda function. Set the function as the destination for the S3 notifications. Configure the function to invoke the Step Functions workflow when new data uploads.

Incorrect. S3 Event Notifications provide data upload notifications. However, S3 Event Notifications cannot directly invoke Step Functions. This step requires an additional Lambda function as an intermediary. This step introduces extra cost because each Lambda invocation is billed by the request and execution time.

Learn more about [S3 Event Notifications](#).

Learn more about [S3 Event Notifications targets](#).

B

Enable S3 Storage Lens for the bucket to collect prefix-level usage metrics for each clinic. Use Amazon Athena to query the metrics, calculate summarization costs, and generate clinic reports.

Incorrect. S3 Storage Lens provides metrics on storage usage. For example, the metrics include object count, object size, request activity, and prefix-level storage trends. These metrics can help analyze how each clinic uses S3 storage space. However, these metrics do not record or attribute inference usage or costs that occur in Amazon Bedrock or Lambda. Querying S3 Storage Lens data by using Athena provides storage-related insights, not operational or billing data. Therefore, this solution cannot track summarization costs for each clinic.

Learn more about [S3 Storage Lens](#).

C

Create an Amazon Bedrock inference profile for each clinic ID. Modify the summarization Lambda function to use the profiles based on the S3 key prefix from the uploaded data.

Correct. Amazon Bedrock application inference profiles are specifically designed to manage and track FM costs in multi-tenant environments. This step efficiently handles cost attribution. The summarization Lambda can select the appropriate profile based on the clinic ID from the S3 key prefix. This step achieves per-clinic cost tracking with minimal additional configuration logic.

Learn more about [Amazon Bedrock application inference profiles](#).

D

Deploy an Amazon SageMaker AI endpoint for each clinic that is tagged with the clinic ID. Route summarization Lambda requests to each endpoint based on the S3 key prefix from the uploaded data.

Incorrect. Amazon Bedrock is a serverless service that provides pay-as-you-go pricing. The endpoints would be billed based on the hour. You can tag SageMaker AI endpoints to track costs and to manage multiple endpoints, scaling policies, and compute

resources. However, deploying a separate SageMaker AI endpoint for each clinic would introduce continuous infrastructure cost. Each endpoint incurs hourly charges even when idle.

Learn more about [SageMaker AI endpoints](#).

E

Create an Amazon EventBridge rule to capture PutObject events. Set the Step Functions state machine as the destination when a matching event occurs on the event bus.

Correct. EventBridge provides serverless event routing that directly integrates with Step Functions. EventBridge can consume S3 events. EventBridge supports advanced prefix-based filtering. Therefore, this step provides clinic-specific routing without additional code. This step minimizes cost by eliminating intermediary Lambda functions (no per-invocation charges) and by providing direct state machine triggering. EventBridge pricing is based on the number of published events and matched rules. Therefore, this step provides pricing that is far lower than maintaining and invoking a Lambda for every event.

Learn more about [EventBridge rules](#).

Learn more about [EventBridge with Step Functions as the target](#).

5/75

A global investment company wants to use Amazon Bedrock to build a generative AI (GenAI) powered conversational assistant. The assistant needs to perform multiple tasks, including research and calculations. The assistant must analyze market data, process the latest news in the financial market, perform calculations, and generate investment insights.

Which solution will meet these requirements?

A Fine-tune an FM by using the market data, news in the financial market, and past analysis output as examples. In the context of a conversational flow, provide a system prompt that specifies instructions on how to achieve tasks such as performing calculations.

Incorrect. You can use fine-tuning to customize FMs with additional training data from a specific domain. A system prompt is a way to set expectations of the "personality" and style of a conversational assistant. Fine-tuning an FM with market data and analysis

examples does not ensure accurate calculations or real-time market insights. FMs are not designed to perform precise mathematical calculations. Fine-tuning does not provide the required calculation capability. Additionally, this solution could provide outdated insights because the model would know data only from its training period.

Learn more about [model customization](#).

Learn more about [system prompts](#).

B

Perform model distillation on an FM. In the context of a conversational flow, provide a system prompt that specifies instructions on how to achieve tasks such as performing calculations.

Incorrect. You can use model distillation to create a smaller and faster model that provides a balance of accuracy and cost. A system prompt is a way to set expectations of the "personality" and style of a conversational assistant. Model distillation cannot provide accurate calculations or real-time market insights. This solution would likely reduce model capabilities. This solution cannot provide calculations or current market insights.

Learn more about [model customization](#).

Learn more about [system prompts](#).

C

Create a decentralized network of agents where each agent independently processes data and communicates directly with end users. In the context of a conversational flow, provide a system prompt that specifies instructions on how to achieve tasks such as performing calculations.

Incorrect. AI agents can connect to different systems, APIs, and data sources. AI agents can automate tasks. A system prompt is a way to set expectations of the "personality" and style of a conversational assistant. For an application that requires multiple agents to complete different tasks, you need a supervisor agent to manage the complex workflows of task-specific agents. The task-specific agents are named sub-agents. Creating a decentralized network of agents without proper orchestration might cause inconsistent results and communication conflicts. Agents that communicate directly with end users might provide contradictory information. This solution would not provide coordinated analysis across different aspects of the financial data.

Learn more about [multi-agent collaboration](#).

Learn more about [system prompts](#).

D Create a system with a supervisor agent that orchestrates specialized sub-agents for quantitative analysis, news processing, and smart summarization.

Correct. AI agents can connect to different systems, APIs, and data sources. AI agents can automate tasks. For an application that requires multiple agents to complete different tasks, you need a supervisor agent to manage the complex workflows of task-specific agents. The task-specific agents are named sub-agents. Amazon Bedrock Agents supports hierarchical agent systems with specialized roles. You can use a supervisor agent to coordinate overall workflow and to deploy specialized sub-agents for specific tasks. Task examples include quantitative analysis, news processing, and summarization. This solution provides coherent output through centralized orchestration. Each agent performs a specialized task. This solution provides consistent and coordinated analysis.

Learn more about [multi-agent collaboration](#).

Learn more about [system prompts](#).

6/75

A research company is using a customizable FM on Amazon Bedrock to develop an internal generative AI (GenAI) powered search interface. During testing, a GenAI developer discovers that the model's performance is suboptimal for a specific niche topic. The company has 3 TB of unlabeled proprietary research papers, technical documentation, and historical reports that cover the niche topic.

The company wants the model to develop a deeper understanding of the niche topic's domain terminology, concepts, and relationships. The GenAI developer must improve the model's accuracy for the niche topic by using the proprietary data.

Which solution will meet these requirements?

A Use the proprietary data to fine-tune the model in Amazon Bedrock.

Incorrect. Fine-tuning is a technique that can improve model performance. However, fine-tuning requires training data in the form of labeled examples. The model learns to associate which types of outputs to generate for certain types of inputs. In the scenario, the company wants the model to develop a deeper understanding of a topic, not to answer specific questions.

Learn more about [model customization](#).

B

Use the proprietary data to perform continued pre-training of the model in Amazon Bedrock.

Correct. With continued pre-training, a model can learn from large amounts of unlabeled domain-specific data. This solution continues general language model training on new content. The model can develop a deeper understanding of the domain's terminology, concepts, and relationships from the 3 TB of proprietary data.

Learn more about [model customization](#).

C

Use the proprietary data as an input for knowledge distillation to transfer insights from the current model to a new model.

Incorrect. Knowledge distillation is a technique that creates a smaller, more efficient model by transferring knowledge from a larger model. Knowledge distillation can transfer knowledge. However, knowledge distillation is not an effective way to incorporate large amounts of new specialized data. This solution focuses on model compression and efficiency. This solution does not improve domain understanding.

Learn more about [knowledge distillation](#).

D

Use an LLM to summarize the proprietary data. Incorporate the summary as a system prompt for the current model.

Incorrect. System prompts are instructions or context that can guide a model's behavior during inference. System prompts can provide context to the model. However, this solution is not an effective way to incorporate large amounts of specialized data. Prompts are suitable to guide model behavior, not to extensively expand a model's knowledge base.

Learn more about [prompt engineering](#).

7/75

A digital content company is building a generative AI (GenAI) application that summarizes news articles. The application needs to route requests to different LLMs based on language and content types. For regulatory compliance, certain content types must use specific model providers.

A GenAI developer must create a solution that can switch between model providers without code changes. The model providers include Amazon Bedrock and third-party APIs. The solution must securely store API keys and maintain consistent response formatting regardless of the underlying model. The solution must optimize costs by using cached responses when appropriate.

Which solution will meet these requirements?

A

Create separate Amazon API Gateway REST APIs for each model provider with unique endpoints. Use a client-side routing application to determine which API endpoint to call based on language and content type. Store API keys in client-side code. Cache responses at the client layer for improved performance.

Incorrect. Separate API Gateway REST APIs with client-side routing is an architectural pattern where each model provider gets its own dedicated API endpoint. This solution implements routing logic in the client application. This solution exposes API keys in client-side code and relies on client-side caching. Therefore, this solution creates security vulnerabilities and inconsistent performance across different client sessions.

Learn more about [API Gateway security best practices](#).

B

Create a single Amazon API Gateway REST API with an AWS Lambda proxy integration. Configure routing logic in the Lambda function to select the appropriate model based on request parameters. Store API keys in AWS Secrets Manager. Configure the function to retrieve the secrets.

Incorrect. Lambda proxy integration passes the entire request to the Lambda function without transformation. Then, the function handles routing logic and provider selection. This solution requires code changes in the Lambda function to support new providers. Additionally, this solution does not provide a built-in caching mechanism to optimize costs.

Learn more about [Lambda proxy integrations](#).

C

Create a single Amazon API Gateway REST API with non-proxy integrations. Configure mapping templates to transform requests and responses for each model provider. Use header-based routing that directs traffic to store endpoint URLs based on content type and stage variables. Use AWS Secrets Manager for API key storage.

Correct. API Gateway non-proxy integrations with mapping templates provide request and response transformation without code changes. You can combine header-based routing with stage variables for dynamic provider selection. Mapping templates ensure consistent response formatting across different providers. Secrets Manager provides secure API key storage. The API Gateway built-in caching capabilities optimize costs by caching responses when appropriate.

Learn more about [parameter mapping in API Gateway](#).

D

Deploy all models to Amazon SageMaker AI endpoints. Create a single Amazon API Gateway REST API with a SageMaker AI integration. Use path parameters to determine which SageMaker AI endpoint to invoke. Add model metadata to SageMaker AI endpoints to ensure consistent response formatting. Store credentials in the SageMaker AI configuration.

Incorrect. SageMaker AI endpoints provide managed infrastructure to host ML models with built-in scaling and monitoring capabilities. However, you must deploy all models to SageMaker AI. Therefore, this solution does not support integration with third-party APIs. Path-based routing is less flexible for complex routing requirements. SageMaker AI configuration storage is not designed to manage multiple provider credentials securely.

Learn more about [SageMaker AI deployment options](#).

8/75

A company is building a generative AI (GenAI) powered application that uses Amazon API Gateway, AWS Lambda, and Amazon Bedrock. The application must support summarization, classification, and translation tasks. A separate FM performs each task. A GenAI developer must configure the application to meet the following requirements:

- Route inference requests to different FMs dynamically based on task type and customer configuration.
- Update routing logic at runtime without redeploying the application.
- Implement automatic failover to an alternate model or AWS Region if the primary model or Region is unavailable.
- Maintain low latency, resilience, and cross-Region support for multiple providers.

Which solution will meet these requirements with the LEAST operational overhead?

A

Create a Lambda function that retrieves model routing rules from an AWS AppConfig hosted configuration profile at runtime. Use an AWS Step Functions state machine with branching paths for each task type and a circuit breaker pattern for failover. Invoke Amazon Bedrock by using Regional endpoints. Retry in a secondary Region if the primary model is unavailable.

Correct. An AWS AppConfig hosted configuration is a managed way to provide dynamic application configuration updates without redeployment. Step Functions is a serverless workflow service that orchestrates multiple AWS services by using state machines with built-in error handling. AWS AppConfig provides runtime routing updates. Step Functions provides resilient failover patterns. Regional endpoints ensure cross-Region

support. Therefore, this solution provides flexibility and resilience with the least operational overhead.

Learn more about [AWS AppConfig](#).

Learn more about [error handling in Step Functions workflows](#).

Learn more about [Amazon Bedrock endpoints](#).

B

Embed a hardcoded task type-to-model mapping dictionary in a Lambda function. Call Amazon Bedrock InvokeModel synchronously from the Lambda function. Use a try/catch block to retry with an alternate model or Region if the primary model or Region is unavailable. Deploy separate Lambda versions for each Region to manage routing.

Incorrect. Lambda is a serverless compute service that runs code in response to events. Lambda functions can implement try/catch error handling and model invocation logic. However, hardcoding model mappings does not meet the requirement to update routing logic at runtime without redeployment.

Learn more about [Lambda environment variables](#).

C

Deploy a Flask-based model router in Amazon ECS with routing metadata stored in Amazon Aurora. Route inference requests from API Gateway to the router. Configure the router to select and invoke the appropriate model by using the Amazon Bedrock SDK. Set up Amazon CloudWatch alarms to trigger route table updates when model errors occur.

Incorrect. Amazon ECS is a container management service. With Amazon ECS, you must maintain container infrastructure. Aurora is a relational database service that requires ongoing database administration. Implementing a custom Flask-based router requires additional operational overhead for container management, database maintenance, and custom routing logic. You can use CloudWatch alarms for monitoring and alerting. However, using CloudWatch alarms to route updates increases complexity and could cause reliability issues.

Learn more about [Amazon ECS](#).

Learn more about [CloudWatch alarms](#).

D

Configure API Gateway request mappings to send each task type to a dedicated Lambda function with a fixed model and Region configuration. Use

AWS Step Functions for fallback processing if the Lambda invocation fails. Implement fallback logic in separate state machines.

Incorrect. API Gateway is a fully managed service to create, publish, and manage APIs. API Gateway supports request mapping and Lambda integration. However, using fixed configurations in dedicated Lambda functions cannot dynamically route updates. Step Functions can handle fallback processing. However, implementing separate state machines for each fallback scenario increases operational overhead. This solution does not provide the flexibility that you need to route updates at runtime without redeployment.

Learn more about [template transformations in API Gateway](#).

9/75

A company is implementing a data pipeline to feed customer transaction data into Amazon Bedrock FMs. The company wants to generate personalized recommendations for customers.

A GenAI developer wants to avoid data quality issues that could affect model output accuracy. The GenAI developer wants to implement automated data validation before the FMs use the data. The company stores the data in Amazon S3 and catalogs the data in AWS Glue Data Catalog. The solution must detect anomalies and filter out low-quality data before the data reaches the FMs.

Which combination of steps will meet these requirements with MINIMAL operational overhead? (Select TWO.)

A

Use Amazon SageMaker Data Wrangler to profile the data. Use SageMaker Model Monitor to detect data drift. Implement custom pre-processing logic in SageMaker processing jobs.

Incorrect. Data Wrangler and Model Monitor primarily focus on model development and monitoring, not data pipeline validation. Model Monitor can detect data drift. However, this approach is designed to monitor production models, not to validate data before entering the FM pipeline.

Learn more about [Data Wrangler](#).

Learn more about [Model Monitor](#).

B

Create an AWS Step Functions workflow to orchestrate serverless validation. Create AWS Lambda functions that run validation checks on the data and store validation results in Amazon DynamoDB.

Incorrect. A custom validation solution that uses Step Functions and Lambda functions requires significant development effort. You must implement validation logic, manage state, and handle errors. This approach introduces more operational overhead than using managed services that are specifically designed for data quality validation.

Learn more about [Step Functions](#).

C

Use AWS Glue Data Quality for the data catalog with rule-based validation and anomaly detection. Create an Amazon EventBridge rule to send alerts when quality scores fall below defined thresholds.

Correct. Data Quality provides both rule-based validation and ML-powered anomaly detection capabilities. Data Quality can evaluate data quality against custom rules that are written in DQDL. Data Quality can detect anomalies by analyzing data statistics over time. You can publish results to EventBridge. EventBridge provides automated alerting when quality issues are detected. This step provides custom validation rules and anomaly detection over time.

Learn more about [Data Quality anomaly detection](#).

D

Implement AWS Glue Data Quality for ETL jobs with Data Quality Definition Language (DQDL) rules that validate the data during processing. Configure the job to filter out records that fail validation before passing the data to Amazon Bedrock.

Correct. You can use Data Quality with DQDL rules to implement data validation during ETL processing. This step can filter out low-quality data before the data reaches Amazon Bedrock. Therefore, this step prevents potential issues with model outputs. This approach integrates validation directly into the data processing pipeline without requiring additional services or custom code.

Learn more about [DQDL](#).

Learn more about [Data Quality](#).

E

Use Amazon Athena to run SQL queries that validate data constraints. Create an Amazon EventBridge rule that invokes an AWS Lambda function to query the results and generate quality metrics.

Incorrect. Athena can run SQL queries for validation. However, you must write and maintain custom SQL queries and Lambda functions to process the results. This step requires more operational overhead than using purpose-built data quality services. For

example, Data Quality provides built-in validation capabilities and integration with the existing data catalog.

Learn more about [Athena](#).

10/75

A financial analytics company wants to create a generative AI (GenAI) solution that can analyze a large amount of unstructured data. The unstructured data includes financial filing forms, quarterly earnings reports, analyst presentations, and audio/video (A/V) recordings of earnings calls. A GenAI developer must create a solution that can process the large amount of unstructured data.

Which solution will meet these requirements with the LEAST operational overhead?

A

Store the data in an Amazon S3 bucket. Create a knowledge base by using Amazon Bedrock Knowledge Bases for RAG workflows.

Incorrect. Amazon S3 can store unstructured data. Knowledge Bases can provide RAG workflows. However, Knowledge Bases does not support audio or video files. This solution does not process the A/V recording data types. Storing raw unstructured data without proper processing would limit the effectiveness of RAG implementation. This solution could reduce the quality of generated insights.

Learn more about [supported document formats for knowledge base data](#).

B

Use Amazon Textract and Amazon Transcribe to process multimodal content. Store extracted information in JSON format in an Amazon S3 bucket. Create a knowledge base by using Amazon Bedrock Knowledge Bases for RAG workflows.

Incorrect. Amazon Textract and Amazon Transcribe can process documents and A/V content. However, you must manage multiple services and coordinate outputs. Storing intermediate results in Amazon S3 and managing the JSON formatting requires additional operational overhead.

Learn more about [Amazon Textract](#).

Learn more about [Amazon Transcribe](#).

C

Use an Anthropic Claude FM in Amazon Bedrock with structured prompts to process multimodal content into standardized formats. Configure prompt

templates for different document types. Create JSON schemas for validation. Store parsed data in Amazon S3 with automatic versioning for RAG workflows.

Incorrect. You can use Claude with structured prompts for multimodal content processing. However, this solution requires extensive prompt engineering and ongoing maintenance. You must create and maintain prompt templates and JSON schemas for different document types. Therefore, this solution requires additional operational overhead.

Learn more about [Claude capabilities](#).

D

Use Amazon Bedrock Data Automation (BDA) as a parser to extract insights from multimodal content. Create a knowledge base by using Amazon Bedrock Knowledge Bases for RAG workflows.

Correct. BDA extracts insights from unstructured multimodal content including documents, forms, and A/V recordings. BDA streamlines the processing of diverse financial data sources. BDA can automatically create knowledge bases for RAG workflows. This solution automatically handles the processing of different data types and the integration with Knowledge Bases. Therefore, this solution requires the least operational overhead.

Learn more about [BDA](#).

Learn more about [how to build an AI-powered assistant](#).

11/75

A retail company needs to process product catalog data from multiple sources to enhance an AI-powered recommendation system. The data includes product descriptions, specifications, and customer reviews in various formats and languages. The system must improve data quality and ensure consistent inputs for the company's FMs.

Which solution will meet these requirements?

A

Extract product attributes by using Amazon Comprehend entity recognition through an AWS Lambda function. Normalize product categories and specifications. Use Amazon Bedrock to reformat product descriptions for optimal FM processing.

Correct. Lambda is a serverless compute service that you can use for data processing tasks. Amazon Comprehend has entity recognition capabilities that extract structured

information from text and help standardize product attributes. Amazon Bedrock has text reformatting capabilities that ensure consistent input structure for FMs. This solution improves input data quality through normalization, entity extraction, and reformatting. Therefore, this solution enhances model response consistency.

Learn more about [Amazon Comprehend entities](#).

Learn more about [Lambda](#).

Learn more about [Amazon Bedrock text generation](#).

B

Process the input data through Amazon Comprehend to extract product entities and sentiment. Store the extracted features in Amazon DynamoDB to achieve a consistent structure that is optimized for FM consumption. Use Amazon CloudWatch to monitor data quality metrics before passing the data to the FMs. Implement formatting rules that check data consistency and structure. Send the data to Amazon Bedrock FMs.

Incorrect. Amazon Comprehend can extract entities and sentiment from text. However, storing these features in DynamoDB without proper text reformatting does not ensure consistent FM inputs. CloudWatch metrics alone cannot improve data quality. This solution does not provide the text reformatting capabilities that you need for this use case.

Learn more about [Amazon Comprehend entities](#).

Learn more about [DynamoDB](#).

Learn more about [CloudWatch](#).

Learn more about [Amazon Bedrock text generation](#).

C

Create an AWS Lambda function to normalize product data formats. Use Amazon Comprehend custom classification to categorize products. Implement custom validation rules that check data consistency for product categories and specifications before sending the data to the Amazon Bedrock FMs.

Incorrect. You can create a Lambda function to normalize data formats. However, using Amazon Comprehend custom classification focuses on categorization, not entity extraction. Custom validation rules without text reformatting capabilities do not provide the necessary input quality improvements.

Learn more about [Amazon Comprehend custom classification](#).

Learn more about [Lambda](#).

Learn more about [Amazon Bedrock text generation](#).

D

Create an AWS Step Functions workflow that processes input data through Amazon Comprehend for entity extraction. Configure the workflow to use an AWS Lambda function for data validation and to implement custom filtering logic. Send the validated data to the Amazon Bedrock FMs for processing.

Incorrect. You can use Amazon Comprehend for entity extraction. You can use Lambda functions for validation. However, this solution does not provide text reformatting capabilities. You cannot properly structure the input data for optimal FM processing without text reformatting capabilities.

Learn more about [Step Functions](#).

Learn more about [Amazon Comprehend entity extraction](#).

Learn more about [Lambda](#).

Learn more about [Amazon Bedrock text generation](#).

12/75

A legal services company wants to integrate diverse document management systems with an AI solution to enhance contract generation. The company needs to connect an existing contract template repository, internal legal knowledge bases, historical case documentation, and compliance wikis. The solution must maintain consistent access patterns. The solution must provide comprehensive data integration across all sources.

Which solution will meet these requirements with the LEAST operational overhead?

A

Deploy Amazon OpenSearch Service with vector search capabilities. Create standardized connectors by using AWS Lambda functions to index content from each source. Implement custom authentication handlers. Maintain separate backup procedures for each system.

Incorrect. OpenSearch Service with vector search provides search capabilities across document repositories. Lambda functions can create standardized connections. However, you must maintain custom connectors, authentication handlers, and backup procedures for each system. Therefore, this solution requires more operational overhead than using managed document integration services.

Learn more about [OpenSearch Service](#).

B

Use Amazon Bedrock Knowledge Bases to create unified access to all document sources. Configure data source connectors for the template repository and knowledge bases. Set up automated synchronization to maintain the current content.

Correct. Knowledge Bases provides built-in capabilities to integrate multiple document sources through standardized connectors. Knowledge Bases handles authentication, synchronization, and content updates automatically. Therefore, this solution requires the least operational overhead. Additionally, this solution provides consistent access patterns across all document sources. This solution maintains data freshness through automated synchronization.

Learn more about [Knowledge Bases](#).

C

Create a hybrid integration. Use Amazon AppFlow for software as a service (SaaS) based sources. Use AWS Transfer Family for on-premises sources. Deploy Amazon OpenSearch Service for unified search. Implement AWS Step Functions workflows to orchestrate data synchronization across sources.

Incorrect. Amazon AppFlow can integrate with SaaS applications. Transfer Family can handle on-premises sources. However, combining Amazon AppFlow and Transfer Family with OpenSearch Service and Step Functions increases operational overhead. For this hybrid approach, you must manage multiple services, orchestration workflows, and synchronization patterns. Therefore, this solution requires additional operational overhead and creates potential points of failure.

Learn more about [Amazon AppFlow](#).

D

Implement Amazon Kendra with custom data source connectors. Set up incremental synchronization by using Amazon EventBridge rules. Create AWS Glue ETL jobs to standardize document formats across sources. Use Amazon S3 as the central document repository.

Incorrect. Amazon Kendra provides enterprise search capabilities. Using Amazon Kendra with custom data source connectors, EventBridge rules, and AWS Glue ETL jobs requires additional operational overhead. For this solution, you must maintain multiple integration components and transformation pipelines.

Learn more about [Amazon Kendra](#).

A company is designing an advanced vector database architecture for FM augmentation. A GenAI developer must configure each architectural requirement.

Select the correct AWS service configuration from the following list for each architectural requirement. Select each service configuration one time or not at all. (Select THREE.)

- Amazon Aurora PostgreSQL with pgvector
- Amazon Neptune with vector search
- Amazon DynamoDB with vector support
- Amazon OpenSearch Service with the k-nearest neighbor (k-NN) plugin

Support distributed similarity search across billions of product catalog vectors with custom relevance scoring.

Amazon OpenSearch Service with the k-nearest neighbor (k-NN) plugin

Combine metadata filtering with vector search while maintaining strong consistency for financial records.

Amazon Aurora PostgreSQL with pgvector

Enable graph-based vector search to identify relationships between research papers and their citations.

Amazon Neptune with vector search

OpenSearch Service with the k-NN plugin can handle distributed similarity search across billions of product catalog vectors with custom relevance scoring. OpenSearch Service provides a distributed architecture that can scale to billions of vectors across multiple shards. OpenSearch Service is unique in providing custom plugins to fine-tune search relevance compared to the other options.

Aurora PostgreSQL with pgvector is the only option that can combine metadata with vector search while maintaining strong consistency for financial records. Aurora PostgreSQL with pgvector provides atomicity, consistency, isolation, and durability (ACID) compliance. This configuration provides complex SQL querying capabilities and vector similarity search. This configuration is suitable for applications that require both precise metadata filtering and reliable transaction handling.

Neptune with vector search is the most suitable option for graph-based vector search applications that require relationship analysis between connected data points. Neptune combines graph database capabilities with vector search. Therefore, Neptune provides semantic similarity search that traverses relationships between entities. You can use

Neptune when understanding connections is crucial. For example, Neptune is the most suitable option for research paper citation networks.

DynamoDB with vector support is a NoSQL database service that provides vector storage capabilities. This configuration provides storage and retrieval of vector embeddings with other item attributes. DynamoDB can store vectors and perform basic similarity search. However, DynamoDB does not support custom relevance scoring or distributed similarity search at a billion-vector scale. DynamoDB does not provide the strong consistency and complex metadata filtering capabilities that you need for financial record keeping. Additionally, DynamoDB does not provide automated content processing and built-in FM optimization. DynamoDB vector support is most suitable for less complex vector storage use cases where NoSQL flexibility is the primary requirement.

Learn more about [k-NN in OpenSearch Service](#).

Learn more about [Aurora with pgvector](#).

Learn more about [Neptune](#).

Learn more about [DynamoDB vector search](#).

14/75

A global academic publishing company is developing a comprehensive research system that will store millions of scientific papers in Amazon S3. The system will use FMs in Amazon Bedrock to generate insights, recommend related papers, and provide natural language querying. The system must do the following:

- Record when authors publish, submit, and update papers.
- Store complete authorship information including name and contact details.
- Classify papers across multiple scientific disciplines, sub-disciplines, and methodologies.
- Provide search and filter capabilities by using complex criteria that combines metadata elements.
- Provide FMs with rich contextual information about each paper to improve response quality.

Which metadata framework design will meet these requirements with the FASTEST query response?

A

Store timestamps as S3 user-defined metadata. Store authorship details and classifications in Amazon DynamoDB. Use S3 object keys as partition keys. Use S3 object tags for disciplines. Deploy Amazon OpenSearch Service for indexing and search.

Incorrect. Storing timestamps as S3 user-defined metadata has limitations for complex querying. Using S3 object tags only for high-level disciplines does not support the granular classification that you need across multiple scientific disciplines. DynamoDB

provides fast access to authorship details. However, this design lacks the semantic search capabilities that you need for effective FM integration.

Learn more about [metadata in Amazon S3](#).

Learn more about [S3 tables](#).

Learn more about [object tags](#).

Learn more about [vector search](#).

B

Store timestamps as S3 system-defined metadata. Store authorship details and classifications in an Amazon RDS for PostgreSQL database that links to S3 object keys. Use Amazon Kendra with custom document attributes for search.

Incorrect. RDS for PostgreSQL can store complex metadata relationships. However, using a separate relational database poses potential synchronization challenges. Amazon Kendra document attributes are useful for search. However, the document attributes do not provide the optimized vector-based capabilities that you need for FM interactions.

Learn more about [metadata in Amazon S3](#).

Learn more about [S3 tables](#).

Learn more about [Amazon Kendra document enrichment](#).

C

Store all metadata in Amazon DynamoDB. Use composite keys to enable various access patterns. Store the papers in Amazon S3 with minimal metadata. Set up S3 Event Notifications to maintain synchronization between services. Implement Amazon SageMaker Feature Store to prepare metadata features for FM interactions.

Incorrect. DynamoDB can support various access patterns through composite keys. However, when you store all metadata separately from the documents, maintaining consistency is more challenging. Feature Store is designed for ML feature management and is not optimized for metadata preparation.

Learn more about [DynamoDB composite keys](#).

Learn more about [Feature Store](#).

D

Store timestamps as S3 system-defined metadata. Store authorship details as S3 user-defined metadata. Create a hierarchical S3 object tag structure for scientific classifications. Integrate with Amazon OpenSearch Service enhanced with document vectors from Amazon Bedrock for semantic search capabilities.

Correct. You can use S3 system-defined metadata for timestamps to provide reliable tracking. User-defined metadata efficiently stores authorship information. The hierarchical S3 object tags structure provides multi-level scientific classification. OpenSearch Service enhanced with document vectors from Amazon Bedrock provides powerful semantic search capabilities. This design provides both efficient metadata management and the rich contextual information that you need for FM interactions. This design provides the fastest query response to support advanced filtering and semantic search across millions of documents.

Learn more about [metadata in Amazon S3](#).

Learn more about [S3 tables](#).

Learn more about [object tags](#).

Learn more about [vector search](#).

15/75

An aircraft repair company receives repair requests from various airlines. The company responds to the repair requests by providing an initial quote of the estimated labor hours, required spare parts, and a schedule for completion. The repair requests from airlines include a description of the defect and the aircraft model. The company uses repair manuals in PDF format to help resolve the repair requests. The repair manuals contain thousands of pages with nested and cross-referenced sections from the aircraft manufacturers. The repair manuals explain the necessary repair procedures and spare parts for the repair.

The company wants to extract the repair procedures and spare parts from the repair manuals automatically by using RAG. The solution must provide high retrieval accuracy.

Which solution will meet these requirements with the LEAST operational overhead?

A

Store repair manuals on Amazon DocumentDB serverless with search keys for aircraft model and part. Use Amazon Bedrock Knowledge Bases with a semantic chunking strategy. Use DocumentDB serverless as a vector store.

Incorrect. DocumentDB serverless provides managed database capabilities to store JSON documents. DocumentDB is not optimized to store or manage large PDF documents. Knowledge Bases does not support DocumentDB as a data source. Additionally, semantic chunking is not suitable for nested and cross-referenced technical documentation. Semantic chunking could split content across important section boundaries. Therefore, this solution could reduce retrieval accuracy for repair procedures and spare parts.

Learn more about [DocumentDB](#).

Learn more about [knowledge base data sources](#).

Learn more about [semantic chunking](#).

B

Store repair manuals in Amazon S3 partitioned by aircraft model and part. Use Amazon Bedrock Knowledge Bases with a hierarchical chunking strategy. Use Amazon OpenSearch Serverless as a vector store.

Correct. Knowledge Bases is a fully managed end-to-end RAG workflow. Knowledge Bases integrates with OpenSearch Serverless as a vector store. Knowledge Bases supports Amazon S3 as a data source. Knowledge Bases supports hierarchical chunking as a chunking strategy. This solution provides a fully managed end-to-end RAG workflow. Therefore, this solution requires the least operational overhead. Hierarchical chunking is most suitable for documents with hierarchical structures such as technical repair manuals. Hierarchical chunking provides high retrieval accuracy.

Learn more about [fully managed RAG with knowledge base](#).

Learn more about [hierarchical chunking](#).

C

Store repair manuals on Amazon DocumentDB serverless with search keys for aircraft model and part. Use Amazon OpenSearch Serverless with the neural plugin hierarchical chunking strategy.

Incorrect. DocumentDB serverless provides managed database capabilities to store JSON documents. DocumentDB is not optimized to store and manage large PDF documents. OpenSearch Serverless with the neural plugin provides vector search capabilities. However, this solution is not a fully managed RAG workflow. Creating a custom hierarchical chunking strategy requires custom code to chunk and ingest

documents. You must manage the chunking logic, embeddings generation, and vector search integration separately. Therefore, this solution requires additional operational overhead.

Learn more about [OpenSearch Serverless data ingestion](#).

Learn more about [DocumentDB](#).

D

Store repair manuals in Amazon S3 partitioned by aircraft model and part. Use Amazon Bedrock Knowledge Bases with a semantic chunking strategy. Use Amazon OpenSearch Serverless as a vector store.

Incorrect. This solution provides a fully managed RAG workflow. However, semantic chunking is not suitable for nested and cross-referenced technical documentation. Semantic chunking could split content across important section boundaries in repair manuals. Therefore, semantic chunking could break the relationships between related repair procedures and spare parts. This solution would reduce retrieval accuracy when you search for specific repair procedures and spare parts.

Learn more about [semantic chunking](#).

16/75

A media company uses various AI agents to automate content preparation tasks. One system automatically generates social media posts from news articles. One of the agentic workflows frequently encounters errors when interacting with the company's legacy content management system (CMS) API. The CMS API has inconsistent endpoint designs and poorly documented response schemas. Multiple AI agent workflows need to interact with the CMS API. A development team is spending significant time handling edge cases and API inconsistencies.

The development team needs to implement a solution that standardizes and reduces the complexity of interactions with the CMS. The solution must maintain the existing API endpoints for legacy applications. The solution must be reusable across different AI agent workflows.

Which solution will meet these requirements with the LEAST operational overhead?

A

Create a new REST API transformation layer that standardizes the CMS API responses and provides detailed OpenAPI documentation. Deploy the layer as a proxy service between AI agents and the CMS.

Incorrect. This solution would improve documentation and standardize responses. However, this solution does not address the needs of the AI agents. This solution

requires custom integration work for each agent. This solution does not use function calling interfaces that are specifically designed for AI interactions. This solution could introduce additional latency. This solution does not provide the semantic understanding that MCP provides.

Learn more about [developing REST APIs by using OpenAPI in Amazon API Gateway](#).

B

Develop a custom middleware layer that transforms API requests and responses. Deploy the custom middleware layer as a sidecar container alongside each AI agent to handle CMS interactions.

Incorrect. This solution could address the immediate issue. However, this solution is not scalable or maintainable. You must deploy and maintain separate middleware for each AI agent. Therefore, this solution requires additional operational overhead. Additionally, this solution does not use the standardized function calling interface that would help AI agents interact with the CMS semantically.

Learn more about [sidecar containers](#).

C

Refactor the existing CMS API to follow modern REST principles and add comprehensive documentation. Update all AI agents to use the new standardized endpoints.

Incorrect. This solution requires significant changes to the existing CMS. Therefore, this solution could break compatibility with legacy applications. Additionally, this solution does not address the needs of AI agents for function calling interfaces. This solution requires substantial operational overhead to update all existing AI agents.

Learn more about [REST API development in Amazon API Gateway](#).

D

Implement a Model Context Protocol (MCP) server that provides a standardized interface to the CMS. Define function schemas for CMS operations. Implement the functions to handle API inconsistencies internally. Configure Amazon Bedrock AgentCore to interact with the CMS through the MCP function calls.

Correct. MCP is designed to provide a consistent interface for AI models and agents to interact with external tools and APIs. The MCP server provides function schemas that define a standardized way to interact with the CMS. The implementation of the functions handles the API inconsistencies internally. This solution abstracts the complexities of the underlying CMS API from the agents. Therefore, this solution provides a clean, consistent interface. This solution is reusable across different agent workflows. The

workflows can all interact with the CMS through the same MCP functions. This solution reduces integration complexity and operational overhead. You can update API changes or new edge cases in one central location rather than in each individual agent.

Learn more about [MCP for richer context](#).

Learn more about [MCP](#).

17/75

A medical diagnostics company runs a chat-based AI application to help customers find appropriate tests from a catalog of diagnostic tests. Each test contains detailed descriptions, target conditions, specimen types, and specimen collection guidelines. The application uses Amazon Bedrock Knowledge Bases supported by Amazon OpenSearch Serverless to search the catalog of available diagnostic tests.

Initially, the search provides sufficient recall. However, the search is unable to prioritize the most relevant documents. As a result, the company decides to continue using hybrid search. To achieve the desired accuracy, the company increases response results to 50 to pass to the LLM for summarization.

The company experiences an increase in customer use of the application. The company notices an increase in token usage. Now, the company wants to reduce token usage for each customer interaction without impacting accuracy.

Which solution will meet these requirements with the LEAST effort?

A

Limit the context window by passing only the top five retrieved documents to the LLM for summarization.

Incorrect. Limiting the context window to pass only five documents can reduce token usage. However, this solution does not solve the root problem of poor relevant ranking. Without reranking, the five chosen documents might not be the most relevant. Without reranking, this solution could reduce the accuracy and quality of the answers. Any method that reduces document count without improving the ranking mechanism does not meet the requirement to maintain accuracy while reducing token usage.

Learn more about [Knowledge Bases](#).

Learn more about [the maximum number of retrieved results](#).

B

Reconfigure the chunking strategy to use semantic-based adaptive chunking with overlap. Reduce the retrieval set to the top ten documents to pass to the LLM for summarization.

Incorrect. Adaptive semantic chunking with overlap is a strategy that you can use to improve retrieval quality. This strategy splits documents at logical boundaries rather than by fixed token costs. Reducing the retrieval set to ten documents would lower token usage and provide a broader context. However, you must reprocess and reindex the document corpus. Therefore, this solution requires additional effort.

Learn more about [semantic chunking](#).

C

Configure the knowledge base to invoke a reranker model. Pass only the top five ranked documents to the LLM for summarization.

Correct. Knowledge bases support reranker models that can reorder retrieval results to improve precision. You can enable reranking and limit the retrieval set to the top five ranked documents. This solution maintains accuracy while reducing the tokens that pass to the LLM. This solution requires minimal effort because you can directly make a change in the knowledge base to improve search precision and reduce token costs.

Learn more about [reranker models](#).

D

Configure the knowledge base to use semantic search as the retrieval method.

Incorrect. Semantic search retrieves results based on vector similarity rather than keywords alone. Semantic search can improve relevance. Semantic search provides good recall, but not precision. The company uses hybrid search through OpenSearch Serverless. Therefore, the search already includes semantic search. Switching to semantic-only would not address the ranking issue or token usage issue.

Learn more about [semantic search](#).

Learn more about [hybrid search](#).

18/75

A GenAI developer at a media company is building a question-answering AI assistant by using Amazon Bedrock Knowledge Bases. The AI assistant needs to answer user questions accurately based on only the most recent documents. The GenAI developer must ensure that the AI assistant ignores older documents.

Which solution will meet these requirements?

A

Use a prompt template to instruct the model to ignore outdated documents.

Incorrect. Prompt templates control how the model responds. However, prompt templates do not control what documents the model retrieves. You must handle retrieval filtering through metadata filters.

B

Add a metadata filter for modification time.

Correct. You can use the metadata filter `modification_time` to restrict the source documents based on timestamps. You can add a metadata filter to ensure that the model retrieves only recently updated documents.

Learn more about [knowledge base configurations](#).

C

Set the search type to semantic.

Incorrect. Semantic search helps match user queries with semantically similar content based on meaning rather than exact keywords. However, semantic search does not filter based on document recency. Without a metadata filter, the model can still retrieve old documents.

D

Enable query modification.

Incorrect. Query modifications improve the handling of complex or multi-part questions. However, query modifications do not affect which documents the model retrieves based on recency.

19/75

A company has a large collection of HTML documents. The documents contain articles with varying lengths and complex hierarchical structures. Headers identify each article. Each article contains multiple paragraphs that can range from a few sentences to several pages in length.

A GenAI developer must build a RAG solution that preserves the relationships between articles and the articles' contained paragraphs. The solution must retrieve relevant content for user inquiries. The solution must minimize irrelevant or inaccurate responses.

Which solution will meet these requirements?

A

Create an AWS Lambda function that implements a custom hierarchical chunking strategy. Use the LangChain framework for the HTML documents. Deploy LangChain through a Lambda function layer. Create an Amazon Bedrock knowledge base with the documents. Use the Lambda function as the custom chunking configuration.

Correct. Amazon Bedrock Knowledge Bases supports custom chunking through Lambda functions. This solution can process documents with highly variable lengths. A custom Lambda function with LangChain provides a specialized chunking strategy. This solution can properly handle articles and paragraphs of any length. This solution can preserve the hierarchical relationships in the articles and paragraphs. This solution provides the control and flexibility that you need for complex document structures.

Learn more about [custom chunking](#).

B

Create an Amazon Bedrock knowledge base with the HTML documents. Use the built-in hierarchical chunking strategy. Configure the chunking strategy with parent chunks and child chunks. Set the estimated sizes based on the average length observed in the documents.

Incorrect. Amazon Bedrock provides built-in hierarchical chunking that maintains parent-child relationships in structured documents. However, setting fixed chunk sizes based on averages is inadequate for documents with highly variable paragraph lengths. This solution can lead to inappropriate splitting of content when paragraphs are significantly longer or shorter than the average. This solution could break important contextual relationships in the document structure.

Learn more about [hierarchical chunking](#).

C

Create an Amazon Bedrock knowledge base with the HTML documents. Use the built-in semantic chunking strategy. Configure the chunking strategy to divide the documents into chunks based on their semantic meaning and content rather than syntactic structure.

Incorrect. Amazon Bedrock provides built-in semantic chunking that automatically divides documents based on meaning and content coherence. This solution uses natural language understanding to identify logical breaks. This solution prioritizes meaning over structure. Therefore, this solution could break the required hierarchy between articles and paragraphs. This solution could lose the explicit relationships that you need for accurate information retrieval.

Learn more about [semantic chunking](#).

D

Use Amazon Textract to extract the text from the HTML documents. Create chunk files based on the paragraph extractions from Amazon Textract. Upload the extractions to an Amazon S3 bucket. Create an Amazon Bedrock knowledge base with the chunked files. Use the no chunking option.

Incorrect. Amazon Textract is designed to extract text from image-based documents and PDFs, not from HTML documents. Additionally, creating chunks from extracted paragraphs without considering the document hierarchy will not maintain the relationships between articles and paragraphs. The no chunking option would treat each extraction as an independent unit. Therefore, you could lose the contextual relationships that you need for accurate information retrieval.

Learn more about [Amazon Textract supported document types](#).

20/75

A company developed a tool to assist customer support representatives. The tool summarizes relevant support documents for the case that a customer support representative works on. The tool uses a custom RAG system. The system is backed by a third-party vector store that stores document embeddings. The RAG system retrieves the top-k relevant chunks based on the current support case. The generative model that summarizes the documents runs on Amazon Bedrock.

Recently, customer support representatives report that the summaries are contextually irrelevant and do not directly relate to the support cases. A GenAI developer verifies that the chunks retrieved from the vector store have high embedding similarity scores. The GenAI developer validates that the embedding model produces accurate representations and that the chunking strategy is consistent. The GenAI developer wants to improve the summarization to return more contextually relevant summaries. The GenAI developer wants to continue using Amazon Bedrock hosted models. The GenAI developer does not want to re-train any LLMs.

Which strategies will improve the relevance of retrieved context? (Select TWO.)

A

Fine-tune the underlying model on company-specific documents to increase the contextual relevance of the summaries.

Incorrect. Fine-tuning the underlying model can improve the quality of the summaries themselves or the domain alignment. However, this strategy does not improve retrieval contextual relevance. The issue is about irrelevant retrieved chunks, not about model generation fidelity.

Learn more about [Knowledge Bases](#).

B

Increase the number of retrieved documents to ensure that more contextually relevant documents are included in the generation process.

Incorrect. Increasing the number of retrieved documents can make relevance worse. This strategy adds more low-value results instead of focusing on the best matches.

Learn more about [Knowledge Bases](#).

C

Migrate to using Amazon Bedrock Knowledge Bases for retrieval. Configure reranking when retrieving chunks.

Correct. Reranking is a feature of Amazon Bedrock that reorders chunks based on relevancy to the query. Knowledge Bases provides built-in reranking mechanisms that improve contextual relevance. You can migrate to an Amazon Bedrock knowledge base to offload relevance scoring. This strategy avoids manual reranking.

Learn more about [reranking in Amazon Bedrock](#).

D

Re-chunk the source documents into larger segments to capture more context for the summarization model to use.

Incorrect. Larger chunks could decrease relevance by including more unrelated text in each chunk. This strategy can impair embedding similarity scoring. This strategy can lead to embedding mismatches. The issue is not a lack of context. The issue is poor prioritization of relevant chunks.

Learn more about [reranking in Amazon Bedrock](#).

E

Add a reranking step after initial retrieval by invoking a rerank model in Amazon Bedrock. Configure the rerank model to rescore and sort retrieved chunks before generation.

Correct. Reranking is a feature of Amazon Bedrock that reorders chunks based on relevancy to the query. This strategy reflects the manual implementation of a reranking layer. After retrieving documents from the vector store, you can use an Amazon Bedrock rerank model to reorder chunks based on semantic relevance to the query. This strategy improves context selection without re-training the LLM or altering the embedding model.

Learn more about [reranking in Amazon Bedrock](#).

21/75

A company is deploying a generative AI (GenAI) solution across different business units to assist with documentation. The solution requires extensive testing and logging to provide audit trails and approval workflows of modifications throughout the solution's lifecycle. The solution must remain consistent in its instructions and responses. The solution must enforce role-based access to prompt templates and prevent inappropriate content generation.

Which solution will meet these requirements?

A

Manage prompt templates in Amazon S3 with object versioning enabled. Use Amazon Macie for toxic-content detection in generated text. Use AWS CloudTrail for audit logging.

Incorrect. S3 Versioning maintains file histories. However, S3 Versioning does not provide the necessary governance features to manage prompt templates or ensure consistent responses. Macie is designed to detect sensitive data in static content that is stored in Amazon S3. However, Macie cannot perform real-time content filtering or enforce safety policies for GenAI outputs. Additionally, Macie cannot consider the requester's role or enforce prompt consistency requirements. CloudTrail provides audit logging capabilities. However, this combination of services lacks the necessary features to manage prompt templates, enforce safety controls, and implement approval workflows to meet the requirements.

Learn more about [S3 Versioning](#).

Learn more about [Macie](#).

B

Manage prompts in Amazon SageMaker Model Registry. Use SageMaker Clarify for real-time bias filtering and real-time toxicity filtering of model outputs. Set up AWS CloudTrail for audit logging.

Incorrect. Model Registry tracks model artifacts, not prompt templates. Clarify can evaluate bias and toxicity. However, Clarify lacks real-time blocking integration with Amazon Bedrock prompts. CloudTrail can log API calls. However, this solution does not address template governance and role consistency.

Learn more about [Model Registry](#).

Learn more about [Clarify](#).

C

Use Amazon Bedrock Prompt Management for parameterized templates and role definitions. Configure Amazon Bedrock Guardrails to enforce safety policies. Enable AWS CloudTrail with Amazon CloudWatch Logs to record all prompt invocations and template changes.

Correct. Prompt Management provides version-control and role-aware templates with built-in approval workflows. Guardrails apply configurable policies that automatically block unsafe generations. CloudTrail records every Amazon Bedrock API call. Examples of API calls include template creation, changes, and invocations. CloudWatch Logs can capture full prompt and response data. Therefore, this solution provides end-to-end traceability for auditors.

Learn more about [Prompt Management](#).

Learn more about [Guardrails](#).

Learn more about [CloudTrail](#).

Learn more about [CloudWatch Logs](#).

D

Use Amazon Bedrock Prompt Management to create prompt templates. Configure Amazon Bedrock Guardrails to enforce safety policies and role-based access controls (RBAC). Enable AWS CloudTrail with Amazon CloudWatch Logs to record all prompt invocations and template changes.

Incorrect. CloudTrail and CloudWatch Logs provide proper audit trails. Prompt Management and Guardrails cover template governance and safety. However, guardrails do not manage RBAC to templates.

Learn more about [Prompt Management](#).

Learn more about [Guardrails](#).

Learn more about [CloudWatch Logs](#).

22/75

A marketing company generates creative briefs for clients by using Amazon Bedrock. Each client requires tailored brand tone, formatting, and output rules. The company wants to implement a configuration that defines reusable and parameterized prompt templates. The configuration must enforce style constraints such as restricting emojis or informal phrases. The configuration must track prompt usage and changes for compliance and auditing purposes. The configuration must ensure that prompt updates are reviewed and approved before activation.

Which Amazon Bedrock Prompt Management configuration will meet these requirements?

A

Create templates for each client. Apply versioning to track changes and automatically activate new versions as the versions save. Set up Amazon CloudWatch Logs to track prompt invocations for compliance.

Incorrect. Prompt Management supports versioning. However, automatically activating new versions does not provide the required review and approval workflow. CloudWatch Logs captures metrics including invocation counts. However, CloudWatch Logs does not provide a compliance-grade audit trail.

Learn more about [Prompt Management versioning](#).

Learn more about [CloudWatch Logs](#).

B

Create reusable templates with parameters for client-specific rules. Store prompt usage history in Amazon S3 by exporting logs from the application layer. Use tags to indicate whether a prompt is approved for production.

Incorrect. Prompt Management provides reusable templates. However, using tags for approval is a manual approach that does not enforce controlled review before activation. Exporting logs to Amazon S3 provides history. However, this configuration does not provide immutable audit logs. This configuration does not provide a mechanism to enforce style rules such as restricting emojis.

Learn more about [Prompt Management reusable templates](#).

C

Create reusable templates with versioning and review workflows. Apply Amazon Bedrock Guardrails to enforce style rules. Enable AWS CloudTrail to log prompt usage and template changes for compliance.

Correct. You can use Prompt Management to create, store, and manage reusable, parameterized prompt templates. You can define system instructions to establish an assistant's role and add parameterized variables. The variables can include company name and document content. Then, you can reuse prompts across clients.

Prompt Management provides built-in versioning and review workflows. Prompt Management can help ensure that new prompt versions are reviewed and approved before activation. To enforce style and content rules, you can use Guardrails. You can configure word filters to block specific words, tokens, or characters. For example, you can block emoji glyphs. You can set denied topics to prohibit informal or noncompliant

language. CloudTrail records API activity for Prompt Management and Guardrails. Therefore, this configuration provides an immutable audit log of prompt usage and changes.

Learn more about [Prompt Management](#).

Learn more about [Guardrails](#).

Learn more about [word filters](#).

Learn more about [CloudTrail](#).

D

Define prompt templates, and parameterize tone and formatting. Configure IAM identity policies to restrict who can create or edit templates. Create an AWS Config custom rule to evaluate prompt resources for compliance against style rules.

Incorrect. IAM can control who edits templates. However, IAM and AWS Config custom rules cannot enforce linguistic or stylistic rules. This configuration cannot prevent emojis or informal phrases. IAM policies improve access control. However, IAM policies do not provide the review workflow for compliance logging that you need for prompt governance.

Learn more about [IAM policies](#).

23/75

An investment company wants to use Amazon Bedrock to summarize complex financial documents. The solution must catalog and manage prompts from the summarization process. The prompts need to be templated with variables for client company names and document content. The solution must provide prompt versioning, a comparison of different versions, and prompt testing before deployment.

Which solution will meet these requirements with the LEAST operational overhead?

A

Store prompts in Amazon S3 as JSON templates. Use Amazon Bedrock Prompt Management for A/B testing of different prompt versions. Create a custom AWS Lambda function to handle prompt versioning and parameter substitution. Use Amazon CloudWatch to track prompt performance metrics.

Incorrect. Amazon S3 is an object storage service that offers versioning capabilities to track changes of the objects over time. Amazon S3 can store JSON templates. However, Amazon S3 does not provide built-in prompt cataloging, versioning, or testing. Using Amazon S3 requires additional Lambda logic to implement parameter substitution

and version control. Therefore, this solution requires additional operational overhead. You can use CloudWatch for monitoring. However, this solution duplicates built-in capabilities that Prompt Management already provides.

Learn more about [Amazon S3](#).

B

Use Amazon Bedrock Prompt Management to create separate prompts for each financial document type. Implement a naming convention to track prompt versions. Use the compare versions feature to test different prompt versions and to select the highest-performing version for each document type. Create static system prompts that define the model's role as a financial analyst. Use the prompt testing feature to validate prompts before deployment.

Incorrect. Prompt Management supports prompt cataloging, version comparison, and prompt testing. You can manage versioning by creating separate prompts for each document type by using naming conventions. However, this method alone is error-prone and not scalable. This solution does not provide the formal prompt versioning and parameterized templating that the company requires. Static system prompts do not provide customization of the model's role for different client company contexts. This solution uses Prompt Management features to test and compare versions. However, this solution does not meet the requirements for parameterized variables to handle client company names and document content dynamically. This solution requires additional manual effort to maintain and update prompts for each new document type.

Learn more about [Prompt Management](#).

C

Create prompts in Amazon Bedrock Prompt Management. Define system instructions that establish the model's role as a financial analyst. Use prompt templates with parameterized variables for client company names and document content. Test each version by deploying the version to a staging environment through Amazon Bedrock APIs. Select the appropriate option for production after testing.

Incorrect. Prompt Management provides a centralized way to store, catalog, and manage prompts. Prompt Management supports parameterized templates with variables and system instructions to control the model's role and tone. However, this solution requires manual deployment to a staging environment for testing. This solution requires additional operational overhead to deploy the versions.

Learn more about [Prompt Management](#).

Learn more about [version comparison in Prompt Management](#).

D

Create prompts in Amazon Bedrock Prompt Management. Define system instructions that establish the model's role as a financial analyst. Include parameterized variables in the user message template. Use the compare versions feature to test different prompt versions without deployment. Create a prompt version with the selected prompt to run by using Amazon Bedrock APIs.

Correct. Prompt Management provides a centralized service to store, catalog, and manage prompts. Prompt Management supports parameterized templates with variables and system instructions to control the model's role and tone. This solution offers built-in capabilities for prompt versioning, testing, and deployment without requiring custom infrastructure. The compare versions feature provides prompt testing and side-by-side evaluation without deployment. Prompt versioning ensures that you can lock and execute the selected prompts through Amazon Bedrock APIs.

Learn more about [Prompt Management](#).

Learn more about [version comparison in Prompt Management](#).

24/75

A GenAI developer is creating a chat application. The application integrates an Amazon Bedrock FM through AWS Lambda and exposes a REST API. The application requires conversation history functionality that supports concurrent user sessions with real-time updates. Users must be able to resume conversations from any point in the history.

The application must provide metadata-based search and filtering while maintaining conversation context for the FM. The application requires low-latency retrieval of recent conversations. The application requires retention policies that automatically delete older conversations when the conversations expire.

Which solution will provide the MOST scalable implementation of conversation history?

A

Use Amazon DynamoDB with a composite key of user ID and timestamp for conversation storage. Implement DynamoDB Streams to maintain a conversation cache in Amazon ElastiCache (Redis OSS). Configure DynamoDB TTL to delete older conversations.

Incorrect. DynamoDB with a composite key of user ID and timestamp provides time series storage. You can use DynamoDB Streams to keep an ElastiCache (Redis OSS) cache updated for faster reads. DynamoDB TTL automatically deletes expired items to enforce retention. However, this solution supports only simple time-ordered queries. This solution does not help you filter or resume conversations by using metadata. An

example of metadata is conversation ID. This solution does not meet the requirement for flexible conversation search and filtering.

Learn more about [DynamoDB Streams](#).

B

Create an Amazon DynamoDB table with global secondary indexes (GSI) for user ID and conversation ID. Use a single-table design with hierarchical sort keys to store messages, metadata, and conversation state. Implement DynamoDB Accelerator (DAX) to cache recent conversations. Configure DynamoDB TTL to delete older conversations.

Correct. DynamoDB with GSI and a single-table design provides a highly scalable solution for conversation storage. This solution supports metadata-based queries by user ID, conversation ID, and date ranges. You can use hierarchical sort keys to efficiently query conversation history while maintaining relationships between messages and metadata. DAX provides in-memory caching for sub-millisecond retrieval of recent conversation history. Therefore, this solution meets the low-latency requirement. DynamoDB TTL automatically deletes expired items to enforce data retention.

Learn more about [DynamoDB single-table design patterns](#).

Learn more about [DAX](#).

Learn more about [DynamoDB Streams](#).

C

Store conversations in Amazon OpenSearch Service with conversation metadata and full-text search capabilities. Use time-based indices to query. Configure Index State Management (ISM) policies to delete indices older than the specified retention window.

Incorrect. OpenSearch Service provides full-text search and advanced filtering. Time-based indices with ISM can automatically delete older data. This solution provides search and deletion capabilities. However, this solution does not optimize metadata-based search, filtering, and low-latency retrieval of recent conversations. This solution introduces possible indexing delays. This solution has scaling limitations for high-concurrency workloads.

Learn more about [OpenSearch Service scaling](#).

D

Implement a hybrid storage architecture. Use Amazon ElastiCache (Redis OSS) for active conversations with persistence enabled. Set key TTLs for active conversation entries. Use Amazon Aurora PostgreSQL with the

pgvector extension for searchable conversation history and context retrieval. Use pg_cron in Aurora PostgreSQL to run scheduled deletes for rows older than the specified retention window.

Incorrect. A hybrid architecture using ElastiCache (Redis OSS) with key TTLs for active sessions and Aurora PostgreSQL with pgvector can support real-time interactions. However, this solution does not scale as needed. This solution requires managing two separate datastores and custom scheduling logic. Redis cluster management and PostgreSQL connection handling become bottlenecks as user concurrency increases. This solution requires manual scaling operations.

Learn more about [ElastiCache scaling](#).

25/75

A company develops an AI-powered product support chat assistant for a website. The architecture requires the chaining of the following three LLM calls:

- The first LLM call classifies the sentiment of the messages.
- The second LLM call summarizes documents from a product database.
- The third LLM call creates the final response.

The company wants to maintain versions of the LLM prompts. The company wants to be able to roll back quickly if a new prompt underperforms.

Which solution will meet these requirements with the LEAST development effort?

A

Create an Amazon Bedrock knowledge base to retrieve documents from the product database. Use Amazon Bedrock Prompt Management to store the LLM prompts for each of the three LLM calls. Orchestrate the three LLM calls in a sequential workflow by using Amazon Bedrock Flows.

Correct. Knowledge Bases is a managed RAG service that you can use to securely connect an LLM to enterprise data. Prompt Management provides lifecycle control for prompts. Flows is a visual orchestration service that you can use to chain multiple LLM calls. This solution chains multiple LLM calls together, maintains versions of prompts, and retrieves information from the product database.

Learn more about [Knowledge Bases](#).

Learn more about [Prompt Management](#).

Learn more about [Flows](#).

B

Create an Amazon Bedrock knowledge base to retrieve documents from the product database. Use Amazon Bedrock Prompt Management to store the LLM prompts for each of the three LLM calls. Orchestrate the three LLM calls in a sequential workflow by using AWS Step Functions.

Incorrect. Knowledge Bases is a managed RAG service that you can use to securely connect an LLM to enterprise data. Prompt Management provides lifecycle control for prompts. Step Functions is an orchestration service. You can use Step Functions to orchestrate the three LLM calls. However, you must create and manage the Step Functions workflow and task functions. Therefore, this solution requires additional development effort.

Learn more about [Knowledge Bases](#).

Learn more about [Prompt Management](#).

Learn more about [Step Functions and Amazon Bedrock](#).

C

Create an Amazon Bedrock knowledge base to retrieve documents from the product database. Use AWS Systems Manager Parameter Store to store the LLM prompts for each of the three LLM calls. Orchestrate the three LLM calls in a sequential workflow by using Amazon Bedrock Flows.

Incorrect. Knowledge Bases is a managed RAG service that you can use to securely connect an LLM to enterprise data. Parameter Store is a service that can manage configuration data such as environment variables. Flows is a visual orchestration service that you can use to chain multiple LLM calls. Parameter Store is not the most suitable service to store the prompts. Parameter Store supports the versioning of parameters and the rollback to previous versions. However, Parameter Store is not designed to store prompts for LLMs.

Learn more about [Knowledge Bases](#).

Learn more about [Parameter Store](#).

Learn more about [Flows](#).

D

Use Amazon Q Business to retrieve documents from the product database. Use Amazon Bedrock Prompt Management to store the LLM prompts for each of the three LLM calls. Orchestrate the three LLM calls in a sequential workflow by using Amazon Bedrock Flows.

Incorrect. Amazon Q Business is a low-code no-code (LCNC) GenAI assistant that you can use to connect enterprise data and applications. Prompt Management provides lifecycle control for prompts. Flows is a visual orchestration service that you can use to chain multiple LLM calls. Amazon Q Business includes capabilities to automate multi-step processes. However, Amazon Q Business is not suitable for this scenario. You must manage Amazon Q Business prompts and conversational flows within Amazon Q Business itself. Amazon Q Business prompts and conversational flows cannot be version-controlled through Prompt Management. Therefore, Amazon Q Business is not suitable for this scenario.

Learn more about [Amazon Q Business](#).

Learn more about [Prompt Management](#).

Learn more about [Flows](#).

26/75

A GenAI developer is designing a tool for an Amazon Bedrock agent. The tool provides sophisticated financial risk analysis by performing the following two key functions.

1. The tool loads a 10 GB complex proprietary risk model into memory upon initialization.
2. The tool maintains a persistent, long-lived WebSocket connection to a third-party service to receive real-time market data streams.

The agent will invoke the tool frequently to answer user queries. The solution must ensure that the risk model is not reloaded for each invocation and that the data stream connection is stable and always available.

Which approach will meet these requirements with the LEAST operational overhead?

A

Deploy the tool as a containerized application on Amazon ECS. Use the AWS Fargate launch type.

Correct. This approach provides the most suitable architecture for a complex, stateful, and long-running tool. Amazon ECS on Fargate can run persistent containerized applications without server management. With this approach, the large risk model can be loaded into memory when the container starts. Therefore, this approach avoids unacceptable latency on each call. Additionally, this approach supports the required long-lived WebSocket connection for the real-time data stream.

Learn more about [ECS on Fargate](#).

Learn more about [architectural patterns for Amazon Bedrock agent tools](#).

B

Package the tool and risk model into a container image. Deploy the image as an AWS Lambda function.

Incorrect. Lambda is ephemeral and event-driven. Therefore, Lambda is not suitable for this scenario. The multi-gigabyte risk model would need to be loaded during every cold start. Therefore, this approach would likely exceed the quotas for Lambda initialization and execution time. This approach could cause timeouts and poor performance. Lambda functions are not designed to maintain persistent, long-lived WebSocket connections.

Learn more about [Lambda quotas](#).

C

Deploy the tool on an Amazon SageMaker Real-Time Inference endpoint. Use an initialization script to load the model.

Incorrect. SageMaker AI is designed to host large models. However, SageMaker AI is optimized specifically for ML inference. SageMaker AI has a defined API contract. SageMaker AI is not a general-purpose application hosting service. SageMaker Real-Time Inference endpoints are designed for model serving. However, real-time inference endpoints are not optimized to maintain persistent WebSocket connections to external services.

Learn more about [real-time endpoints](#).

D

Deploy the tool on Amazon EC2 instances. Use a launch template with a preconfigured AMI that includes the risk model and WebSocket configuration.

Incorrect. You can use Amazon EC2 with launch templates and AMIs to preconfigure an instance with the risk model. You can use WebSocket setup to make an MCP server run persistently. However, this approach requires lifecycle management, patching, and scaling of the EC2 instances. Therefore, this approach requires additional operational overhead.

Learn more about [EC2 launch templates](#).

27/75

A company is building a data retrieval and workflow system. The company uses an FM in Amazon Bedrock through an API call to build the system. The company wants to use AWS Step Functions in a structured way to make the FM break down each problem into a series of logical reasoning steps.

Which solution will implement this reasoning pattern in the MOST consistent way?

A

Create a state machine with ReAct pattern states. Configure each state to invoke the FM with specific prompts for observation, reasoning, and action planning. Use the built-in Step Functions error handling and retry logic for model invocations.

Correct. You can use Step Functions to define state machines that implement complex logic through a series of steps and branches. The ReAct pattern provides a structured way for an FM to systematically break down and reason through each step of a complex problem. Using built-in Step Functions error handling ensures reliability in model invocation.

Learn more about [how to invoke an FM by using Step Functions](#).

Learn more about [state machines](#).

B

Create a state machine that implements chain-of-thought reasoning with explicit prompt templates for each step. Use Choice states to dynamically adjust the reasoning path based on intermediate responses.

Incorrect. This solution implements sequential reasoning steps. However, using Choice states to dynamically adjust the reasoning path can cause inconsistent problem-solving approaches across different scenarios. Dynamic branching is not suitable to maintain a predictable, structured decomposition of complex problems. This solution does not provide a consistent framework to break down problems into logical steps that you can apply across multiple domains.

Learn more about [Choice states in Step Functions](#).

C

Create a state machine that attempts to parallelize all reasoning steps simultaneously. Configure concurrent FM invocations to process different aspects of the problem and aggregate results in a final state.

Incorrect. Attempting to parallelize all reasoning steps simultaneously cannot provide structured, sequential reasoning. Complex problem-solving often requires each step to build upon the insights and conclusions from previous steps. Step Functions supports parallel execution. However, using Step Functions for all reasoning steps would break the logical flow that you need for effective problem-solving.

Learn more about [Step Functions parallel workflows](#).

D

Create a state machine that uses the ReAct pattern by passing all previous reasoning steps as context in each FM invocation. Store the complete conversation history in the Step Functions execution state and include the entire history with each new prompt.

Incorrect. Passing the complete conversation history with each model invocation is inefficient and could exceed model context limits. This solution does not properly separate the reasoning steps into distinct states. This solution could lead to performance issues as the conversation history grows.

Learn more about [how to invoke an FM by using Step Functions](#).

Learn more about [state machines](#).

28/75

An online subscription video streaming company has a chat-based AI assistant that helps users by answering support queries. The assistant can handle simple single-step queries such as, "Can you help me change my subscription tier?"

However, the assistant fails to understand more complex, multi-step queries such as, "Can you help me change my subscription tier, apply a discount code, and update my payment methods?" As a result, multi-step queries often escalate to a human for resolution.

The company wants to implement agentic AI to improve the assistant's ability to handle complex queries. The solution must be able to dynamically adjust if something fails. For example, if a discount code is invalid, then the assistant should ask the user for a new code before proceeding.

Which orchestration approach will meet these requirements with the LEAST development effort?

A

Create a single agent by using Amazon Bedrock Agents. Define an OpenAPI schema for each task. Use AWS Lambda functions to implement tools for each task. For example, implement a "BillingTool" to handle subscription tier changes. Write an agent prompt that defines the agent persona and available tools.

Incorrect. Agents can help build GenAI agents that can plan, reason, and perform tasks. Agents are suitable for focused single tasks. Using a single agent configuration is difficult to control and test because each task might require a different model configuration. This approach does not provide reliable orchestration, such as "if this, then that" logic and error handling.

Learn more about [Agents](#).

B

Use AWS Step Functions and define a workflow. Start the workflow with a "choice" state that determines which workflow is needed based on the incoming request. Configure each task in the workflow as a separate AWS Lambda function that calls an Amazon Bedrock model with specific prompts and configurations for that task. Include retry logic to handle errors.

Incorrect. Step Functions is a serverless workflow service that can orchestrate deterministic processes. Step Functions is most suitable for well-defined, predictable workflows. This scenario requires dynamic, conversational interactions that might include unpredictable combinations of queries that would be difficult to define by using Step Functions.

Learn more about [Step Functions workflows](#).

C

Use the Strands Agents SDK. Create a specialist agent for each individual task. For example, create a "BillingAgent" to handle subscription tier changes. Deploy the agents as AWS Lambda functions. Register the specialized agents in Strands. Configure the orchestration flow so user requests route through the agents in the correct sequence.

Correct. Strands Agents is a framework that can compose and orchestrate multiple specialized GenAI agents. Strands Agents can streamline the orchestration of multi-agent patterns that would be too difficult or cumbersome to implement through a single agent.

Learn more about [Strands Agents](#).

Learn more about [how to use the Strands Agents SDK](#).

D

Use Amazon EventBridge Pipes. Create an Amazon API Gateway endpoint to accept the initial request. Route the request to an AWS Lambda function that uses an FM in Amazon Bedrock to understand the request. Based on the request, route to a series of Lambda functions that can individually handle each task.

Incorrect. EventBridge Pipes connects event sources to targets. Pipes are designed to flow in a single direction. An example flow is source to enrichment to target. Pipes do not include built-in branching, decision-making capabilities, or conditional execution. This approach is not suitable for the types of queries in this scenario.

Learn more about [EventBridge Pipes](#).

29/75

A company fine-tunes a Meta Llama model by using proprietary training data in Amazon SageMaker AI. The company stores model weights in Hugging Face format. The company wants to import the model into Amazon Bedrock. The model files include Safetensors weights, configuration files, and tokenizer files. The model files are 45 GB in total size. The company needs a solution to provide a specific level of throughput for production workloads.

Which solution will meet these requirements with the LEAST operational overhead?

A

Use Amazon Bedrock Custom Model Import to import the model files from Amazon S3. Deploy the model by using Amazon Bedrock Provisioned Throughput.

Correct. Custom Model Import supports importing Llama models in Hugging Face format from Amazon S3. For example, the Hugging Face format can include Safetensors weights, config.json files, and tokenizer files. After importing the model, the company can purchase Provisioned Throughput to provide dedicated compute capacity and throughput for production workloads. This approach meets both the import and throughput requirements.

Learn more about [using Custom Model Import to import customized models](#).

Learn more about [purchasing Provisioned Throughput for custom models](#).

B

Use Amazon Bedrock Custom Model Import to import the Hugging Face format model files from Amazon S3. Deploy the model by using on-demand inference.

Incorrect. Custom Model Import can import Hugging Face format models from Amazon S3. However, on-demand inference does not provide guaranteed throughput. The company specifically requires a specific level of throughput for production workloads. On-demand inference operates on a pay-per-use basis without capacity guarantees.

Learn more about [how to deploy custom models for on-demand inference](#).

C

Convert the Hugging Face model to Amazon Bedrock format. Use Amazon Bedrock fine-tuning to recreate the model by using the training data.

Incorrect. You do not need to convert Hugging Face models to Amazon Bedrock format. Custom Model Import directly supports Hugging Face format models. Additionally, recreating the model through fine-tuning would require access to the original training data. This approach would be unnecessarily complex because the fine-tuned model already exists.

Learn more about [supported model architectures for Custom Model Import](#).

D

Deploy the model directly on SageMaker AI endpoints. Integrate the model with Amazon Bedrock by using custom Amazon API Gateway configurations.

Incorrect. This approach does not use the Custom Model Import feature. This approach would require complex custom integration work. Custom Model Import is specifically designed to import models from SageMaker AI and other environments. Therefore, you do not need custom API Gateway configurations.

Learn more about [using Custom Model Import to import customized models](#).

30/75

A startup company is building a general-purpose generative AI (GenAI) assistant to handle customer questions for a variety of use cases. The company builds the GenAI assistant by using Amazon Bedrock FMs.

The company wants to implement a query-routing mechanism based on the complexity of the query. Simple queries should route to a small Meta Llama model. Complex queries should route to a large Anthropic Claude model. For example, complex queries require generating creative responses or in-depth explanations. The solution must be scalable and able to maintain low-latency responses.

Which solution will meet these requirements?

A

Configure an Amazon Bedrock intelligent prompt router for the Llama and Claude models. Set a fallback model with a reliable baseline. Test the router by using different response quality differences between the models. Use an AWS Lambda function to invoke the configured Amazon Bedrock router by using the query.

Incorrect. You can use Amazon Bedrock intelligent prompt routing to route requests to different models based on the predicted response quality of each model. The intelligent prompt routing feature supports routing only between models from the same model

families. Therefore, you cannot use this feature to route between Llama and Claude models.

Learn more about [Amazon Bedrock intelligent prompt routing](#).

B

Use an AWS Lambda function to invoke a small Amazon Bedrock model by using the query. In the system prompt, instruct the model to determine if the query is complex and reply only if the query is too complex. When the model response returns to the Lambda function, inspect the response. If the response is too complex, invoke a larger model by using the same query.

Correct. Model cascading through sequential model invocation provides an efficient way to handle query complexity. First, you can use a small model to determine complexity and then invoke the larger model only when necessary. This solution optimizes both cost and performance. The smaller model quickly handles simple queries. Complex queries get the full capabilities of a larger model.

Learn more about [Amazon Bedrock pricing](#).

C

Create an Amazon Bedrock knowledge base to store query patterns and complexity indicators. Configure an AWS Lambda function to first check the knowledge base to determine query complexity and then route to the appropriate model based on the complexity assessment.

Incorrect. You can use Amazon Bedrock Knowledge Bases to retrieve relevant information from your data sources to augment model prompts. Knowledge bases cannot analyze query complexity or route between different models. This solution would not efficiently determine query complexity.

Learn more about [Knowledge Bases](#).

D

Create an AWS Step Functions workflow that first invokes an AWS Lambda function. Configure the function to analyze query length, keyword presence, and structural complexity by using regex patterns. Configure the workflow to route the query to either the small Llama model or the large Claude model based on predefined complexity thresholds.

Incorrect. Using Step Functions with regex patterns for complexity analysis is too simplistic to accurately determine query complexity. Regex patterns are text-matching patterns that you can use to search, match, or manipulate strings of text based on specific rules. This solution would miss semantic complexity that cannot be detected

through simple pattern matching. This solution would require ongoing maintenance of regex patterns.

Learn more about [Step Functions](#).

31/75

A financial services company creates a multi-account generative AI (GenAI) development environment by using Amazon Bedrock FMs.

Employees must authenticate through the corporate Microsoft Active Directory. Employees should only be able to access FMs based on their department assignments. The company requires consistent permissions across AWS accounts. The company requires control over which models each department can use. The solution must ensure Regional resilience for the authentication service.

Which solution will meet these requirements with the LEAST operational overhead?

A

Create IAM users in each AWS account that match the corporate Active Directory usernames. Establish cross-account roles with department-specific permissions that include condition keys for Amazon Bedrock model access.

Incorrect. Creating individual IAM users that mirror Active Directory violates AWS best practices by not using federation. This solution creates long-term credentials, not temporary credentials. This solution requires additional operational overhead. This solution does not provide Regional resilience.

Learn more about [federation best practices](#).

B

Configure federation between Active Directory and Amazon Cognito. Create identity pools for each department. Use AWS STS to assume roles with specific Amazon Bedrock model access permissions in each account.

Incorrect. Amazon Cognito can federate with Active Directory. However, this federation is primarily designed for customer-facing applications. This federation is not designed for workforce identity management across multiple AWS accounts. This solution does not provide a centralized way to manage permissions consistently across accounts. This solution requires additional operational overhead.

Learn more about [Amazon Cognito](#).

C

Configure SAML 2.0 federation directly by using IAM in each AWS account. Create IAM roles with trust policies for each department that include the SAML provider. Use condition keys to restrict model access based on department attributes.

Incorrect. This solution correctly uses SAML federation. However, you must maintain separate SAML configurations in each AWS account. This solution requires additional operational overhead. This solution does not provide a centralized way to manage permissions consistently across multiple accounts. Additionally, this solution makes consistent permission management difficult as workloads scale.

Learn more about [SAML 2.0 federation](#).

D

Configure AWS IAM Identity Center as the identity federation service and connect to Active Directory. Create permission sets with model access policies that map to departments. Configure multi-account access with Regional failover.

Correct. IAM Identity Center provides centralized identity federation with Active Directory. IAM Identity Center provides consistent permissions across multiple accounts through permission sets. This solution supports department-based access control to AWS resources including Amazon Bedrock models. This solution supports Regional failover for resilience.

Learn more about [IAM Identity Center](#).

32/75

An airline company uses AWS Organizations to manage multiple AWS accounts. The company wants to use generative AI (GenAI) and Amazon Bedrock to enhance a computerized maintenance management system (CMMS). The company has an existing identity provider (IdP) based on Microsoft Entra ID. The company needs to ensure that only authorized employees can access Amazon Bedrock based on their job roles. The solution must provide centralized access control and integration with the existing IdP.

Which combination of steps will meet these requirements? (Select TWO.)

A

Configure AWS IAM Identity Center with Microsoft Entra ID as an external IdP. Use custom permission sets to control access to Amazon Bedrock.

Correct. You can use IAM Identity Center as a centralized way to manage access to multiple AWS accounts and applications. IAM Identity Center supports federation with

external IdPs, including SCIM or Entra ID through SAML. This step provides centralized access control. This step integrates with the company's existing Entra ID system. This step provides support to enforce job role–based access to Amazon Bedrock.

Learn more about [identity federation](#).

B

Create IAM roles for each job function. Implement cross-account access using AssumeRole API calls from the company's on-premises applications to access Amazon Bedrock.

Incorrect. IAM roles can define fine-grained permissions. You can use AssumeRole for cross-account delegation. However, this step requires custom integration from on-premises applications. This step does not use the company's existing Entra ID IdP. This step does not provide a mechanism for integration with an external IdP.

Learn more about [IAM roles](#).

Learn more about [AssumeRole](#).

C

Create Amazon Cognito user pools to authenticate users. Create Amazon Cognito identity pools to provide temporary AWS credentials with appropriate permissions to access Amazon Bedrock.

Incorrect. Amazon Cognito provides managed user pools for authentication. Amazon Cognito provides identity pools for temporary AWS credentials. Amazon Cognito is useful for new or standalone identity management. However, the company already has Entra ID as the IdP. This step does not provide a mechanism for integration with an external IdP.

Learn more about [user pools](#).

Learn more about [identity pools](#).

D

Set up SAML-based federation between Entra ID and IAM. Create IAM roles mapped to Entra ID groups with appropriate permissions to access Amazon Bedrock.

Correct. IAM federation allows Entra ID users to assume IAM roles by using SAML. You can map IAM roles to Entra ID groups to enforce job role–based access policies. IAM SAML-based federation between Entra ID and IAM is a well-established pattern for enterprise identity federation. This step provides direct integration with the existing IdP. This step ensures that only authorized employees can access Amazon Bedrock.

Learn more about [how to configure SAML with Entra ID and IAM Identity Center](#).

E

Configure Amazon API Gateway as a proxy in front of Amazon Bedrock. Create custom authorization logic tied to Entra ID tokens.

Incorrect. To use API Gateway with custom authorization logic, you must build and maintain a proxy layer that validates Entra ID tokens and forwards requests to Amazon Bedrock. This step does not provide a mechanism for seamless integration with the external IdP.

Learn more about [API Gateway](#).

33/75

A company has a mobile app for users to record short videos. On the app, users can apply proprietary video and audio codecs to enhance the videos locally. The company wants to add features to summarize content and generate transcripts. The company wants features to detect objects and identify celebrities in the videos.

Which solution will meet these requirements with the LEAST operational overhead?

A

Use Amazon S3 PutObject to upload videos to Amazon S3. Create an S3 event notification that invokes an AWS Step Functions state machine. Set up the state machine to orchestrate processing by using AWS Lambda functions. Use Amazon Rekognition for object detection and celebrity recognition. Use an Amazon Bedrock FM for summarization and transcription.

Incorrect. Amazon S3 PutObject API operations require granting direct IAM permissions to users or applications. However, this approach violates the principle of least privilege. S3 Event Notifications cannot directly invoke Step Functions. You can use Lambda as an intermediary. However, this solution requires additional operational overhead to create and manage the functions.

Learn more about [S3 presigned URLs](#).

Learn more about [S3 Event Notifications and targets](#).

Learn more about [AWS service SDK integration with Step Functions](#).

B

Use Amazon S3 PutObject to upload videos to Amazon S3. Create an S3 event notification that invokes an AWS Lambda function. Configure the

function to process videos in parallel. Use AWS Step Functions for error handling and retries. Use Amazon Rekognition for object detection and celebrity recognition. Use Amazon Bedrock FMs to generate summaries and transcripts.

Incorrect. Amazon S3 PutObject API operations require granting direct IAM permissions to users or applications. However, this approach violates the principle of least privilege. Implementing each processing step as separate Lambda functions creates additional operational overhead. Managing multiple Lambda functions requires additional development effort. The solution relies solely on Amazon Bedrock FMs for all tasks. However, Amazon Rekognition is a specialized computer vision service that is more suitable for celebrity recognition and object detection in videos.

Learn more about [S3 presigned URLs](#).

Learn more about [event bus targets](#).

Learn more about [AWS service SDK integration with Step Functions](#).

Learn more about [supported models in Amazon Bedrock](#).

C

Use an Amazon S3 presigned URL to upload videos to Amazon S3. Configure Amazon S3 to send events to Amazon EventBridge. Create an EventBridge rule that invokes an AWS Step Functions state machine. Set up the state machine to orchestrate the processing steps by directly calling AWS service APIs. Use Amazon Rekognition for object detection and celebrity recognition. Use Amazon Bedrock FMs to generate summaries and transcripts.

Correct. This solution implements secure video uploads by using S3 presigned URLs. This solution follows the principle of least privilege. EventBridge is a serverless event bus service that efficiently routes S3 events to Step Functions for workflow orchestration. Step Functions has direct service integration that eliminates the need for intermediate Lambda functions. Therefore, this solution reduces operational overhead. This solution uses Amazon Rekognition for video analysis tasks including celebrity recognition and object detection. This solution uses Amazon Bedrock FMs for content summarization and transcript generation. Therefore, this solution maximizes the use of managed capabilities and minimizes custom code requirements.

Learn more about [S3 presigned URLs](#).

Learn more about [event bus targets](#).

Learn more about [AWS service SDK integration with Step Functions](#).

Learn more about [supported models in Amazon Bedrock](#).

Learn more about [Amazon Rekognition celebrity recognition](#).

D

Use an Amazon S3 presigned URL to upload videos to Amazon S3. Create an S3 event notification that invokes a Bedrock Data Automation (BDA) blueprint to orchestrate the processing steps. Use Amazon Rekognition for object detection and celebrity recognition. Use Amazon Bedrock FMs to generate summaries and transcripts.

Incorrect. Using S3 presigned URLs for secure uploads is the most suitable for this scenario. However, BDA has limitations for complex media processing workflows. S3 Event Notifications cannot directly trigger BDA blueprints. BDA has restricted FM options. This solution requires additional custom integration work and might not support all the required video processing capabilities through the BDA blueprint framework.

Learn more about [S3 presigned URLs](#).

Learn more about [S3 Event Notifications and targets](#).

Learn more about [BDA](#).

34/75

A company is developing an AI agent by using Amazon Bedrock AgentCore Runtime. The agent needs to authenticate users from an existing Microsoft Entra ID environment. Users must access the agent securely by using corporate credentials. The company wants to implement OpenID Connect (OIDC) integration. The OIDC integration must validate tokens from the company's identity provider (IdP) and allow access only to users with valid corporate credentials.

Which solution will meet these requirements with the LEAST operational overhead?

A

Configure AgentCore Identity with Microsoft as an inbound provider. Set the allowed audiences to the application ID from Microsoft Entra ID.

Correct. AgentCore Identity supports Microsoft Entra ID as an inbound IdP for OIDC authentication. For setup, you must configure the discovery URL to the Microsoft v2.0 OIDC metadata endpoint. Then, you set the allowed audiences to match the application ID from the Microsoft Entra ID application registration. This built-in integration eliminates the need for additional infrastructure components or custom code. Therefore, this solution provides direct OIDC authentication with the least operational overhead.

Learn more about [Microsoft IdP configuration in AgentCore Identity](#).

B

Create an Amazon Cognito user pool with Microsoft Entra ID as a SAML IdP. Configure AgentCore Runtime to use Amazon Cognito for authentication.

Incorrect. Amazon Cognito can integrate with Microsoft Entra ID. However, this solution requires the configuration and maintenance of a separate Amazon Cognito user pool. Therefore, this solution requires additional operational overhead.

Learn more about [Amazon Cognito user pools](#).

C

Set up AWS IAM Identity Center with Microsoft Entra ID as an external IdP. Configure AgentCore Runtime to use IAM roles for authentication.

Incorrect. With IAM Identity Center, you must configure external IdP mappings and IAM roles. Therefore, this solution requires additional operational overhead compared to using AgentCore Identity. Additionally, IAM Identity Center is not designed to be an authentication mechanism for AgentCore Runtime.

Learn more about [IAM Identity Center](#).

D

Deploy an Amazon API Gateway REST API with a custom AWS Lambda authorizer. Configure the authorizer to validate Microsoft Entra ID tokens and forward authenticated requests to AgentCore Runtime.

Incorrect. To implement a custom Lambda authorizer, you must develop, test, and maintain custom code for token validation. Additionally, you must manage API Gateway configurations. This solution requires additional operational overhead compared to using AgentCore Identity.

Learn more about [AgentCore Runtime authentication](#).

35/75

A company is developing an application by using Amazon Bedrock. Multiple development teams will access the application across different departments. The company uses AWS Organizations with OUs for each department.

The company must enforce least privilege access to FMs and provide role-based access control (RBAC) for each department. The solution must integrate with the company's existing enterprise identity provider (IdP) for centralized governance. Only

approved development teams should be allowed to invoke specific models. The solution must provide an auditable access trail.

Which solution will meet these requirements MOST securely?

A

Configure AWS IAM Identity Center with SAML federation from the enterprise IdP. Create permissions sets that restrict Amazon Bedrock model invocation for each OU. Create an SCP that prevents unauthorized actions outside of approved models.

Correct. IAM Identity Center provides centralized workforce access with support for SAML federation to enterprise IdPs. You can configure permissions sets that enforce least privilege IAM policies to scope access to specific Amazon Bedrock model actions for each department. You can apply SCPs at the OU level to provide an organizational guardrail that blocks non-approved actions. This solution enforces RBAC, least privilege, and centralized governance.

Learn more about [external IdPs](#).

Learn more about [SCPs](#).

B

Create identity-based IAM policies with model-specific conditions and Amazon Bedrock Guardrails ARNs. Set up Amazon EventBridge rules for anomaly detection and violation alerts. Use AWS CloudTrail to audit Amazon Bedrock API calls.

Incorrect. IAM identity-based policies can use conditions to restrict access. Examples of conditions include model IDs or guardrail ARNs. EventBridge rules and CloudTrail logs provide monitoring and alerting for anomalous activity. However, this solution does not provide centralized federation with the enterprise IdP or a scalable RBAC design across OUs.

Learn more about [identity-based and resource-based policies](#).

C

Deploy AWS PrivateLink connections for all development teams. Allow `bedrock:InvokeModel` permissions by attaching resource-based policies to Amazon Bedrock endpoints.

Incorrect. PrivateLink provides private, secure access to Amazon Bedrock without traversing the public internet. Resource-based policies can control access to endpoints. However, granting `bedrock:InvokeModel` permissions through these policies does not enforce least privilege. This solution does not enforce RBAC across departments. This

solution secures the network layer. However, this solution does not meet the access control requirements.

Learn more about [identity-based and resource-based policies](#).

D

Set up federated access through the enterprise IdP. Configure default model access by using Organizations service-linked role. Allow access to approved Amazon Bedrock FMs at the department OU level.

Incorrect. Federated access through the enterprise IdP and service-linked roles in Organizations can help you manage access. However, configuring default model access at the OU level does not follow the principle of least privilege. This solution does not provide granular controls within departments. This solution does not provide the required RBAC enforcement to restrict access to only approved development teams within each department.

Learn more about [service-linked roles](#).

36/75

A global company is building a multilingual customer service AI assistant by using Amazon Bedrock. The company has fine-tuned multiple Amazon Bedrock FMs, each for a different support topic. For example, billing-related queries must route to a model that is fine-tuned for finance. Technical troubleshooting queries must route to a model that is fine-tuned for product diagnostics. All incoming messages are processed through an Amazon API Gateway API. The company wants to build an event-driven solution that handles routing logic and is scalable.

Which solution will meet these requirements with the LEAST operational overhead?

A

Configure the API Gateway API to proxy the request to an AWS Lambda function. Configure the Lambda function to call Amazon Comprehend to detect the dominant language of the query. Route the request to the appropriate Amazon Bedrock model based on the detected language.

Incorrect. Amazon Comprehend is a natural language processing service that can identify the dominant language in text. Detecting the language is not sufficient to determine the support topic or the intent of the query.

Learn more about [Amazon Comprehend language detection](#).

Learn more about [the detect dominant language API](#).

B

Fine-tune an FM in Amazon Bedrock for topic classification. Configure the API Gateway API to proxy the request to an AWS Lambda function. Configure the Lambda function to invoke the trained FM to determine the topic. Route the request to the appropriate Amazon Bedrock model based on the detected topic.

Incorrect. Amazon Bedrock model customization provides training data to a model to enhance its performance for targeted or specific use cases. This solution introduces unnecessary model fine-tuning and increases operational overhead. Hosting fine-tuned models on Amazon Bedrock requires provisioned throughput. Provisioned throughput can add cost depending on the expected throughput.

Learn more about [fine-tuning Amazon Bedrock models](#).

C

Build and deploy a custom classification model on an inference endpoint in Amazon SageMaker AI. Configure the API Gateway API to proxy the request to an AWS Lambda function. Configure the Lambda function to call the classification inference endpoint to identify the topic of the query. Route the request to the appropriate Amazon Bedrock model based on the detected topic.

Incorrect. SageMaker AI is a managed service that you can use to build, train, and deploy ML models. To use SageMaker AI, you must train a model. You must provision and maintain an inference endpoint. Therefore, this solution requires additional operational overhead.

Learn more about [SageMaker AI model training](#).

D

Train a custom Amazon Comprehend classification model. Configure the API Gateway API to proxy the request to an AWS Lambda function. Configure the Lambda function to call Amazon Comprehend custom classification to identify the topic of the query. Route the request to the appropriate Amazon Bedrock model based on the detected topic.

Correct. You can use Amazon Comprehend custom classification to train a custom model to classify text into labels. This solution can detect user-defined categories, such as billing and technical support. Amazon Comprehend is fully managed and requires no model hosting. This solution provides dynamic routing to specialized Amazon Bedrock FMs depending on the detected support topic.

Learn more about [Amazon Comprehend custom classification](#).

37/75

A GenAI developer is implementing a real-time AI assistant application. The application uses Amazon API Gateway WebSocket APIs to stream responses from an AWS Lambda function that calls an Amazon Bedrock FM with response streaming. The application must support connection management, including session state across multi-step interactions, retries, and disconnect cleanup.

Which combination of steps will provide this functionality with MINIMAL operational overhead? (Select THREE.)

A

Create a custom domain name for the WebSocket API.

Incorrect. You can create a custom domain name for production applications. However, you do not need this configuration to enable streaming functionality between Amazon Bedrock and clients. The WebSocket API will work with the default API Gateway endpoint without a custom domain.

Learn more about [WebSocket APIs](#).

B

Configure an IAM role for the Lambda function with permissions that include `bedrock:InvokeModelWithResponseStream` and `execute-api:ManageConnections`. Add resource ARNs that include the API Gateway WebSocket API ID.

Correct. The Lambda function requires specific IAM permissions to both invoke Amazon Bedrock models with streaming and to manage WebSocket connections. The resource ARNs must include the specific API Gateway WebSocket API ID to properly scope the permissions.

Learn more about [WebSocket APIs](#).

Learn more about [how to set up a WebSocket API integration](#).

C

Configure the Lambda function to use HTTP/1.1 chunked transfer encoding to manually implement response streaming.

Incorrect. You can implement custom HTTP streaming in Lambda by using chunked transfer encoding. However, you do not need this step because Amazon Bedrock already provides built-in response streaming capabilities. This step requires complex custom code to break down model responses into chunks and manage the streaming protocol manually. Additionally, this step does not use the built-in integration between

the Amazon Bedrock streaming API and API Gateway WebSocket connections. Therefore, this step increases operational overhead and adds potential points of failure.

Learn more about [WebSocket APIs](#).

D

Set up a WebSocket API in API Gateway with route selection expressions. Integrate the API with a Lambda function that handles connection management and streams responses back to clients using the API Gateway Management API.

Correct. You need a WebSocket API in API Gateway to maintain persistent client connections. You must configure the API with the appropriate route selection expressions. The route selection expressions include \$connect, \$disconnect, and custom routes such as chat. Then, you must integrate with a Lambda function. These routes provide proper connection lifecycle handling. The Lambda function uses the API Gateway Management API to stream Amazon Bedrock responses back to clients in real time.

Learn more about [WebSocket APIs](#).

Learn more about [how to set up a WebSocket API integration](#).

E

Set up an Amazon DynamoDB table to persist active WebSocket connection IDs and session metadata. Update the table on \$connect and \$disconnect and use TTL.

Correct. API Gateway WebSocket APIs provide connection IDs but do not persist the connection IDs. For session management across multi-step interactions, you must use a database to store connection IDs and session metadata. For example, you can use DynamoDB as the database. The \$connect and \$disconnect routes can update DynamoDB when clients join or leave. This step enables retries. This step provides multiple Lambda workflows and the cleanup of stale connections.

Learn more about [WebSocket APIs](#).

Learn more about [DynamoDB WebSocket connection management](#).

F

Set up an Amazon DynamoDB table to buffer Amazon Bedrock streaming tokens for each session. Enable DynamoDB Streams to invoke a Lambda function that forwards new tokens to clients over the WebSocket connection.

Incorrect. Using DynamoDB as a token buffer introduces latency. You can store streaming tokens in DynamoDB and then use DynamoDB Streams to trigger another

Lambda function. However, the Lambda function alone can handle the request to directly stream responses to clients by using the API Gateway Management API. Therefore, this step increases operational overhead for real-time streaming.

Learn more about [DynamoDB WebSocket connection management](#).

38/75

A company is deploying a customer service AI assistant. The assistant uses Amazon API Gateway to invoke an AWS Lambda function. The function calls the Amazon Bedrock API to generate responses.

A development team performs load testing that simulates peak business hours. The development team observes occasional latency spikes and intermittent `ThrottlingException` errors from the Amazon Bedrock API. The development team must improve system reliability to handle the load-based errors. The development team must preserve the assistant's real-time responsiveness.

Which solution will meet these requirements?

A

Create an Amazon SQS queue that decouples API Gateway from the Lambda function.

Incorrect. You can use Amazon SQS to create resilient decoupled systems. However, this solution changes the architecture from synchronous to asynchronous. For a real-time customer service AI assistant, the user expects an immediate response. Introducing a queue breaks the synchronous interaction. This solution does not provide a direct path through API Gateway for the response to return to the waiting user. This solution does not meet the requirement to preserve real-time responsiveness.

Learn more about [SQS queues](#).

B

Create an AWS Step Functions workflow that manages synchronous retries with a fixed-interval strategy.

Incorrect. A fixed-interval retry strategy is not an effective way to resolve `ThrottlingException` errors. A fixed-interval retry strategy can contribute to a thundering herd problem. In a thundering herd problem, many clients retry simultaneously and worsen the throttling condition.

Learn more about [Step Functions workflows](#).

C

Configure exponential backoff with jitter in the AWS SDK used by the Lambda function. Configure per-client throttling limits in API Gateway.

Correct. Configuring exponential backoff with jitter in the AWS SDK is the recommended client-side pattern to handle transient errors such as `ThrottlingException`. Exponential backoff with jitter avoids overwhelming the API with retries. Configuring throttling limits in API Gateway is a server-side protection mechanism that manages request bursts from clients. This solution ensures that the backend services remain stable and responsive for all users. This solution improves reliability without breaking the synchronous, low-latency user experience.

Learn more about [retry behavior and exponential backoff in AWS SDKs](#).

Learn more about [throttling API requests for better throughput](#).

Learn more about [ThrottlingException](#).

D

Increase the allocated memory for the Lambda function. Place an Amazon CloudFront distribution in front of API Gateway to cache responses.

Incorrect. Increasing Lambda memory provides more CPU and RAM. However, increasing memory does not affect downstream API call limits such as those from Amazon Bedrock. CloudFront is a content delivery network that can cache static or repeatable dynamic content. CloudFront cannot cache the highly dynamic, non-repeatable responses of an AI assistant. Each response is unique to the user's prompt.

Learn more about [how to configure Lambda memory](#).

Learn more about [CloudFront distributions](#).

39/75

A company is building a contract analysis system by using intelligent document processing capabilities in Amazon Bedrock. The system uses a blueprint to extract fields, such as `AuthorizedSigner`, from legal agreements. An example of an extracted value is Mr. John Allen Doe III, Senior Legal Counsel. The company needs to split the fields into the following individual components: `TITLE`, `FIRST_NAME`, `MIDDLE_NAME`, `LAST_NAME`, `SUFFIX`, and `JOB_TITLE`.

The company will reuse the structured name format across multiple extracted fields including `AuthorizedSigner`, `WitnessName`, and `ReviewerName` in the pipeline.

Which Bedrock Data Automation (BDA) capability will meet these requirements?

A

Use extraction to map subfields directly from text by assigning aliases.

Incorrect. Extraction retrieves field values. However, extraction does not parse or split complex text strings such as names with titles and suffixes. Aliases can rename fields but not split fields.

Learn more about [extraction](#).

B

Use normalization to parse and split the value using pattern-based replacements.

Incorrect. Normalization standardizes field formats. For example, normalization can convert "NY" to "New York". Normalization does not support semantic splitting or creating structured field groups such as name components.

Learn more about [normalization](#).

C

Use transformation with a reusable custom type to split the AuthorizedSigner field into subcomponents.

Correct. Transformation can split complex fields into structured components. For example, transformation can split full names. You can use a custom type to define and reuse this structure across fields such as AuthorizedSigner or ReviewerName.

Learn more about [transformation](#).

D

Use validation to enforce required subfields, such as FIRST_NAME and LAST_NAME, and to reject malformed names.

Incorrect. Validation helps enforce constraints after transformation. For example, validation can check for null values or incorrect formats. However, validation cannot parse or restructure fields.

Learn more about [validation](#).

40/75

A real estate company needs to automate the extraction of specific fields from various utility bills in PDF format. The company manages thousands of commercial and residential properties and receives utility bills monthly. The utility bill types include

electricity, water, and gas depending on the property location. Each bill type has its own unique format and a predefined set of fields that the company needs to extract. The solution must automatically identify the bill type and extract corresponding information when bills upload to Amazon S3.

Which solution will meet these requirements with the LEAST operational overhead?

A

Use Amazon Rekognition to create an Amazon Rekognition Custom Labels model that is trained with sample images from each bill type. Use Amazon Bedrock to create three separate Bedrock Data Automation (BDA) projects, each dedicated to a specific bill type with a corresponding blueprint and field definitions. Configure an Amazon EventBridge rule to detect S3 upload events and invoke an AWS Lambda function. Configure the function to first use Custom Labels to identify the document type, and then invoke the Amazon Bedrock InvokeDataAutomationAsync API with the corresponding project to extract the fields.

Incorrect. You can use Custom Labels to train custom computer vision models to detect objects and scenes that are specific to your business needs. This solution requires additional operational overhead. You must maintain multiple BDA projects and a Custom Labels model. Using two different services for document classification (Amazon Rekognition and Amazon Bedrock) adds complexity without providing additional benefits.

Learn more about [BDA](#).

Learn more about [Custom Labels](#).

B

Use Amazon Rekognition to create an Amazon Rekognition Custom Labels model that is trained with sample images from each bill type. Configure an Amazon EventBridge rule to detect S3 upload events and invoke an AWS Lambda function. Configure the function to invoke the Custom Labels model to identify the bill type, invoke Amazon Textract AnalyzeDocument with queries to extract text from the document, and run custom Python code. Configure the Python code to parse and extract the required fields based on the identified document type.

Incorrect. You can use Custom Labels to train custom computer vision models to detect objects and scenes that are specific to your business needs. Amazon Textract AnalyzeDocument with queries can extract structured fields from scanned utility bills. However, this solution requires additional operational overhead. You must train and manage a custom Amazon Rekognition model and maintain custom Python code for field extraction. You must orchestrate workflows across Amazon Rekognition, Amazon

Textract, and Lambda. Additionally, changes in bill formats would require re-training models and updating code.

Learn more about [Custom Labels](#).

Learn more about [Amazon Textract](#).

Learn more about [AnalyzeDocument with queries](#).

C

Use Amazon Bedrock to create a single Bedrock Data Automation (BDA) project that contains multiple blueprints. Create one blueprint for each bill type, including the bill type description and fields to extract. Configure an Amazon EventBridge rule to detect S3 upload events and trigger an AWS Lambda function. Configure the function to invoke the Amazon Bedrock InvokeDataAutomationAsync API to process the document with the created project, automatically select the appropriate blueprint, and extract the corresponding fields.

Correct. BDA is a fully managed document processing service that automates the extraction of data from documents by using AI. BDA blueprints are templates that define the structure and rules to process specific document types. You can use a single project with multiple blueprints to streamline management while maintaining functionality. BDA automatically detects the blueprint to use based on the document type. BDA handles field extraction. This solution does not require custom code maintenance or multiple service orchestration.

Learn more about [BDA](#).

D

Use Amazon Bedrock to create three separate Bedrock Data Automation (BDA) projects, each dedicated to a specific bill type with a corresponding blueprint and field definitions. Configure an Amazon EventBridge rule to detect S3 upload events and invoke an AWS Lambda function. Configure the function to invoke Amazon Bedrock InvokeDataAutomationAsync API to analyze the document and automatically select the appropriate project and extract the corresponding fields.

Incorrect. BDA is a fully managed document processing service that automates the extraction of data from documents by using AI. BDA does not support automatic project selection from incoming documents. Each document must be explicitly processed against a specific project.

Learn more about [BDA](#).

41/75

A GenAI developer must use Amazon Q Business to enhance an internal content management system (CMS) for a company. The company wants users to be able to search, query, and receive AI-assisted insights from the company's data.

The company has technical documentation in the CMS and internal knowledge bases with proprietary guidelines. The company has enterprise directory services for authentication, strict role-based access controls (RBAC), and compliance and governance policies for data access.

Which combination of steps will meet these requirements with MINIMAL operational overhead? (Select TWO.)

A

Configure Amazon Q Business with data source connectors to integrate the CMS and internal knowledge bases. Use AWS IAM Identity Center authentication for RBAC.

Correct. Amazon Q Business data source connectors are managed integration points that provide secure connections to external content systems. IAM Identity Center is a centralized service to manage user access and permissions across AWS accounts. This step uses built-in connectors to integrate content securely. This step uses IAM Identity Center enterprise-grade authentication and RBAC. Therefore, this step meets the compliance requirements without custom development.

Learn more about [Amazon Q Business data sources](#).

Learn more about [IAM for Amazon Q Business](#).

B

Configure Amazon Q Business with custom Amazon API Gateway endpoints for content access. Enable SAML-based SSO through AWS IAM Identity Center. Create AWS Lambda functions that validate access permissions before serving responses.

Incorrect. You can use API Gateway to create and manage APIs. SAML-based SSO through IAM Identity Center is suitable for this scenario. However, creating custom API endpoints and validation Lambda functions requires additional operational overhead. Amazon Q Business already provides built-in access controls and content integration.

C

Configure Amazon Q Business with enterprise Active Directory integration. Create data source filters based on security groups. Create Amazon CloudWatch alarms to monitor unauthorized access attempts to knowledge bases.

Incorrect. Active Directory integration provides enterprise authentication by using existing identity systems. CloudWatch alarms provide monitoring and alerting capabilities. However, using data source filters based on security groups does not provide sufficient granular access control. Additionally, creating CloudWatch alarms requires additional operational overhead. Amazon Q Business already provides built-in monitoring features.

D

Configure Amazon Q Business data sources with automatic content synchronization and security group mapping for access control.

Correct. Amazon Q Business automatic content synchronization is a feature that updates knowledge bases from source systems in real time. Security group mapping provides automatic access control based on existing enterprise roles. This step uses built-in capabilities to maintain current content and enforce proper access controls. This step does not require additional services or custom code.

Learn more about [Amazon Q Business data sources](#).

Learn more about [IAM for Amazon Q Business](#).

E

Create custom AWS Lambda functions to process and index documentation before importing the documentation into Amazon Q Business knowledge bases.

Incorrect. Lambda is a serverless compute service that runs custom code in response to events. Creating custom Lambda functions for document processing and indexing requires additional operational overhead. Amazon Q Business already includes built-in document processing capabilities.

42/75

A retail company wants to reduce delays and manual effort in inventory replenishment. The company wants employees to be able to check inventory levels and submit inventory requests in natural language. The solution must automatically invoke internal supply chain processes with minimal ongoing maintenance. The solution must provide built-in Model Context Protocol (MCP) support. The company stores all inventory and data in Amazon Aurora.

Which solution will meet these requirements with the LEAST operational overhead?

A

Build an AI-powered chat assistant by using Amazon Lex. Create an AWS Lambda function that parses inventory requests and pushes request details

to an Amazon SQS queue. Use the queue for backend supply chain microservices that run on Amazon ECS on AWS Fargate.

Incorrect. Amazon Lex is a conversational AI service that you can use for conversational AI assistants. Amazon Lex provides natural language processing. Lambda, Amazon SQS, and ECS on Fargate can coordinate backend processing. However, this solution does not provide the built-in MCP support that you need. This solution introduces multiple components to manage, including Lambda, Amazon SQS, and Amazon ECS. Therefore, this solution requires additional operational overhead.

Learn more about [Amazon Lex](#).

Learn more about [ECS on Fargate](#).

B

Deploy an agent by using Amazon Bedrock AgentCore Runtime with Strands Agents. Configure a prebuilt MCP server to expose Aurora inventory and store data as MCP tools. Integrate the agent with a company chat assistant interface.

Correct. You can use AgentCore Runtime with Strands Agents to build AI agents with built-in MCP support. A prebuilt MCP server can directly expose Aurora inventory and store data as MCP tools. This solution eliminates the need for custom containers or API management. This solution provides natural language access to data. This solution automates supply chain actions with the least operational overhead because Amazon Bedrock handles orchestration and hosting.

Learn more about [AgentCore Runtime](#).

C

Create a REST API by using Amazon API Gateway in front of an AWS Lambda function. Configure the Lambda function to query Aurora for inventory information and invoke an AWS Step Functions workflow to handle replenishment tasks.

Incorrect. API Gateway and Lambda provide a pattern to build REST APIs. Step Functions can orchestrate workflows. However, this solution requires additional operational overhead to build and maintain the architecture. Additionally, this solution does not use MCP and requires manual API definitions and workflow coordination.

Learn more about [Lambda](#).

Learn more about [API Gateway](#).

D

Develop and host a custom MCP server on Amazon ECS on AWS Fargate. Implement MCP tools to connect to Aurora and invoke supply chain APIs. Integrate the server with an Amazon Bedrock agent.

Incorrect. A custom MCP server deployed on Amazon ECS can connect to Aurora and expose supply chain APIs as MCP tools. For this solution, you must build, deploy, and scale your own containerized MCP implementation. Therefore, this solution requires additional operational overhead compared to using prebuilt MCP support in AgentCore.

Learn more about [ECS on Fargate](#).

43/75

A company receives large merged PDF files from employees. Each PDF file includes multiple pages with distinct content types, including images and text. The images and text can be categorized into a predefined list. A GenAI developer creates an Amazon Bedrock Data Automation (BDA) project. The GenAI developer uses the BDA project in an AWS Step Functions workflow.

The GenAI developer defines custom outputs and provides relevant blueprints as expected. However, the extraction results are inconsistent. The first two pages are correct. However, most of the other pages are missed entirely. Downstream systems receive incomplete metadata.

Which combination of steps will resolve this issue with MINIMAL operational overhead? (Select TWO.)

A

Route all PDF files to the image modality.

Incorrect. You can route certain file types to specific modality types by configuring manual modality routing. Routing PDF files to a modality is a way to explicitly map how BDA processes specific content types. Routing all files to the image modality would mishandle text-heavy pages in the PDF files.

Learn more about [how to route files to modality types](#).

B

Enable PDF page splitting in the BDA project.

Correct. Enabling PDF page splitting in the BDA project provides proper processing for multipage PDF files. This built-in feature automatically handles page segmentation. Therefore, all pages are processed correctly, without requiring additional custom code or services. If this feature is not turned on, the best matching blueprint will be used for the entire PDF file.

Learn more about [how to split documents](#).

C

Disable all modalities except the text modality.

Incorrect. There are four modalities in BDA: document, image, video, and audio. You can disable a modality for a project if you do not want processing for all types of files. Disabling all modalities except the document modality will disable the image modality. You need the image modality to handle the images in this scenario. The PDF files that include images would be mishandled.

Learn more about [how to disable modality types](#).

D

Refine the blueprint names and definitions. Include only one blueprint for each content type.

Correct. You can refine blueprint names and definitions. You can use one clear blueprint for each document type. This step can improve the accuracy and consistency of document classification. This step reduces potential conflicts in document type identification. This step ensures that each page is correctly categorized. When you combine this step with document splitting, you have a comprehensive solution to process multipage PDF files with minimal operational overhead. One well-defined blueprint for each content type follows BDA best practices. Multiple blueprints of the same type in a project could lead to worse performance.

Learn more about [best practices for creating blueprints](#).

E

Refine the blueprint names and definitions. Provide multiple blueprints of the same content type.

Incorrect. You can refine blueprint names and definitions to improve the accuracy and consistency of document classification. However, providing multiple blueprints of the same document type directly goes against BDA best practices. This step could degrade PDF file processing performance.

Learn more about [best practices for creating blueprints](#).

44/75

A GenAI developer must create dynamic content filtering for a customer service chat-based AI assistant. Multiple departments in a company use the assistant. The assistant is deployed on an Amazon Nova Pro model. AWS Lambda functions handle the chat logic. The company has the following requirements:

- Stricter content policies must be enforced during business hours.

- Each department must have department-specific filtering rules.
- Amazon Bedrock Guardrails must be used for content filtering.

Which solution will meet these requirements with the LEAST operational overhead?

A

Create a single guardrail with maximum restrictions. Create an AWS Step Functions workflow that orchestrates several Lambda functions to post-process responses based on business hours and department. Store filtering rules in AWS Systems Manager Parameter Store.

Incorrect. Guardrails can provide content filtering at the model invocation level. Guardrails has built-in capabilities to manage different filtering contexts. You can create a single guardrail with maximum restrictions. You can use Step Functions with Lambda for post-processing. However, this solution requires additional operational overhead. This solution wastes tokens on responses that would later be filtered out. You must maintain multiple serverless components. A more suitable solution would be to use the built-in features of Guardrails. Filtering at the post-processing stage undermines the fundamental function and granular control of guardrails.

B

Use guardrail APIs to dynamically create and update guardrails. Create a Lambda function that checks current time and user department. Cache guardrail configurations in Amazon ElastiCache (Redis OSS).

Incorrect. The Guardrails API includes operations to manage guardrail configurations. The API operations include CreateGuardrail, UpdateGuardrail, and DeleteGuardrail. These APIs are designed for administrative operations to set up and modify guardrails. The APIs are not designed for real-time request processing. Using the APIs to dynamically create and update guardrails for each request introduces unnecessary latency and creates potential race conditions. Additionally, implementing an ElastiCache (Redis OSS) cluster to cache configurations adds complexity and cost. A more suitable solution would be to use the built-in features of Guardrails.

C

Create multiple static guardrails for each scenario. Create a function that selects the appropriate guardrail based on time and department. Store guardrail selection logic in Amazon DynamoDB.

Incorrect. You can use Guardrails to create many guardrail configurations. You can use Lambda functions to choose between configurations based on various criteria. You can create several static guardrails for each department and time combination. However, this solution requires additional operational overhead. You must update several guardrails for each policy change. Additionally, maintaining the DynamoDB table to store guardrail selection logic increases operational overhead.

D

Create multiple guardrails with tags for different business hours and department contexts. Create a Lambda function that is triggered by an Amazon EventBridge rule to select the appropriate guardrail based on time.

Correct. Guardrails supports tagging for organizational purposes and context-based filtering. EventBridge provides built-in time-based event handling capabilities. This solution effectively uses these built-in features. You can configure guardrails with appropriate tags for different contexts. You can use EventBridge to handle time-based guardrail selection automatically. This solution provides the least operational overhead by using built-in AWS services without requiring additional databases or complex orchestration.

Learn more about [how to automate building guardrails](#).

Learn more about [Guardrails](#).

45/75

A travel company is developing a new AI-powered travel recommendation application by using Amazon Bedrock Agents. Customers can use the application to input travel preferences and receive personalized itinerary suggestions. The company wants to avoid prompt injection attacks that could manipulate the FM to provide unauthorized access or bypass content filters. A GenAI developer must implement security measures to protect against prompt injection attacks.

Which combination of actions will meet these requirements? (Select THREE.)

A

Associate an Amazon Bedrock guardrail with the agent to implement content filtering and topic boundaries.

Correct. Guardrails provide built-in protection against prompt injection. Guardrails provide content filtering and enforce topic boundaries. This action validates and filters inputs before the inputs reach the FM.

Learn more about [how to block denied topics](#).

Learn more about [prompt injection](#).

B

Deploy AWS WAF in front of the application to block malicious requests before the requests reach Amazon Bedrock.

Incorrect. AWS WAF can mitigate web application vulnerabilities. However, AWS WAF is not designed to detect or prevent prompt injection attacks. AWS WAF operates at the web application layer and cannot effectively analyze the semantic content of prompts.

Learn more about [AWS WAF](#).

C

Enable the default pre-processing prompt for the Amazon Bedrock agent to evaluate if user input is safe to process.

Correct. Every agent has a default pre-processing prompt that can determine if user input is safe to use. You can enable default pre-processing prompts in Agents to add an additional layer of security. A pre-processing prompt will evaluate user input before the input reaches the FM. This action helps identify and filter potentially malicious prompts.

Learn more about [pre-processing prompts](#).

D

Create an Amazon CloudWatch anomaly detection alarm to identify unusual prompt traffic patterns.

Incorrect. CloudWatch anomaly detection provides historical metrics to flag unusual spikes or drops and can notify you about anomalies. This action is a reactive monitoring control. This action does not prevent or filter prompt injection content or constrain model behavior at inference time. This action does not actively protect against prompt injection.

Learn more about [CloudWatch anomaly detection](#).

E

Create AWS Config custom rules to audit agent configuration settings and ensure that all deployed agents have an associated guardrail policy.

Incorrect. AWS Config can evaluate resource configurations against rules to detect drift or noncompliance. You can use AWS Config rules to deploy agents with certain settings. This action can confirm if guardrails are attached. However, this action does not inspect or block malicious prompts at runtime. Using AWS Config rules does not mitigate prompt injection attacks.

Learn more about [AWS Config custom rules](#).

F

Update the system prompt to clearly define the scope of what the agent can and cannot do.

Correct. You can update the system prompt to clearly define boundaries and limitations to help prevent prompt injection. This action establishes strict behavioral constraints for the agent. This action creates a strong foundation for secure operation when combined with other protective measures. Newer models differentiate between system and user prompts. System prompts can define the scope of what an agent can and cannot do.

Learn more about [system prompts](#).

Learn more about [prompt injection](#).

46/75

A software company is launching an AI assistant by using Amazon Bedrock. The AI assistant will help users troubleshoot issues by quickly exploring logs and documentation, and then recommending actions for remediation.

A GenAI developer wants to set up Amazon Bedrock Guardrails. The GenAI developer wants to add protection against SQL injection. The GenAI developer wants to add a post-generation factuality check to prevent recommendations based on inaccurate information.

Which combination of actions will meet these requirements? (Select TWO.)

A

Add a prompt attack filter.

Correct. Guardrails provide safeguards for GenAI applications. You can configure guardrails to protect against prompt attacks, including jailbreaks and prompt injection.

Learn more about [prompt attack filters in Guardrails](#).

B

Add a misconduct content filter.

Incorrect. Guardrails provide safeguards for GenAI applications. Misconduct content filters in guardrails prevent the GenAI application from engaging in criminal activity or providing information that could be potentially harmful. This filter does not protect against SQL injection.

Learn more about [content filters in Guardrails](#).

Learn more about [prompt attack filters in Guardrails](#).

C

Add a contextual grounding check.

Correct. Guardrails support contextual grounding checks to detect and filter hallucinations in model responses.

Learn more about [grounding checks in Guardrails](#).

D

Use an LLM-as-a-judge job to check for factuality.

Incorrect. Amazon Bedrock evaluations support LLM-as-a-judge jobs. These jobs use an LLM to score the responses from another model. You can use LLM-as-a-judge jobs to check for factuality. However, LLM-as-a-judge jobs run as batch jobs or offline assessments to compare model performance. These evaluations do not integrate into runtime guardrails for live post-generation checks. Therefore, this approach does not meet the requirement to enforce factuality during customer interactions.

Learn more about [Amazon Bedrock evaluations and LLM-as-a-judge](#).

Learn more about [grounding checks in Guardrails](#).

E

Add a denied topic to prevent the chatbot from discussing SQL queries.

Incorrect. Guardrails provide safeguards for GenAI applications. Guardrails support denied topics. Denied topics check that the prompt and completion do not engage in a particular topic. For SQL queries, this approach would prevent the chatbot from generating any SQL queries, including those that are benign. Additionally, this approach would prevent the users from exploring logs.

Learn more about [denied topics in Guardrails](#).

Learn more about [prompt attack filters in Guardrails](#).

47/75

A legal company is developing an AI-powered contract analysis system by using Amazon Bedrock. The system must analyze legal documents to provide responses and recommendations for complex legal questions. All responses must come from an authoritative legal source. All responses must cite the authoritative legal source. The system must prevent the generation of inappropriate legal advice and filter sensitive personal information.

The legal documents are stored across multiple data sources, including Amazon S3 and internal databases. A GenAI developer has already created an Amazon Bedrock

knowledge base with vector embeddings from the company's legal document repositories.

Which solution will meet these requirements with the LEAST operational overhead?

A

Enable RAG functionality through the RetrieveAndGenerate API. Implement content filtering logic within AWS Lambda functions attached to the model endpoint.

Incorrect. The RetrieveAndGenerate API provides RAG functionality with integrated citation tracking. However, implementing custom content filtering through Lambda functions requires significant development effort and ongoing maintenance. This solution lacks the built-in AI safety features of Guardrails. You would need custom filtering logic to handle complex legal content analysis, topic detection, and sensitive information identification. Therefore, this solution increases system complexity and adds potential compliance risks.

Learn more about [the Amazon Bedrock RetrieveAndGenerate API](#).

B

Enable document retrieval through the Retrieve API. Attach Amazon Bedrock Guardrails policies to the FM endpoint. Include content filters and topic restrictions in the policies.

Incorrect. Guardrails provide content filtering capabilities. However, the Retrieve API returns only document chunks without generating responses. This solution requires additional operational overhead to combine retrieved content with model inference and response generation. This solution does not support generating comprehensive recommendations for complex legal questions.

Learn more about [the Amazon Bedrock RetrieveAndGenerate API](#).

Learn more about [Guardrails](#).

C

Enable RAG functionality through the RetrieveAndGenerate API. Attach Amazon Bedrock Guardrails policies to the FM endpoint. Include content filters and topic restrictions in the policies.

Correct. This solution combines vector embeddings for semantic search capabilities with the RetrieveAndGenerate API. This solution provides document retrieval with response generation in a single call. Therefore, this solution automatically provides citations that are essential for legal compliance. Guardrails integration provides built-in content filtering and topic restriction capabilities that are specifically designed for AI

safety. The system can prevent inappropriate legal advice generation and filter sensitive information without custom development or additional API orchestration.

Learn more about [Knowledge Bases](#).

Learn more about [the Amazon Bedrock RetrieveAndGenerate API](#).

Learn more about [Guardrails](#).

D

Enable document retrieval through the Retrieve API. Implement content filtering logic within AWS Lambda functions attached to the model endpoint.

Incorrect. The Retrieve API returns only document chunks without response generation. Therefore, this solution requires additional implementation for complete RAG functionality. Additionally, implementing custom content filtering through Lambda functions does not provide the built-in AI safety features of Guardrails.

Learn more about [the Amazon Bedrock Retrieve API](#).

48/75

A company is building a generative AI (GenAI) application that uses Amazon Bedrock. The company uses AWS Organizations for its AWS accounts. The company's AWS accounts are divided into two OUs: development and production. The company uses IAM roles to grant the application access to Amazon Bedrock across both development and production accounts.

Access to the Amazon Bedrock API from production accounts in the organization must not traverse the public internet. A GenAI developer must enforce this restriction, regardless of IAM role configuration or application behavior.

Which solution will meet this requirement?

A

Create an interface VPC endpoint for Amazon Bedrock in each production VPC where the application that requires access is deployed. Create an SCP that denies Amazon Bedrock actions unless the request comes through an approved VPC endpoint. Attach the SCP to the production OU.

Correct. Interface VPC endpoints provide private connectivity to Amazon Bedrock within the AWS network. SCPs are an organization policy that you can use to manage permissions across accounts in an organization. The SCP enforces this control regardless of the IAM configuration in the accounts. This solution ensures that Amazon Bedrock actions are denied unless the requests originate from an approved VPC endpoint.

Learn more about [Amazon Bedrock interface VPC endpoints](#).

Learn more about [SCPs](#).

B

Create an interface VPC endpoint for Amazon Bedrock in each production VPC where the application that requires access is deployed. Create an IAM policy in each production account that denies Amazon Bedrock actions unless the request comes through an approved VPC endpoint. Attach the policy to the IAM roles in the account.

Incorrect. Using interface VPC endpoints provides private access to Amazon Bedrock. However, IAM policies are attached to individual roles. Account administrators can modify or misconfigure IAM policies. This solution does not provide organization-wide enforcement. This solution does not ensure compliance if IAM roles are changed or improperly assigned.

Learn more about [IAM policies](#).

C

Create a NAT gateway in each production VPC where the application that requires access is deployed. Set the NAT gateway connectivity type to private. Update the route tables associated with the subnets to route Amazon Bedrock traffic to the NAT gateway.

Incorrect. NAT gateways are a managed NAT service that allows network access from private subnets to the internet or other AWS services. NAT gateways do not allow inbound access from the internet. NAT gateways provide outbound connectivity from private subnets to other VPCs or on-premises networks. However, NAT gateways cannot route traffic to public AWS services unless the gateways are exposed through private connectivity mechanisms like interface VPC endpoints. An example of a public AWS service is Amazon Bedrock. This solution does not enforce organization-wide controls or block direct access to public Amazon Bedrock endpoints.

Learn more about [NAT gateways](#).

D

Create an interface VPC endpoint for Amazon Bedrock in each production VPC where the application that requires access is deployed. Attach endpoint policies to the endpoints to allow access from only the approved application.

Incorrect. VPC endpoint policies are resource-based policies that you can attach to VPC endpoints. Endpoint policies can enforce which principals can access the service behind the endpoint. However, endpoint policies control access to only the endpoint itself. Endpoint policies do not control access to the service if accessed through the public

endpoint. This solution does not prevent an application from bypassing the endpoint and calling the public Amazon Bedrock API directly.

Learn more about [VPC endpoint policies](#).

49/75

A GenAI developer is building a serverless application. The application uses AWS Lambda functions that are deployed in private subnets to process sensitive customer data. The Lambda functions need to invoke Amazon Bedrock FMs for AI-powered analytics. All API communication must remain within the AWS private network without internet exposure.

The GenAI developer tests the Lambda function. The Lambda function consistently times out when attempting to call Amazon Bedrock APIs. The Lambda function has proper IAM permissions for Amazon Bedrock access.

Which solution will resolve this connectivity issue?

A

Configure Amazon Route 53 private hosted zones for the Amazon Bedrock service endpoints that resolve to the internal VPC CIDR range. Enable custom DNS resolution in the VPC settings. Ensure that the Lambda execution role has Route 53 Resolver permissions for private DNS queries.

Incorrect. Route 53 private hosted zones can resolve custom domain names within a VPC. However, private hosted zones cannot automatically create network connectivity to external AWS services without underlying VPC endpoints. You can use private hosted zones for internal resources. However, private hosted zones cannot route traffic to AWS service endpoints unless VPC endpoints are already configured. Additionally, the resolver permissions are unnecessary because DNS resolution does not establish the required private network path to Amazon Bedrock.

Learn more about [private DNS](#).

Learn more about [private hosted zones](#).

B

Deploy dedicated proxy Lambda functions in the private subnets that handle all Amazon Bedrock API communications. Configure the proxy functions with enhanced networking permissions. Set up an internal Application Load Balancer between the application Lambda functions and the proxy Lambda functions to distribute API calls.

Incorrect. Proxy Lambda functions are a valid architectural pattern for some use cases. Lambda to Lambda communication within a VPC is possible without additional networking infrastructure. However, proxy functions would have the same connectivity issue with Amazon Bedrock for this scenario. The proxy functions would need internet access or VPC endpoints. This solution adds latency without solving the fundamental private network requirement for Amazon Bedrock access.

Learn more about [how to connect inbound interface VPC endpoints for Lambda](#).

C

Create an AWS Client VPN endpoint in the VPC. Configure the Lambda functions to route Amazon Bedrock API calls through the VPN connection. Add a route table entry to direct traffic to Amazon Bedrock through the Client VPN endpoint.

Incorrect. Client VPN is designed for secure remote access to AWS and on-premises networks. You cannot use Client VPN to connect to AWS services such as Amazon Bedrock. Lambda functions in a VPC do not need VPN connectivity to access AWS services securely. This solution does not meet the requirement for private API access.

Learn more about [Client VPN](#).

D

Create interface VPC endpoints for the Amazon Bedrock Runtime service in the VPC. Ensure that the endpoints are associated with the private subnets where Lambda functions are deployed. Verify that security groups allow HTTPS traffic on port 443 between Lambda and the VPC endpoints.

Correct. Interface VPC endpoints allow Lambda functions in private subnets to access Amazon Bedrock APIs without internet connectivity. The Runtime service requires the `com.amazonaws.region.bedrock-runtime` endpoint for model invocation operations. Security groups must allow outbound HTTPS traffic on port 443 from Lambda to the VPC endpoint. The endpoint's security group must allow inbound HTTPS traffic. Private DNS is enabled by default, so existing Lambda code can continue to work without modification. AWS automatically routes API calls through the VPC endpoint.

Learn more about [VPC endpoints for Amazon Bedrock](#).

50/75

A financial services company needs to access Amazon Bedrock from an application that runs in a private subnet. For security and compliance reasons, the application must make Amazon Bedrock API calls without traversing the public internet. The solution needs to enable logging through VPC Flow Logs and AWS CloudTrail. The solution must operate without internet gateway access.

Which solution will meet these requirements?

A

Configure an Application Load Balancer (ALB) in a public subnet to proxy Amazon Bedrock API requests from the private subnet to the Amazon Bedrock public endpoints.

Incorrect. ALBs can proxy requests. However, placing an ALB in a public subnet sends Amazon Bedrock calls to public endpoints over the internet. Therefore, this solution does not meet the requirement to avoid traversing the public internet.

Learn more about [ALBs](#).

B

Create a VPC interface endpoint for the Amazon Bedrock Runtime service. Configure the private subnet route tables to direct Amazon Bedrock API traffic through the endpoint.

Correct. You can create a VPC endpoint for the Runtime service to provide secure, private access to Amazon Bedrock directly through the AWS network. This solution provides API calls without internet access. This solution maintains security through private networking. This solution supports logging through VPC Flow Logs and CloudTrail.

Learn more about [Amazon Bedrock VPC interface endpoints](#).

Learn more about [interface VPC endpoints](#).

C

Deploy a NAT gateway in a public subnet. Configure the private subnet route tables to direct Amazon Bedrock API traffic through the NAT gateway.

Incorrect. A NAT gateway is a managed AWS service that allows instances in private subnets to access the internet while preventing inbound connections from the internet. A NAT gateway would allow private subnet resources to access Amazon Bedrock. However, this solution requires internet gateway access and routes traffic over the public internet. Therefore, this solution does not meet the requirement to avoid traversing the public internet.

Learn more about [NAT gateway](#).

D

Create a VPC gateway endpoint for the Amazon Bedrock Runtime service. Configure the private subnet route tables to direct Amazon Bedrock API traffic through the endpoint.

Incorrect. A VPC gateway endpoint is a gateway that you specify as a target for a route in your route table. The traffic must be destined to either Amazon S3 or Amazon DynamoDB. Runtime service does not support VPC gateway endpoints. VPC gateway endpoints are only available for Amazon S3 and DynamoDB.

Learn more about [gateway endpoints](#).

51/75

A GenAI developer builds an AI-powered customer service chat application for a company. The GenAI developer uses Amazon Bedrock to build the application. The application processes natural language inputs from users and generates real-time responses. The responses reference users' personally identifiable information (PII).

The GenAI developer must configure the application to handle PII appropriately. According to internal privacy policies, PII should not be inadvertently exposed during or after inference. Prompt data cannot be retained longer than necessary. The handling of PII must comply with defined storage and retention policies.

Which solution will meet these requirements?

A

Use Amazon Macie to scan stored user prompts and responses in Amazon S3 for PII. Apply Amazon Comprehend for PII detection on stored logs to identify PII post-inference. Configure S3 Lifecycle policies to transition or expire objects after the retention period. Use AWS Config rules to monitor lifecycle policies that apply to S3 buckets and enforce remediation if a bucket is not compliant.

Incorrect. In this solution, all actions occur after the data is already written to Amazon S3. Therefore, this solution could expose PII during inference. Additionally, PII could be logged in raw form. Lifecycle policies and AWS Config rules enforce storage compliance. However, this solution does not ensure that PII is redacted before or during model interaction. This solution is primarily reactive and does not meet the requirements.

Learn more about [Amazon Comprehend PII detection](#).

Learn more about [AWS Config rules](#).

B

Use Amazon Bedrock Guardrails to mask PII in user prompts before inference and redact PII from generated responses. Store prompts and model responses in Amazon S3. Use Amazon Macie to automatically classify

and alert on PII stored in Amazon S3. Configure S3 Lifecycle policies to enforce data retention limits.

Correct. Guardrails can mask PII in user prompts before the prompts reach the model. Therefore, guardrails can reduce privacy risk at inference. Guardrails ensure that unredacted PII does not persist in logs or stored outputs. The application can still present user-appropriate responses in real time. Macie provides automated sensitive data discovery in Amazon S3. S3 Lifecycle policies enforce retention limits to meet retention requirements.

Learn more about [guardrail sensitive data filters](#).

Learn more about [sensitive data discovery by using Macie](#).

Learn more about [data retention through S3 Lifecycle management](#).

C

Use Amazon Bedrock Guardrails to mask PII in user prompts before inference and redact PII from generated responses. Store prompts and model responses in Amazon S3. Use Amazon Macie to scan stored data for PII and trigger alerts for compliance violations. Apply an S3 Lifecycle policy to move data for archival storage. Configure IAM roles to control access to PII.

Incorrect. Guardrails can mask PII before and after inference. However, this solution moves data to archival storage rather than deleting the data. Therefore, this solution might violate retention requirements. Macie can classify sensitive data post-inference. However, this solution does not ensure the removal of sensitive data in compliance with retention policies.

Learn more about [Guardrails](#).

D

Use Amazon Bedrock Guardrails to filter PII from prompts and responses. Store interaction logs in Amazon S3 with server-side encryption using AWS KMS. Enable AWS CloudTrail to log Amazon Bedrock API usage and apply Amazon Macie to generate compliance reports. Configure S3 Lifecycle policies to enforce data retention limits.

Incorrect. Guardrails can prevent the generation of harmful or sensitive outputs. This solution uses Amazon S3 with AWS KMS for secure storage and CloudTrail for API visibility. This solution uses Macie for compliance scans. However, redacting PII from responses prevents the application from delivering user details. Additionally, compliance scans and S3 Lifecycle policies alone do not ensure the deletion of PII as required by retention policies.

Learn more about [Guardrails data filters](#).

Learn more about [data retention through S3 Lifecycle management](#).

52/75

A large company is using Amazon Bedrock. The company wants to limit access to FMs to specific AWS and Anthropic models within designated development accounts. The company strictly prohibits third-party marketplace models. The company requires comprehensive logging of all model interactions for auditing purposes.

The company uses AWS Organizations and AWS IAM Identity Center for account and user management. A security team must implement the solution while maintaining operational efficiency.

Which combination of steps will meet these requirements with MINIMAL operational overhead? (Select TWO.)

A

Create an RCP that denies access to marketplace models and unapproved built-in models. Apply the policy to the designated development accounts in the organization. Use a condition block to allow only approved AWS and Anthropic model IDs for `bedrock:InvokeModel*` actions.

Incorrect. RCPs are an organization policy that you can use to manage the maximum permissions for resources in an organization. RCPs do not support all AWS services. RCPs do not support Amazon Bedrock. You cannot use an RCP to control access to Amazon Bedrock models.

Learn more about [RCPs](#).

Learn more about [how Amazon Bedrock works with IAM](#).

Learn more about [security in Amazon Bedrock](#).

B

Create an SCP that denies `bedrock:InvokeModel*` actions for unapproved or marketplace models by using the `bedrock:ModelID` condition key. Apply the policy to the root of the organization. Enable Amazon Bedrock model invocation logging.

Correct. SCPs provide organization-wide preventive controls. SCPs can effectively deny access to marketplace models across all accounts. You can scope `bedrock:InvokeModel*` actions to only approved AWS and Anthropic model IDs. You can deny all other marketplace models. This step provides organization-wide governance with minimal operational overhead. You can enable Amazon Bedrock model invocation logging to capture all model interactions for auditing purposes, including inputs and outputs.

Learn more about [how Amazon Bedrock works with IAM](#).

Learn more about [SCPs](#).

C

Deploy AWS CloudFormation StackSets to provision standardized IAM roles across development accounts. Create IAM roles that allow access to only approved AWS and Anthropic models. Enable AWS CloudTrail logging for all Amazon Bedrock interactions.

Incorrect. StackSets can deploy standardized IAM roles across multiple accounts. CloudTrail can capture API calls for Amazon Bedrock. However, this step is redundant and increases operational overhead. You must create and manage IAM roles in each account. The company is already using IAM Identity Center to manage permissions. Therefore, this step requires additional management overhead and could conflict with existing IAM Identity Center configurations.

Learn more about [IAM Identity Center](#).

Learn more about [StackSets](#).

D

Create a permission set in IAM Identity Center that allows `bedrock:InvokeModel*` actions only for specific AWS and Anthropic model ARNs by using IAM policy conditions. Apply the permission set to designated development accounts.

Correct. IAM Identity Center permission sets provide a centralized way to manage user access to AWS services across accounts. You can scope Amazon Bedrock permissions to only specific AWS and Anthropic model ARNs by using condition keys. This step provides control over which models developers can invoke. This step uses the existing IAM Identity Center implementation to manage permissions. Therefore, this step minimizes operational overhead.

Learn more about [IAM Identity Center permission sets](#).

E

Create a custom AWS Config rule to detect when Amazon Bedrock model invocations include unapproved model IDs. Configure Amazon EventBridge to capture the noncompliant findings and invoke an AWS Lambda function. Configure the function to notify the security team and remove offending IAM permissions.

Incorrect. You can create AWS Config custom rules to evaluate Amazon Bedrock usage against compliance requirements. EventBridge can trigger a Lambda function. However, this approach is reactive rather than preventative. This approach detects violations only after they occur. The company requires preventative enforcement.

Learn more about [custom rules](#).

Learn more about [EventBridge and Lambda](#).

53/75

A GenAI developer builds an application by using Amazon Bedrock. The application summarizes customer feedback from multiple media platforms. Currently, the GenAI developer stores all prompt inputs and generated summaries in Amazon S3 for auditing and analytics.

Because of new copyright and compliance policies, the GenAI developer must implement the following governance mechanisms:

- Maintain an auditable trail for prompt data sources.
- Log FM usage for auditing purposes.
- Automatically track prompt lineage and model I/O metadata.

Which solution will meet these requirements?

A

Store all prompts and summaries in Amazon DynamoDB with metadata fields for model identifiers and timestamps. Enable DynamoDB Streams to send lineage events to Amazon EventBridge for auditing.

Incorrect. DynamoDB can store metadata including model identifiers and timestamps. DynamoDB Streams can push change events to EventBridge. This solution provides a custom audit pipeline. However, you must build lineage tracking manually. Therefore, this solution does not meet the requirement for automatic governance of prompt lineage. This solution does not provide integrated logging of Amazon Bedrock usage.

Learn more about [DynamoDB Streams](#).

Learn more about [EventBridge and DynamoDB Streams](#).

B

Create AWS Config rules to monitor Amazon S3 buckets that contain prompts and summaries. Create conformance packs to enforce encryption and retention policies. Export rule evaluations to Amazon Athena for auditing.

Incorrect. AWS Config conformance packs are collections of AWS Config rules and remediation actions. Conformance packs help enforce compliance policies including encryption and retention. Conformance packs are useful for resource governance. However, conformance packs operate at the infrastructure level. Conformance packs do not automatically track prompt lineage, record Amazon Bedrock model usage, or provide I/O metadata.

Learn more about [conformance packs](#).

C

Enable Amazon CloudWatch Logs for Amazon Bedrock model invocations. Set up custom Amazon S3 object tags to capture metadata for each stored prompt and summary. Manage lineage information by using tag policies.

Incorrect. You can use CloudWatch Logs to capture operational metrics and logs that relate to Amazon Bedrock activity. S3 object tags can store metadata values of prompts and summaries. However, this solution relies on manual tagging policies and log management. This solution does not provide consistent or automated governance for prompt lineage or comprehensive auditing of model usage.

Learn more about [how to monitor model invocations by using CloudWatch Logs](#).

Learn more about [S3 object tags](#).

D

Set up Amazon S3 server access logging for all prompt and summary objects. Enable AWS CloudTrail to record Amazon Bedrock API calls. Configure Amazon Bedrock Prompt Management to track template versions and lineage.

Correct. S3 server access logging provides detailed records of requests that are made to S3 buckets. S3 server access logging provides an auditable trail for prompt data sources. CloudTrail captures Amazon Bedrock API activity including model invocations for usage auditing. Prompt Management supports reusable templates, versioning, and lineage tracking for prompts. Therefore, you can use this solution to catalog and review I/O metadata.

Learn more about [S3 server access logging](#).

Learn more about [how to monitor Amazon Bedrock API calls](#).

Learn more about [Prompt Management](#).

54/75

A financial services company is building a chat-based AI assistant to simulate conversations with customers. The company wants to fine-tune an Amazon Bedrock FM on transcripts of real customer support chat conversations. The customer support transcripts are stored in an unstructured format. For regulatory compliance, the company must track where the fine-tuning dataset originated. The company must track how the dataset has been transformed. The company must ensure that only governed and approved data is used in the fine-tuning process.

Which solution will meet these requirements with the LEAST operational effort?

A

Create an Amazon S3 bucket with appropriate governance controls. Store the raw customer support transcripts in the bucket. Scan the bucket by using an AWS Glue crawler. Store the metadata in AWS Glue Data Catalog. Prepare the data for fine-tuning by using AWS Glue ETL jobs. Transform the data to JSONL in the Amazon Bedrock Converse API format. Store the curated data in the original S3 bucket. Create a fine-tuning job in Amazon Bedrock that references the S3 path to the curated and approved data.

Correct. Amazon S3 is an object storage service that you can use to host data lakes. You can use an AWS Glue crawler to crawl a data source and infer the schema and metadata. Then, you can store the data in Data Catalog. AWS Glue ETL provides serverless extract, transform, and load functionality. Amazon Bedrock is a fully managed service that provides access to FMs. This solution tracks and governs data lineage and data sources through AWS Glue. This solution makes the data available to fine-tune a model through Amazon Bedrock. Additionally, all services are serverless. Therefore, this solution requires the least operational effort.

Learn more about [Amazon S3](#).

Learn more about [Data Catalog](#).

Learn more about [AWS Glue crawlers](#).

Learn more about [Amazon Bedrock fine-tuning jobs](#).

B

Create an Amazon S3 bucket with appropriate governance controls. Store the raw customer support transcripts in the bucket. Scan the S3 bucket by using an AWS Glue crawler. Store the metadata in AWS Glue Data Catalog. Prepare the data for fine-tuning by using Amazon EMR with Apache Spark jobs. Transform the data to JSONL in the Amazon Bedrock Converse API format. Store the curated data back in the original S3 bucket. Create a fine-tuning job in Amazon Bedrock that references the S3 path to the curated and approved data.

Incorrect. Amazon S3 is an object storage service that you can use to host data lakes. You can use an AWS Glue crawler to crawl a data source and infer the schema and metadata. Then, you can store the data in Data Catalog. Amazon EMR is a managed service that provides big data processing. You can use Apache Spark or Apache Hadoop with Amazon EMR. Amazon Bedrock is a fully managed service that provides access to FMs. Amazon EMR can transform the data in this scenario. However, Amazon EMR does not provide built-in support to track data lineage. Amazon EMR does not integrate with Data Catalog. For this solution, you must implement the components manually. Therefore, this solution requires additional operational effort.

Learn more about [Data Catalog](#).

Learn more about [AWS Glue crawlers](#).

Learn more about [Amazon Bedrock fine-tuning jobs](#).

Learn more about [Amazon EMR](#).

C

Create an Amazon S3 bucket with appropriate governance controls. Store the raw customer support transcripts in the bucket. Use Amazon Athena to run SQL queries on the raw data and prepare the data for fine-tuning. Use Athena to export the curated results back in the original S3 bucket. Create a fine-tuning job in Amazon Bedrock that references the S3 path to the curated and approved data.

Incorrect. Amazon S3 is an object storage service that you can use to host data lakes. Athena can run SQL queries directly on data in Amazon S3. You do not need to load the data into a database first. Amazon Bedrock is a fully managed service that provides access to FMs. SQL queries are a suitable way to transform structured data. However, the company has unstructured text of customer support transcripts. Therefore, this solution would require additional operational effort to perform queries and transformation. Additionally, this solution alone would not meet the lineage and compliance requirements. You would need to add a service for lineage and compliance such as Data Catalog.

Learn more about [Athena](#).

Learn more about [Amazon Bedrock fine-tuning jobs](#).

D

Store the raw customer support transcripts in an Amazon S3 bucket and reference the transcripts directly in the Amazon Bedrock fine-tuning job.

Incorrect. Amazon S3 is an object storage service that you can use to host data lakes. Amazon Bedrock is a fully managed service that provides access to FMs. This solution does not address the governance, lineage, and compliance requirements. Additionally, the raw customer support transcripts would likely not be formatted correctly for an Amazon Bedrock fine-tuning job. You must perform additional transformations to make the transcripts the correct format.

Learn more about [Amazon Bedrock fine-tuning jobs](#).

55/75

A healthcare company is using Amazon Bedrock to run a customer service AI assistant. A GenAI developer must use Amazon Bedrock Guardrails to ensure compliance with the following guidelines:

- The assistant must comply with healthcare regulations regarding patient privacy.
- The assistant should not expose personally identifiable information (PII).
- The assistant must avoid discussing unauthorized medical topics.
- The assistant should not provide incorrect medical information.

Which combination of configurations will meet these requirements? (Select TWO.)

A

Configure content filters with high strength for the misconduct category. Create word filters to block unauthorized medical topics.

Incorrect. Guardrails provide content filters with configurable strengths. You can use word filters to block specific keywords or phrases. However, misconduct filters and generic word filters do not provide sufficient compliance for healthcare use cases. This configuration cannot redact PII or restrict specific medical topics.

Learn more about [harmful content filters](#).

B

Configure prompt attack filters to prevent users from bypassing safety mechanisms. Create word filters to detect and block PII.

Incorrect. Guardrails support prompt attack filters that can detect malicious input. For example, attempting to override instructions is malicious input. Guardrails provide word filters to block keywords. This configuration is useful for general safety. However, word filters cannot reliably protect PII.

Learn more about [contextual grounding checks](#).

C

Create sensitive information filters to detect and redact PII in user inputs and model responses. Set up denied topics for unauthorized medical topics.

Correct. You can use sensitive information filters in Guardrails to detect and redact PII. You can use denied topics to block model engagement on specific subjects. This configuration complies with healthcare regulations by protecting PII. This configuration prevents the discussion of unauthorized medical topics.

Learn more about [sensitive information filters](#).

D

Enable automated reasoning checks that validate that model responses adhere to healthcare regulations. Implement contextual grounding to prevent hallucinations.

Correct. You can use automated guardrail checks and contextual grounding checks in Guardrails to ensure that responses are accurate and not hallucinated. These checks verify outputs against retrieval sources. Automated reasoning checks are useful when you need to demonstrate the factual basis for an LLM's response in regulated industries such as healthcare. This configuration helps reduce the risk of incorrect medical information. This configuration helps ensure compliance with factual accuracy requirements.

Learn more about [automated reasoning checks](#).

E

Create reusable templates in Amazon Bedrock Prompt Management. Configure the templates with strict system instructions for healthcare scenarios. Configure denied topics to block unauthorized medical topics.

Incorrect. Prompt Management supports reusable templates and system instructions. However, this configuration does not enforce runtime PII detection or prevent hallucination. This configuration does not meet the compliance and privacy requirements.

Learn more about [Prompt Management](#)

56/75

A financial services company wants to develop a generative AI (GenAI) application that provides personalized recommendations to users. A GenAI developer uses Amazon SageMaker AI to build the application. The application must process complex financial data and comply with regulations.

The GenAI developer must implement a comprehensive fairness evaluation framework. The framework must detect subtle biases in model responses across different socioeconomic groups, age brackets, and geographic regions.

Which solution will meet these requirements with the LEAST operational overhead?

A

Deploy real-time performance monitoring by using Amazon CloudWatch. Implement role-based access controls (RBAC). Use Amazon Comprehend for sentiment analysis of model outputs.

Incorrect. CloudWatch monitoring and RBAC are important for application security. However, this solution does not address the requirement for fairness evaluation across demographic groups. Amazon Comprehend has sentiment analysis capabilities. However, Amazon Comprehend cannot effectively measure demographic biases in model outputs.

Learn more about [Amazon Comprehend sentiment analysis](#).

B

Create custom evaluation datasets across demographic groups. Implement batch transform jobs for testing. Configure automatic model re-training when bias is detected.

Incorrect. You can create custom evaluation datasets and implement batch transform jobs to help test model fairness. However, this solution requires additional operational overhead to build the components. Additionally, automatic model re-training when bias is detected could introduce new biases without proper evaluation frameworks in place.

Learn more about [batch transform](#).

C

Use SageMaker Model Monitor to continuously evaluate model predictions for bias drift. Set up automated alerts when bias metrics exceed thresholds.

Incorrect. Model Monitor can detect data drift. However, Model Monitor is not designed for comprehensive fairness evaluation across multiple demographic dimensions. This

solution requires significant customization to detect subtle biases across socioeconomic groups, age brackets, and geographic regions. Therefore, this solution requires additional operational overhead.

Learn more about [Model Monitor](#).

D

Use SageMaker Clarify for automatic bias detection. Use FM evaluations (FMEval) with CrowS-Pairs datasets. Configure Amazon CloudWatch metrics for demographic disparity monitoring.

Correct. Clarify provides comprehensive bias detection capabilities across different demographic groups. Clarify can automatically identify potential unfairness in model responses. You can use FMEval with CrowS-Pairs datasets to test for stereotypical biases. CloudWatch metrics provide ongoing monitoring of demographic disparities. This solution provides a robust framework to detect and measure bias across multiple dimensions by using built-in capabilities of Clarify.

Learn more about [prompt stereotyping by using Clarify](#).

Learn more about [CloudWatch metrics](#).

Learn more about [FMEval](#)

57/75

A financial services company is using Amazon SageMaker AI training jobs to fine-tune a custom FM. The company uses the FM for various sensitive use cases including fraud detection, analytics, and document analysis. The fine-tuning jobs invoke automatically when a user uploads new datasets in designated Amazon S3 buckets.

The company must implement responsible AI practices to ensure compliance with industry regulations. A GenAI developer must ensure that all FM training datasets for new model releases are encrypted at rest by using AWS KMS customer managed keys. Datasets in buckets that use default AWS managed keys or that are not encrypted must be rejected and not used for model fine-tuning.

The GenAI developer must validate that all existing S3 buckets comply with AI best practices for customer managed key encryption.

Which solution will meet these requirements?

A

Use Amazon Macie to scan all training datasets in S3 buckets and flag unencrypted dataset objects. Generate reports on any objects that are not

encrypted with customer managed keys. Mark all buckets that Macie flags as noncompliant.

Incorrect. You can use Macie to retrieve information and analyze objects in S3 buckets. However, you would primarily use Macie for sensitive data discovery such as personally identifiable information (PII). Macie is not the most suitable solution for continuous compliance enforcement.

Learn more about [how to use Macie to analyze encrypted S3 objects](#).

B

Create an AWS Lambda function that runs on an Amazon EventBridge schedule. Configure the function to run periodic Amazon Athena queries. Run the queries on AWS CloudTrail logs. Configure the queries to identify PutObject events on S3 buckets that do not have an x-amz-server-side-encryption: aws:kms header and mark those buckets as noncompliant.

Incorrect. This solution provides visibility into potentially noncompliant PutObject events. This solution provides checks that occur without manual intervention. However, this solution cannot ensure customer managed key enforcement. CloudTrail logs API calls. However, CloudTrail does not log bucket encryption settings or applied customer managed key policies.

Learn more about [CloudTrail](#).

Learn more about [PutObject](#).

C

Create an AWS Config custom rule using the AWS rule development kit (RDK). Set up the custom rule to check the existing S3 buckets for FM training data. Configure the rule to determine if the buckets use server-side encryption with AWS KMS (SSE-KMS) and a customer managed key. Configure the rule to apply remediation and automatically detect noncompliant buckets.

Correct. This solution uses compliance-as-code to set continuous automation checks through AWS Config. AWS Config supports many automatic remediation options, including AWS Systems Manager Session Manager. This solution checks and enforces the use of customer managed keys. Therefore, this solution complies with responsible AI best practices.

Learn more about [AWS RDK](#).

Learn more about [AWS Config rules and GenAI](#).

Learn more about [AWS Config RDK](#).

D

Create Amazon EventBridge rules for PutObject events for dataset S3 buckets. Set up an AWS Step Functions workflow. Configure one state to run an AWS Lambda function that parses events and verifies customer managed key encryption. Configure a Choice state that evaluates encryption status. Configure a third state that either invokes SageMaker AI training jobs for customer managed key-encrypted files or publishes Amazon SNS notifications to skip training for noncompliant datasets.

Incorrect. The Step Functions workflow directly ties compliance checks with the model pipeline. Therefore, this solution prevents you from using unencrypted data in future SageMaker training. The Step Functions workflow can orchestrate all the steps. However, this solution applies only to new individual files, not to the overall bucket encryption policy. Therefore, this solution does not ensure continuous compliance across all datasets or buckets.

Learn more about [Step Functions](#).

Learn more about [how to build an ML workflow by using SageMaker jobs and Step Functions](#).

58/75

A financial services company is developing a customer-facing AI assistant to help with customer questions. The AI assistant will use Amazon Bedrock. The company requires the prevention of harmful content, protection against sensitive data leakage, and automatic blocking of illegal content.

Which implementation approach will meet these requirements with the LEAST operational overhead?

A

Use Amazon Bedrock Guardrails with topic filters for harmful content detection. Configure Amazon CloudWatch to monitor sensitive data access patterns. Implement content filters to automatically block illegal content and log events for auditing.

Incorrect. Guardrails already provide managed filters to detect harmful, sensitive, and illegal content by using built-in logging capabilities. CloudWatch can monitor sensitive data access. However, this approach increases operational overhead. Amazon Bedrock has built-in features that can meet these requirements more efficiently.

Learn more about [Guardrails](#).

Learn more about [content filters](#).

B

Use Amazon Bedrock Guardrails with generic safe completion settings for harmful content. Configure Amazon API Gateway with AWS WAF to block illegal content. Create an Amazon CloudWatch alarm that invokes an AWS Lambda function to identify sensitive data leakage.

Incorrect. You can use guardrails to configure safety thresholds to block harmful or illegal content. API Gateway with AWS WAF is effective for blocking malicious requests at the network layer. CloudWatch alarms can invoke a Lambda function for remediation. However, AWS WAF cannot detect nuanced harmful responses. Additionally, combining AWS WAF and CloudWatch duplicates controls.

Learn more about [how to use AWS WAF to protect API Gateway](#).

C

Use Amazon Bedrock Guardrails content filters for harmful content detection. Use Amazon Bedrock prompts with system instructions to prevent sensitive data leakage. Create an AWS Lambda function to monitor and block illegal content patterns from Amazon Bedrock.

Incorrect. Guardrails provide managed filters that can block harmful, sensitive, and illegal content without additional service configurations. System instructions are a technique to guide model behavior. However, system instructions are not a reliable enforcement mechanism. Lambda functions can implement custom monitoring. However, to create and maintain the functions increases operational overhead.

Learn more about [Guardrails](#).

Learn more about [content filters](#).

D

Use Amazon Bedrock Guardrails content filters for harmful content detection. Set up Amazon Bedrock word filters to identify potentially illegal content. Implement Amazon Bedrock contextual grounding checks to prevent unauthorized sensitive data leakage.

Correct. Guardrails provide built-in support to detect harmful content, protect sensitive data, and block illegal content. You can configure guardrails with content filters for harmful responses and word filters for prohibited terms. You can configure guardrails with contextual grounding checks to reduce hallucinations and prevent the leakage of

sensitive data. This managed configuration meets all compliance requirements and minimizes operational overhead by using built-in features.

Learn more about [Guardrails](#).

Learn more about [content filters](#).

59/75

A GenAI developer is designing a customer-facing application for a company. The GenAI developer uses an FM that is deployed on Amazon SageMaker AI. The application will deliver automated advisory services to customers. The company requires AI governance controls to ensure compliance with internal policies and external regulations. For compliance, the application must meet the following requirements:

- Enforce content restrictions and usage policies during model inference.
- Ensure that all model limitations and compliance risks are documented and centrally accessible.
- Automate compliance checks on model outputs by using programmatic workflows to flag violations.

Which combination of steps will meet these requirements? (Select TWO.)

A

Use Amazon Bedrock Guardrails with customized denied topics and blocked keywords based on usage policies. Create an Amazon EventBridge rule to invoke an AWS Lambda function for post-inference policy validation.

Correct. You can use guardrails to define denied topics and blocked keywords to enforce content during model inference. You can use Guardrails for models that you deploy on SageMaker AI. This step reduces the risk of policy violations in AI-generated responses. To validate model outputs after inference, you can configure an application workflow or an EventBridge rule to trigger the function based on inference-related events. This step provides output filtering and enforcement during inference. After inference, this step provides automated compliance workflows programmatically.

Learn more about [Guardrails](#).

Learn more about [how to integrate EventBridge with Amazon Bedrock](#).

B

Integrate SageMaker Clarify with SageMaker Model Monitor to gather metrics across inference endpoints. Export the reports to Amazon S3 for policy compliance audits and governance insights.

Incorrect. You can use Clarify and Model Monitor for bias detection, explainability, and drift analysis. However, these tools focus primarily on model training and inference monitoring. These tools do not enforce policy compliance or document model limitations. Exporting to Amazon S3 is helpful for audits. However, this approach does not provide the automated post-inference validation workflows that you need.

Learn more about [Clarify](#).

Learn more about [Model Monitor](#).

C

Create model cards by using SageMaker Model Registry. Use Amazon EventBridge to trigger compliance workflows that invoke AWS Lambda functions to validate policies at runtime.

Correct. You can use SageMaker Model Cards to provide structured documentation of FM metadata. The metadata can include limitations, training context, and intended usage. You can use EventBridge to capture model registration events and invoke Lambda to enforce compliance policies. This step meets the requirement to centrally document compliance risks and model limitations.

Learn more about [Model Cards](#).

Learn more about [events that SageMaker AI sends to EventBridge](#).

D

Apply IAM policy conditions and AWS CloudTrail Insights on SageMaker AI model invocations. Detect runtime violations through log analysis pipelines.

Incorrect. IAM and CloudTrail are effective for access control and visibility into account activity. However, IAM policy conditions and CloudTrail do not provide content-level filtering or model output validation workflows. IAM policies cannot inspect or filter FM responses. CloudTrail logs are useful for auditing. However, CloudTrail logs cannot automate compliance enforcement or document model limitations.

Learn more about [IAM policy conditions](#).

Learn more about [CloudTrail Insights](#).

E

Use AWS Glue Data Catalog to tag model training datasets with compliance metadata tags, including training provenance and policy classification for downstream documentation.

Incorrect. Data Catalog is a metadata repository that you can use to organize and discover datasets. Data Catalog is not a metadata repository for models. Tagging helps with dataset-level lineage and classification. However, tagging does not directly support policy enforcement, content filtering, or compliance validation workflows.

Learn more about [Data Catalog](#).

60/75

A large ecommerce company is developing a customer service AI-powered chat assistant by using Amazon Bedrock. The company needs the chat assistant to be able to support both simple product questions and more complex reasoning tasks. Simple product questions are 80% of the queries. Complex reasoning tasks are 20% of the queries. The solution must be cost-effective and require minimal operational overhead while maintaining response quality.

Which solution will meet these requirements?

A

Configure Amazon Bedrock with provisioned throughput for an Anthropic Claude Sonnet model to handle all queries. Implement response caching using Amazon ElastiCache (Redis OSS) to store frequent responses with 24-hour TTL. Use Amazon CloudWatch metrics to monitor cache hit rates and adjust provisioned capacity based on traffic patterns.

Incorrect. Provisioned throughput with Claude Sonnet provides consistent performance. However, this solution uses a high-capability model for all queries, including simple product questions. ElastiCache can improve response times for repeated queries. However, using Sonnet for all traffic is not cost-effective because 80% of the queries are simple product questions that a more efficient model could handle. This solution does not optimize resource usage based on query complexity.

Learn more about [provisioned throughput](#).

B

Use Amazon Bedrock intelligent prompt routing. Use Anthropic Claude Haiku as the primary model. Use Claude Sonnet as the fallback model. Configure the routing policy to send simple queries to Claude Haiku and to send complex queries to Claude Sonnet.

Correct. Amazon Bedrock intelligent prompt routing provides dynamic model selection based on query characteristics. Using Claude Haiku for simple product questions provides faster, more cost-effective processing. Routing complex reasoning tasks to Claude Sonnet ensures high-quality responses for sophisticated queries. This solution

optimizes both cost and performance by matching query complexity to model capabilities.

Learn more about [model selection in Amazon Bedrock](#).

Learn more about [intelligent prompt routing](#).

C

Create a custom caching layer using Amazon DynamoDB with TTL enabled for response storage. Use AWS Lambda with semantic similarity scoring to pre-process queries and match against cached responses with a 0.95 similarity threshold. Invoke Anthropic Claude Sonnet through Amazon Bedrock for queries that do not match cached responses.

Incorrect. DynamoDB with TTL provides managed caching. Lambda can provide similarity matching. However, this solution requires custom development. Implementing semantic similarity scoring and managing cached responses requires additional operational overhead. This solution does not effectively address the different complexity levels of queries. This solution might not maintain response quality for complex reasoning tasks.

Learn more about [Amazon Bedrock pricing](#).

Learn more about [using Lambda with DynamoDB](#).

D

Use a single Anthropic Claude Sonnet model deployment with optimized prompt templates for both simple and complex queries. Implement context windowing to limit token usage to 4K tokens for simple queries and to allow up to 8K tokens for complex reasoning tasks. Use Amazon Bedrock Knowledge Bases to store and retrieve common product information.

Incorrect. Using Claude Sonnet exclusively with context windowing does not optimize for the simple product question majority. Token limiting can help control costs. However, using a high-capability model for all queries is less efficient than routing simpler queries to a more cost-effective model. This solution does not reduce costs by matching model capabilities to query complexity.

Learn more about [Amazon Bedrock model optimization](#).

61/75

A financial services company implemented a RAG-based document analysis system. The company used Amazon Bedrock Knowledge Bases to process quarterly reports

and financial statements. The system currently processes thousands of documents daily.

The system uses a fixed-chunking approach that splits documents into 1,000-token segments without considering content boundaries. Recently, retrieval costs increased because of token inefficiency. Many chunks contain partial or fragmented financial information. The company needs to reduce retrieval costs while maintaining accuracy and comprehension of complex financial data.

Which solution will meet these requirements?

A

Use an Anthropic Claude model in Amazon Bedrock with the maximum context window. Implement recursive summarization before ingestion for long documents.

Incorrect. You can use the maximum context window in a Claude model to process more content at once. Recursive summarization is a technique to condense long documents. However, this solution does not optimize token usage. This solution does not address the issue of inefficient and fragmented fixed-size chunks. Using maximum context windows for all requests can lead to unnecessary token consumption and higher costs. Recursive summarization could cause information loss and increase total token usage through multiple model calls.

Learn more about [TokenUsage](#).

Learn more about [model parameters](#).

B

Use Amazon Nova in Amazon Bedrock for chunk embedding and retrieval pre-processing. Use Anthropic Claude in Amazon Bedrock for complex summarization and analysis tasks.

Incorrect. Amazon Nova Lite is optimized for lower-cost inference. Model orchestration across Nova and Claude can reduce overall compute costs. However, switching between models can increase total token usage because of redundant processing and context recreation. Retrieval will still require excessive tokens. This solution does not address the fundamental token efficiency issues within each model call.

Learn more about [Amazon Nova models](#).

C

Reduce the chunk size in Knowledge Bases. Use Amazon Bedrock Agents with automated retrieval from the knowledge base.

Incorrect. Agents can automate knowledge base queries. However, agents do not inherently optimize token usage. Automated retrieval might increase token consumption by including unnecessary context. Smaller chunks can improve retrieval precision. However, smaller chunks could reduce the accuracy of the model.

Learn more about [contextual retrieval](#).

Learn more about [Agents](#).

D

Enable semantic chunking in Knowledge Bases. Optimize the maximum tokens parameter.

Correct. Semantic chunking can group content into chunks that follow natural boundaries. For example, natural boundaries include paragraphs, tables, or sections. Semantic chunking provides meaningful content divisions while minimizing token waste. Semantic chunking can prevent the fragmentation of financial data across chunks. You can adjust retrieval token limits to ensure that only the most relevant information passes to the model. This solution lowers costs while preserving comprehension. You can optimize the maximum tokens parameter to achieve the right balance between retrieval accuracy and token usage.

Learn more about [Knowledge Bases](#).

Learn more about [semantic chunking](#).

Learn more about [contextual retrieval](#).

62/75

A retail company is implementing a generative AI (GenAI) powered customer service system by using Amazon Bedrock. The system must handle product inquiries and answer various customer questions through the company's website. The system will have significant traffic load level variations throughout the year. The system must access the company's extensive product catalog and customer data. The company wants to improve performance while maintaining response quality and accuracy.

Which combination of configurations will meet these requirements? (Select TWO.)

A

Create Amazon Bedrock knowledge bases with RAG that incorporate the product catalog and customer data. Remove outdated product data regularly.

Correct. You can create knowledge bases with RAG so that the system can incorporate an up-to-date product catalog and customer data. This configuration improves response

accuracy and relevance by providing context from the company's specific information. Regularly removing outdated product data ensures that the knowledge base remains current.

Learn more about [Knowledge Bases](#).

B

Implement response streaming on the FM.

Incorrect. Response streaming can improve perceived latency. However, response streaming does not meet the requirement to handle variable traffic loads or to maintain accuracy when accessing company data.

Learn more about [response generation](#).

C

Enable prompt caching for frequently asked questions and common inquiry patterns.

Correct. Prompt caching for frequently asked questions and common inquiry patterns can improve response times and reduce the load on an FM. This configuration is especially useful for a customer service system where many queries are likely to be repetitive.

Learn more about [prompt caching](#).

D

Use Amazon Bedrock batch inference to process customer inquiry files.

Incorrect. Batch inference can process many requests asynchronously. However, batch inference is not suitable for real-time customer service interactions. This configuration would introduce unnecessary latency and degrade the customer experience.

Learn more about [batch inference](#).

E

Deploy multiple FMs in parallel in Amazon Bedrock. Use A/B testing to dynamically route customer inquiries based on model performance.

Incorrect. Amazon Bedrock supports access to multiple FMs. A/B testing is an approach that you can use to compare model performance. However, deploying models in parallel and dynamically routing all inquiries does not meet the requirement to handle variable traffic loads while maintaining quality and accuracy.

Learn more about [FMs](#).

63/75

A company is developing a product description generator by using Amazon Bedrock. The generator must provide creative but controlled product descriptions between 50–100 words. The descriptions must maintain consistency with brand guidelines but provide some variation in style. The company needs to optimize the model's output parameters to achieve the desired balance.

Which configuration will meet these requirements?

]

A

Set the temperature to 0.2. Set top-k to 4. Configure strict stop sequences for brand-specific terms.

Incorrect. Temperature controls randomness in token selection. A lower temperature produces a more deterministic output. A temperature of 0.2 combined with a top-k of 4 creates a restrictive sampling strategy. This configuration forces the model to choose from only the four highest-probability tokens at each step. This configuration will provide nearly identical outputs for similar prompts. Strict stop sequences can help maintain brand consistency. However, this overly deterministic approach would produce repetitive, uncreative content. Therefore, this configuration would not provide stylistic variation in product descriptions.

Learn more about [randomness and diversity](#).

Learn more about [length parameters](#).

B

Set the temperature to 0.5. Set top-p to 0.8. Configure length penalties for responses that exceed brand guidelines.

Correct. Temperature controls randomness in token selection. A higher temperature increases variability for creative output. Top-p (nucleus sampling) selects tokens from the most likely subset to balance diversity and coherence. Length penalties control verbosity. A temperature of 0.5 provides creativity while maintaining reasonable control. A top-p of 0.8 ensures that the model considers a range of options and stays within acceptable probability bounds. Length penalties help maintain consistency with the guidelines but permit some creative expression.

Learn more about [randomness and diversity](#).

Learn more about [length parameters](#).

C

Set the temperature to 0.5. Configure response length limits. Disable all diversity parameters.

Incorrect. A mid-range temperature balances determinism and randomness. You can disable the diversity parameters top-p or top-k to apply strict length limits. This configuration will keep outputs concise and consistent. This configuration reduces variability and helps enforce brand rules. However, this configuration limits creativity. To meet the requirements, you must balance consistency with creativity.

Learn more about [randomness and diversity](#).

Learn more about [length parameters](#).

D

Set the temperature to 0.9. Set top-k to 50. Remove all response length limitations.

Incorrect. High temperature and large top-k maximizes randomness and creativity. This configuration removes response limits and allows unconstrained text generation. This configuration provides high creativity. However, this configuration does not provide consistency or control. Without constraints, the model is more prone to hallucinations and off-brand outputs. Therefore, this configuration does not meet the requirement for controlled, guideline-aligned text.

Learn more about [randomness and diversity](#).

Learn more about [length parameters](#).

64/75

A company is developing a customer support AI assistant by using Amazon Bedrock FMs. The AI assistant needs to process complex customer inquiries that require detailed analysis and multiple steps to resolve. Currently, users wait for the complete analysis before seeing any response. The delay negatively impacts user satisfaction. The company wants to optimize the user experience by showing progressive responses while performing the analysis.

Which solution will meet these requirements with the LEAST development effort?

A

Create an Amazon API Gateway WebSocket API with AWS Lambda integration. Use the Amazon Bedrock InvokeModelWithResponseStream API to stream tokens to connected clients. Configure a Lambda function to handle WebSocket connections and message delivery.

Correct. API Gateway WebSocket APIs provide managed, bidirectional communication channels that are suitable to stream data. The `InvokeModelWithResponseStream` API provides token-by-token streaming of model responses. Lambda functions can efficiently process the streams and forward tokens through WebSocket connections to clients. This solution provides real-time streaming with minimal latency and development effort. This solution maintains scalable, persistent connections.

Learn more about [InvokeModelWithResponseStream](#).

Learn more about [how to invoke a model with streaming](#).

Learn more about [how to optimize model inference for latency](#).

Learn more about [WebSocket APIs](#).

B

Create an AWS AppSync real-time API with AWS Lambda resolvers. Configure a GraphQL schema to stream responses. Use the Amazon Bedrock `InvokeModel` API and implement subscription resolvers for client updates.

Incorrect. AWS AppSync is a managed GraphQL service that supports real-time updates through subscriptions. However, AWS AppSync requires more development effort for schema design and resolver implementation. Additionally, managing GraphQL types, mutations, and subscriptions adds development effort. The `InvokeModel` API returns only after the model finishes. Therefore, this solution does not provide incremental tokens to forward to clients. The AWS AppSync subscription would publish a single final message, not progressive updates while the analysis runs.

Learn more about [AWS AppSync and Lambda resolvers](#).

Learn more about [InvokeModel](#).

C

Deploy a REST API in Amazon API Gateway. Configure parallel synchronous `InvokeModel` API calls. Store responses in an Amazon DynamoDB table. Implement client-side polling to check for updates in DynamoDB Streams.

Incorrect. API Gateway REST APIs do not support built-in streaming capabilities. Using DynamoDB to store intermediate results and implementing client-side polling with DynamoDB Streams creates additional latency and development effort. This solution increases API calls and database operations. This solution does not improve user experience because of polling delays.

Learn more about [REST APIs](#).

Learn more about [DynamoDB Streams](#).

D

Implement HTTP APIs with AWS Lambda integration. Configure batch processing of responses by using AWS Step Functions. Store partial results in an Amazon DynamoDB table for client retrieval by using scheduled polling.

Incorrect. HTTP APIs with Step Functions batch processing requires additional development effort. Additionally, this solution does not provide real-time updates. Storing partial results in DynamoDB and implementing scheduled polling creates latency through repeated API calls. This solution does not meet the requirement for progressive responses. This solution requires additional infrastructure management.

Learn more about [HTTP APIs](#).

Learn more about [Step Functions](#).

65/75

A company is using an Amazon Bedrock agent that assists customers. The company must implement comprehensive observability capabilities. The company wants to understand and track the agent's reasoning process in making decisions. The solution must provide detailed visibility into the agent's reasoning process. The solution must provide quick identification of potential hallucinations.

Which solution will meet these requirements?

A

Implement OrchestrationTrace with CustomOrchestrationTrace analysis. Use Amazon CloudWatch metrics for token usage patterns and FM performance monitoring. Validate agent behavior against a golden dataset to detect anomalies.

Incorrect. OrchestrationTrace provides insights into action invocations and knowledge base queries. Using OrchestrationTrace with CustomOrchestrationTrace does not capture critical pre-processing and post-processing validation steps. OrchestrationTrace alone traces only the I/O of the orchestration step. This solution does not provide complete visibility into the entire reasoning process.

Learn more about [OrchestrationTrace](#).

Learn more about [CustomOrchestrationTrace](#).

B

Configure GuardrailTrace with RoutingClassifierTrace. Use ModelInvocationInput analysis to validate the agent's decision boundaries and response patterns. Compare outputs against a golden dataset.

Incorrect. GuardrailTrace and RoutingClassifierTrace can help with I/O assessment and routing decisions. However, GuardrailTrace and RoutingClassifierTrace focus primarily on content filtering and routing rather than comprehensive reasoning analysis. GuardrailTrace is specifically designed for content intervention and policy enforcement rather than full reasoning process validation.

Learn more about [GuardrailTrace](#).

Learn more about [RoutingClassifierTrace](#).

C

Enable PreProcessingTrace, OrchestrationTrace, and PostProcessingTrace components with golden dataset validation and systematic trace analysis.

Correct. This solution provides end-to-end visibility into the agent's reasoning process. Each step in the console or trace in the API includes these three essential components. Together these components provide complete coverage of the agent's processing pipeline. Additionally, the golden dataset validation provides systematic detection of hallucinations and reasoning failures.

Learn more about [PreProcessingTrace](#).

Learn more about [OrchestrationTrace](#).

Learn more about [PostProcessingTrace](#).

D

Combine FailureTrace monitoring with PostProcessingTrace analysis. Implement custom parser modes with override AWS Lambda functions for response validation. Use a golden dataset for periodic checks of output consistently.

Incorrect. FailureTrace and PostProcessingTrace with custom parsers can help identify failures and validate final outputs. However, this solution does not capture critical insights from the pre-processing and orchestration phases. FailureTrace provides information only about step failures. FailureTrace does not provide proactive insight into the agent's reasoning process.

Learn more about [FailureTrace](#).

Learn more about [PostProcessingTrace](#).

A GenAI developer deployed an AI assistant by using an FM in Amazon Bedrock. Users report that when asking similar questions, sometimes the responses are inconsistent. The GenAI developer needs to quantitatively assess the model's sensitivity to slight variations in input questions by using a prompt dataset provided by users.

Which solution will quantitatively evaluate the model's responses across similar input variations?

A

Use an Anthropic Claude model in Amazon Bedrock with a temperature setting of 0. Create an AWS Lambda function that calculates Levenshtein distances between responses for similar prompts.

Incorrect. Setting the temperature parameter to 0 can reduce randomness in the token generation process. However, Levenshtein distance is primarily designed to measure string edit distances, not semantic consistency. This solution focuses on surface-level text differences. Therefore, this solution could miss important semantic variations in the model's responses to similar prompts.

Learn more about [model parameters in Amazon Bedrock](#).

B

Deploy an evaluation pipeline using Amazon Bedrock Knowledge Bases and Amazon Titan Embeddings. Calculate semantic similarity scores between responses for similar prompts. Analyze the distribution of embedding distances to quantify consistency.

Incorrect. Embedding-based similarity analysis can measure semantic relationships. Knowledge bases are designed for information retrieval and augmentation, not model evaluation. This solution can provide some insights into response similarities. However, this solution does not provide the comprehensive evaluation framework that you need for a systematic assessment of model consistency.

Learn more about [Knowledge Bases](#).

C

Create a model evaluation job in Amazon Bedrock using the user-provided prompt dataset. Configure evaluation metrics for response consistency analysis. Measure the statistical variance in model outputs across similar input variations.

Correct. Model evaluation jobs in Amazon Bedrock support custom prompt datasets. Model evaluation jobs can produce computed scores and metrics that help you assess the effectiveness of a model and knowledge base. The robustness metric specifically

assesses the sensitivity of generated responses based on small variations in the input questions.

Learn more about [Amazon Bedrock model evaluation jobs](#).

D

Set up an Amazon Bedrock agent with systematic testing workflows. Use the agent's orchestration capabilities to process similar prompts in parallel. Analyze semantic coherence through custom evaluation metrics.

Incorrect. Agents are designed to orchestrate task execution and workflow automation. Agents can process multiple prompts. However, agents do not provide the specialized metrics and analysis capabilities that you need for a quantitative assessment of model response consistency.

Learn more about [Agents](#).

67/75

A company is implementing a RAG-based knowledge management system. The system will use Amazon Bedrock and Amazon OpenSearch Service. The system will ingest hundreds of new documents into the knowledge base on a daily basis. The system must maintain high accuracy and reliability for content across multiple departments.

A GenAI developer wants to use Amazon Bedrock model evaluation to design a comprehensive evaluation process. The process must evaluate correctness, relevance, formality scale, and company-specific tone and style. The GenAI developer must run the evaluation on a weekly basis. The GenAI developer will create a RAG evaluation with LLM-as-a-judge and select the desired metrics.

Which solution will meet these requirements MOST cost-effectively?

A

Use an industry-standard benchmark dataset. Create custom metrics for formality scale and company-specific tone and style.

Incorrect. An industry-standard benchmark dataset lacks enterprise-specific context. The dataset might not accurately represent real production scenarios. You can use LLM-as-a-judge with custom metrics. However, the dataset would compromise the evaluation results. An industry-standard benchmark dataset will not capture company-specific tone and style.

Learn more about [RAG evaluation custom metrics](#).

B

Create a human-validated evaluation dataset. Create custom metrics for formality scale and company-specific tone and style.

Correct. A human-validated dataset ensures an accurate representation of enterprise-specific use cases, terminology, and content patterns. Using LLM-as-a-judge with custom metrics provides an automated, consistent, and scalable evaluation. You can design custom metrics to assess formality scale and company-specific tone and style with consistent criteria.

Learn more about [how to create a dataset for a RAG evaluation](#).

Learn more about [RAG evaluation metrics](#).

Learn more about [RAG evaluation custom metrics](#).

C

Use an industry-standard benchmark dataset. Create a human-based model evaluation for formality scale and company-specific tone and style.

Incorrect. An industry-standard benchmark dataset lacks enterprise-specific context. The dataset might not accurately represent real production scenarios. An industry-standard benchmark dataset will not capture company-specific tone and style. Human-based model evaluation requires higher costs, longer evaluation cycles, and potential inconsistencies in scoring compared to custom metrics. To meet the requirements, you do not need to manage multiple evaluation types.

Learn more about [human-based model evaluation](#).

D

Create a human-validated evaluation dataset. Create a human-based model evaluation for formality scale and company-specific tone and style.

Incorrect. Using a human-validated dataset is most suitable for this scenario. However, human-based model evaluation is less efficient than custom metrics because of higher operational costs, longer evaluation cycles, and potential scoring inconsistencies. Managing both LLM and human evaluations adds unnecessary complexity to the process.

Learn more about [human-based model evaluation](#)

68/75

A retail company is using Amazon Bedrock to develop a generative AI (GenAI) application that will provide fashion recommendations to customers. The company

wants to evaluate the quality of responses from two different FMs to determine which FM provides better fashion advice. Fashion experts who work for the company must perform the evaluations.

Which combination of steps will meet these requirements to set up an evaluation process? (Select THREE.)

A

Create an Amazon Cognito user pool to manage the fashion expert workforce. Assign the fashion experts to a work team.

Correct. You must manage human evaluators as a work team. You can create a new Amazon Cognito managed work team by using the Amazon Bedrock console. For the fashion experts to evaluate the models, you must organize the fashion experts into a work team.

Learn more about [how to create a workforce](#).

Learn more about [how to create a human-based model evaluation job](#).

B

Create an Amazon SageMaker Ground Truth labeling job. Specify "AWS/Bedrock/Evaluation" as the AwsManagedHumanLoopRequestSource.

Incorrect. Amazon Bedrock uses SageMaker Ground Truth for human workers. However, you do not need to create a labeling job in Ground Truth. Instead, you need to create a human-based evaluation in Amazon Bedrock.

Learn more about [how to create a model evaluation job that uses human workers](#).

C

Use Amazon Comprehend to analyze the sentiment of model responses and automatically score responses based on positive fashion experts' feedback.

Incorrect. The company needs human fashion experts to evaluate the models. The company does not need automated sentiment analysis.

Learn more about [how to create a model evaluation job that uses human workers](#).

D

Configure automatic model evaluation with built-in metrics for accuracy and coherence. Set up Amazon CloudWatch to monitor evaluation results.

Incorrect. You can use metrics, such as accuracy, in human evaluations. However, the metrics would be rated by humans, not automatically calculated.

Learn more about [accuracy](#).

E

Create a human-based evaluation job in Amazon Bedrock with custom metrics including "Style Accuracy".

Correct. You can create custom metrics for human evaluators to use when rating model responses. In this scenario, creating custom metrics that are specific to fashion recommendations would be suitable for the fashion experts to use when evaluating the models.

Learn more about [HumanEvaluationCustomMetric](#).

F

Create a custom prompt dataset with fashion-related queries in JSONL format. Store the dataset in an Amazon S3 bucket. Configure cross-origin resource sharing (CORS) permissions on the bucket.

Correct. Human-based model evaluations require a custom prompt dataset in JSONL format that is stored in an S3 bucket. For jobs that you create through the console, you must update the CORS configuration in the S3 bucket.

Learn more about [how to create a human-based model evaluation job](#)

69/75

A company is evaluating multiple FMs in Amazon Bedrock for an AI-powered customer service conversational assistant. The company requires an assessment of response quality and helpfulness. The company requires comprehensive responsible AI metrics including safety evaluations.

There are thousands of customer service scenarios that require assessment. The evaluation framework must provide human-like judgment capabilities to assess nuanced aspects of conversational responses. Examples of nuanced aspects include contextual appropriateness and tone that traditional automated metrics cannot adequately measure. The company needs to select the appropriate model based on performance analysis with statistical validation of differences between models. The solution must use managed evaluator models that provide human-like judgment and scale across thousands of scenarios.

Which solution will meet these requirements with the LEAST operational overhead?

A

Run Amazon Bedrock batch inference jobs to generate responses from all FMs for the customer service scenarios. Run additional batch inference jobs by using a custom LLM-as-a-judge evaluation on the generated outputs. Deploy AWS Lambda functions with Spearman's rank correlation analysis to compare performance across models and to identify the statistical significance of differences.

Incorrect. You can use Amazon Bedrock batch inference with an LLM-as-a-judge pipeline to provide a nuanced evaluation of responses. However, you must run separate batch jobs for generation and evaluation. Statistical analysis is valid. However, you must manage multiple pipelines and orchestration steps. Therefore, this solution requires additional operational overhead.

Learn more about [Amazon Bedrock batch inference](#).

Learn more about [model evaluation by using LLM-as-a-judge](#).

B

Run Amazon Bedrock batch inference jobs with Amazon Bedrock Guardrails enabled for each FM to generate filtered responses. Monitor InvocationsIntervened metrics using Amazon CloudWatch in the AWS/Bedrock/Guardrails namespace with GuardrailPolicyType and GuardrailContentSource dimensions. Create an AWS Lambda function to perform statistical analysis of intervention patterns and to evaluate the significance of safety performance differences across models.

Incorrect. Guardrails provide safety filters. You can use CloudWatch metrics to monitor interventions. However, this solution focuses solely on safety metrics. This solution does not evaluate response quality, helpfulness, or tone. Statistical analysis of intervention patterns does not provide the human-like judgment that you need to assess nuanced aspects of conversational responses. For example, nuanced aspects include contextual appropriateness and tone. This solution does not meet the requirements for a comprehensive evaluation framework.

Learn more about [Guardrails](#).

C

Configure programmatic evaluation jobs by using the Amazon Bedrock CreateEvaluationJob API with a question-answering task type for each FM. Structure evaluation datasets with question and answer pairs in JSONL format. Perform statistical validation of performance differences across models by using the evaluation metrics and confidence intervals provided in the Amazon S3 output results.

Incorrect. The CreateEvaluationJob API supports programmatic evaluations by using task types such as question-answering. However, the question-answering task type produces standard factual correctness metrics. This approach does not provide the human-like judgment capabilities that you need for nuanced aspects such as tone, contextual appropriateness, or helpfulness. Statistical validation of metrics is helpful. However, this solution does not provide the comprehensive responsible AI and safety evaluations you need to meet the requirements.

Learn more about [model evaluation in Amazon Bedrock](#).

D

Implement an automated evaluation pipeline by using the Amazon Bedrock CreateEvaluationJob API with a consistent evaluator model for all candidate FMs. Use managed evaluator jobs to process scenarios in parallel. Retrieve results from Amazon S3 output locations. Create AWS Lambda functions to perform Spearman's rank correlation analysis between model pairs to identify performance consistency patterns and the statistical significance of differences.

Correct. The CreateEvaluationJob API with a consistent evaluator model provides a mechanism to run managed evaluation jobs that can scale across large datasets. By using a consistent evaluator model, Amazon Bedrock provides human-like judgment on nuanced conversational qualities. The qualities include relevance, tone, factual accuracy, and generated built-in responsible AI metrics with confidence intervals. Lambda can then analyze outputs by using Spearman's rank correlation to statistically compare model performance. This solution uses the Amazon Bedrock managed evaluation framework. Therefore, this solution minimizes operational overhead because you do not need to build a custom pipeline.

Learn more about [model evaluation in Amazon Bedrock](#).

70/75

A company deploys an FM to an Amazon SageMaker AI real-time endpoint. Currently the FM serves production traffic for text generation tasks. The company develops a new version of the model with improved accuracy.

A GenAI developer must evaluate the new model's operational performance metrics under real production traffic conditions before deploying the model to production. The operational performance metrics include latency, error rates, and resource utilization. The GenAI developer needs to ensure that the new model performs well with actual user requests without impacting end users during the evaluation period.

Which solution will meet these requirements with the LEAST operational overhead?

A

Deploy the new model to a separate SageMaker AI endpoint and use custom routing.

Incorrect. You can deploy a new model to a separate endpoint for testing without impacting end users. However, you must build and maintain custom routing logic. Therefore, this solution requires additional operational overhead.

Learn more about [SageMaker AI real-time endpoints](#).

B

Implement A/B testing by configuring production variant weights on the SageMaker AI endpoint.

Incorrect. You can gradually shift traffic between model versions by using A/B testing with production variant weights. However, A/B testing exposes end users to the new model before you can validate the model's operational performance. This solution is more suitable to evaluate business metrics based on user feedback, not to evaluate operational metrics. End users could experience performance issues from the new model. A/B testing is not suitable when you need to evaluate performance without affecting production traffic.

Learn more about [A/B testing](#).

C

Use SageMaker Model Monitor to evaluate the new model's performance.

Incorrect. Model Monitor is designed to detect data drift and quality issues in deployed models. Model Monitor does not compare operational performance metrics between different model versions under real production traffic conditions. For this solution, first you must deploy the model to serve end users. Then, you can evaluate how the new model handles actual production requests. Therefore, this solution does not meet the requirements.

Learn more about [Model Monitor](#).

D

Deploy the new model version to a SageMaker AI shadow test.

Correct. You can use SageMaker AI shadow tests to deploy a new model variant alongside your production variant on the same endpoint. The shadow variant receives a copy of the production traffic but does not return responses to users. You can use shadow tests to compare operational metrics including latency, error rates, and resource utilization without any risk to end users. For this solution, you do not need to set up or manage separate infrastructure. SageMaker AI automatically handles the traffic replication and metrics collection. After the test period, you can promote the shadow variant to production if the new model performs well.

Learn more about [shadow tests](#).

71/75

A GenAI developer wants to evaluate FMs by using the automatic model evaluation feature in Amazon Bedrock. The GenAI developer creates a comprehensive custom prompt dataset that contains 5,000 curated prompts. The prompts cover various business scenarios for a text classification task.

The GenAI developer uploads the dataset to Amazon S3 in JSONL format. The GenAI developer attempts to create an evaluation job. However, the evaluation job fails to start. The error indicates an issue with the dataset configuration.

Which approach will resolve this issue?

A

Compress the dataset file by using gzip compression. Re-upload the dataset to Amazon S3.

Incorrect. File compression does not bypass the fundamental quota of 1,000 prompts for each dataset in an evaluation job. The issue is the number of prompts, not the file size. Amazon Bedrock will still process the same number of prompts regardless of compression.

B

Convert the dataset from JSONL format to CSV format. Re-upload the dataset to Amazon S3.

Incorrect. Amazon Bedrock model evaluation requires custom prompt datasets to be in JSONL format with the ".jsonl" file extension. Converting the dataset to CSV will not resolve the issue. Instead, this approach would create a format incompatibility.

C

Enable versioning on the S3 bucket. Update the CORS configuration for console access.

Incorrect. CORS configuration is a requirement for console-created jobs. However, this approach addresses browser access permissions, not dataset size limitations. S3 Versioning does not relate to the evaluation job failure.

D

Split the dataset into multiple smaller datasets with a maximum of 1,000 prompts each. Run separate evaluation jobs.

Correct. Amazon Bedrock automatic model evaluation jobs have a quota of 1,000 prompts for each dataset. The 5,000-prompt dataset exceeds the quota and causes the job to fail. You can split the prompts into smaller datasets such as five datasets of 1,000 prompts each. Then, you can run separate evaluation jobs to resolve the issue.

Learn more about [model evaluation prompt datasets](#).

72/75

A company that provides help desk software uses Amazon Bedrock to automate customer support ticket triage. Following AWS best practices, the system uses a well-structured prompt template that includes defined categories, examples, and validation requirements. The architecture consists of the following elements:

- Amazon API Gateway for ticket ingestion
- An AWS Lambda function for ticket processing and model interaction
- An Amazon Bedrock InvokeModel API for classification
- Response validation before downstream processing

The company notices intermittent issues where some tickets receive unexpected classifications. The company needs to identify the root cause of these anomalies.

Which approach will meet these requirements?

A

Use Amazon CloudWatch Logs Insights to analyze Lambda function logs for error patterns in ticket processing and model responses.

Incorrect. CloudWatch Logs Insights is a query tool that you can use to search and analyze log data that is stored in CloudWatch Logs. You can use CloudWatch Logs Insights to analyze function-level errors and exceptions. However, CloudWatch Logs Insights would not provide detailed visibility into the specific model interactions that cause intermittent classification anomalies. To troubleshoot occasional unexpected classifications, direct inspection of model interactions would be more effective.

Learn more about [CloudWatch Logs Insights](#).

B

Use AWS X-Ray traces to identify if there are timeout or throttling issues between the Lambda function and the Amazon Bedrock API calls.

Incorrect. X-Ray is a distributed tracing service that helps analyze and debug application performance and service interactions. X-Ray traces can help identify performance issues, latency, or API throttling problems in service interactions. However, the architecture already implements best practices and validation. Therefore, the root

cause of the intermittent classification issues relates to specific model interactions, not service integration problems.

Learn more about [X-Ray](#).

C

Enable Amazon Bedrock model invocation logging to inspect the raw prompts being sent and the responses received from the FM.

Correct. Model invocation logging provides detailed logs of model interactions, including prompt content and responses. You can use this approach for direct analysis of cases where unexpected classifications occur. This approach helps identify patterns or specific conditions that led to anomalies despite the well-structured prompts and validation. This approach provides the most direct insight into intermittent model behavior issues.

Learn more about [model invocation logging](#).

D

Create a custom Amazon CloudWatch metric that tracks the distribution of classified categories. Set up an alarm for unexpected spikes in specific labels or null values.

Incorrect. CloudWatch metrics are data points that provide information about the performance of systems and applications. Alarms can monitor metrics and take actions. Monitoring classification distribution through CloudWatch metrics can help detect anomalies. However, this approach would identify only when issues occur, not why the issues occur. The system already includes response validation and follows best practices. Therefore, understanding the specific model interactions in anomalous cases would be more valuable for root cause analysis.

Learn more about [CloudWatch custom metrics](#).

73/75

A GenAI developer is developing a document summarization system by using Amazon Bedrock Knowledge Bases. Users upload large technical research papers that the system must summarize accurately.

The GenAI developer receives reports that generated summaries frequently omit critical sections from longer documents, even when the full source text was successfully uploaded and tokenized. Logs show no API errors or truncation messages. However, summaries frequently miss information near the middle or end of documents.

Which solution will resolve this issue?

A

Configure semantic chunking in Amazon Bedrock. Submit each segment to Amazon Bedrock for summarization. Use prompt chaining to combine the partial summaries into a final consolidated summary.

Correct. Amazon Bedrock supports semantic chunking. Semantic chunking resolves the issue of missing information from longer documents by intelligently segmenting the text into coherent parts. Then, the model can summarize the segments individually. You can use prompt chaining for tasks that exceed a single model call's context or reasoning depth. In prompt chaining, outputs from one step (summaries of individual chunks) pass as structured input into a follow-up prompt (a final summarization request). This solution prevents context window overflow and ensures complete input coverage.

Learn more about [semantic chunking in Amazon Bedrock](#).

Learn more about [prompt chaining by using Amazon Bedrock](#).

Learn more about [the prompt chaining workflow](#).

B

Select an FM with a larger context window. Allow the FM to process full-length documents in a single inference. Apply text compression and prompt shortening strategies when necessary.

Incorrect. You can select an FM with a larger context window to process longer documents. However, a larger context window would not impact the retrieval of the chunks. Client-side compression could remove important details and cause the loss of critical context. Therefore, this solution will not resolve the issue of summaries that omit critical sections from longer documents.

Learn more about [model parameters in Amazon Bedrock](#).

Learn more about [context windows](#).

C

Configure standard chunking in Amazon Bedrock. Split the document into evenly sized segments. Summarize each segment independently before combining the results into a final consolidated summary.

Incorrect. Standard chunking in Amazon Bedrock uses fixed-size and default chunking. Standard chunking can break up sentences or separate related ideas. This solution can lead to summaries that omit details or lack cohesion, especially for documents where context spans multiple segments. This solution can process long inputs. However, semantic chunking is more suitable to preserve accuracy by aligning chunks within a

document's natural structure. Summarizing chunks individually without a synthesis step can produce disjointed results.

Learn more about [chunking strategies in Amazon Bedrock](#).

D

Retrieve a larger number of document chunks from an Amazon Bedrock knowledge base. Summarize each retrieved chunk independently. Return the combined results into a final summary.

Incorrect. Retrieving more chunks increases coverage. However, this solution does not ensure that the summarization process will preserve context across the entire document. This solution might produce summaries that are fragmented or omit details that span multiple chunks. You need a strategy to merge summaries coherently so that you maintain important relationships between sections. Examples of strategies that merge summaries coherently are semantic chunking and prompt chaining.

Learn more about [chunk retrieval in Amazon Bedrock](#)

74/75

A large social media company runs multiple RAG pipelines across different applications. Each pipeline uses an FM embedding endpoint deployed on an Amazon SageMaker AI real-time endpoint. A GenAI developer uses the endpoint to generate embeddings. The GenAI developer stores the embeddings in multiple vector databases. The production applications use the vector databases to drive document retrieval for downstream generation and to provide users with content.

Recently, multiple users report that the response relevance has been poor. New writing styles are appearing more frequently. The GenAI developer wants to determine if the input drift is degrading embedding quality. The GenAI developer must prepare the model for re-training without disrupting production traffic.

Which combination of steps will meet these requirements? (Select TWO.)

A

Create a new SageMaker AI endpoint with Data Capture enabled. Shift all production traffic to the new endpoint. Delete the previous endpoint. Parse captured data in SageMaker Model Monitor to investigate.

Incorrect. Data Capture is a SageMaker AI feature that you can use to log actual production interactions for real-time or batch-transform endpoints. Data Capture reliably logs both requests and responses to Amazon S3. Data Capture ensures that you can

use the responses for further drift detection and analysis. However, this step does not meet the requirement to avoid disrupting production traffic. Re-creating the endpoint will disrupt traffic.

Learn more about [Data Capture](#).

Learn more about [how to monitor data quality](#).

B

Schedule SageMaker Model Monitor jobs by using SageMaker Clarify to improve visibility into potential bias. Store the final violation reports in Amazon S3.

Incorrect. You can use Clarify to detect and mitigate bias in models. Clarify provides model explainability. Clarify integrates with Model Monitor to track model bias and feature attribution drift. This step improves visibility into potential bias that might emerge over time. However, this step does not address semantic embedding drift in production inputs.

Learn more about [Clarify](#).

C

Schedule SageMaker Model Monitor with a custom Amazon ECR image. Compute all the drift metrics between the endpoint captured data and the baseline training dataset used by the model. Store the final violation reports in Amazon S3.

Correct. Model Monitor is a fully managed service that provides continuous monitoring of production ML models. Model Monitor runs scheduled jobs by using custom images. This approach can compute embedding-level drift metrics by comparing captured inference data (the data capture logs) against a baseline dataset. Storing the final violation reports in Amazon S3 ensures that you have an auditable record of drift events.

Learn more about [Model Monitor](#).

Learn more about [how to detect natural language processing data drift by using Model Monitor](#).

D

Create a new SageMaker AI endpoint configuration based on the production variant with Data Capture enabled. Use the UpdateEndpoint API to shift endpoint traffic to the new endpoint with the updated configuration. Parse captured data in SageMaker Model Monitor to investigate.

Correct. Data Capture is a SageMaker AI feature that you can use to log actual production interactions on real-time or batch-transform endpoints. Data Capture reliably logs requests and responses to Amazon S3. Data Capture ensures that you can use responses for further drift detection and analysis. The UpdateEndpoint API call shifts SageMaker AI endpoint traffic to the new instances with the updated endpoint configuration. Then, the call deletes the old instances by using the previous configuration. This step maintains availability and creates an intrinsic blue/green deployment.

Learn more about [Data Capture](#).

Learn more about [how to monitor data quality](#).

Learn more about [UpdateEndpoint](#).

E

Change all RAG pipelines to use an Amazon Bedrock Titan Embeddings model endpoint. Enable model invocation logging. Select embedding as the data type to capture. Set up logs to publish to Amazon S3.

Incorrect. You can use the Amazon Bedrock invocation logging feature to collect the entire request data, responses, and any metadata associated with all calls performed to that endpoint. However, Titan Embeddings is not the FM embedding endpoint that the company owns. Additionally, switching all RAG pipelines to use Amazon Bedrock will cause some downtime.

Learn more about [Amazon Titan Embedding models](#).

Learn more about [Amazon Bedrock invocation logging](#).

75/75

A GenAI developer is troubleshooting performance issues in a production RAG application. The application is built on Amazon Bedrock. The application uses Amazon OpenSearch Service for vector storage. Users report inconsistent response times. Some queries are taking significantly longer than others. The GenAI developer must implement a monitoring solution that provides comprehensive diagnostic information to identify the root cause of the issue.

Which solution will meet these requirements with the LEAST operational overhead?

A

Create custom Amazon CloudWatch metrics that combine OpenSearch Service vector search latency and Amazon Bedrock token usage patterns.

Set up composite alarms that correlate high latency with vector similarity thresholds and token consumption rates.

Incorrect. You can use CloudWatch custom metrics to track application-specific data points. Multiple conditions can trigger composite alarms. However, creating custom metrics and complex alarm conditions requires additional operational overhead. Correlating vector similarity thresholds with token consumption might not effectively identify the root cause of inconsistent response times. Other factors in the RAG pipeline could cause the performance issues.

Learn more about [CloudWatch composite alarms](#).

Learn more about [CloudWatch custom metrics](#).

B

Set up detailed monitoring in OpenSearch Service and Amazon Bedrock. Create Amazon CloudWatch metric math expressions to analyze the correlation between vector search performance and model inference times. Set up anomaly detection on the combined metrics.

Incorrect. Metric math provides complex calculations on metrics. Anomaly detection uses ML to identify unusual patterns. Configuring detailed monitoring and using metric math expressions can provide insights. However, this approach could overcomplicate the analysis. Analyzing correlations between vector search and model inference times through metric math will not clearly identify whether performance issues originate in vector retrieval or model processing. Anomaly detection could generate false positives because of the complexity of RAG operations.

Learn more about [anomaly detection](#).

Learn more about [math expressions](#).

C

Implement distributed tracing through AWS X-Ray that focuses on OpenSearch Service vector query latency and Amazon Bedrock model response times. Create custom subsegments for vector similarity calculations and token processing to identify specific performance bottlenecks.

Incorrect. X-Ray is a distributed tracing system that helps analyze and debug production-distributed applications. X-Ray can help visualize request flow. However, custom subsegments for vector operations and token processing require additional operational overhead. Custom subsegments provide detailed tracing of application-specific operations. However, this level of granularity requires additional operational overhead to identify RAG pipeline bottlenecks.

Learn more about [X-Ray custom subsegments](#).

D

Create a custom Amazon CloudWatch dashboard that combines context retrieval latency metrics with OpenSearch Service operation counts. Analyze Amazon Bedrock invocation logs to identify which knowledge base queries are experiencing degraded performance.

Correct. CloudWatch dashboards provide a customizable view of metrics and alarms. CloudWatch dashboards provide a visual correlation of data from multiple sources. You can combine context-retrieval latency metrics from OpenSearch with operation counts to find a direct correlation between vector search performance and overall response times. You can analyze Amazon Bedrock invocation logs to help identify specific knowledge base queries that are causing issues. Amazon Bedrock invocation logs contain detailed information about model interactions. This solution uses built-in metrics and logs with the least operational overhead.

Learn more about [CloudWatch custom dashboards](#).